

RESUME MATCHER

中文学习教程

合订本 · 共 10 章

生成于 2026-06-03

目录

01	01-项目总览
02	02-环境搭建
03	03-后端架构
04	04-后端核心模块
05	05-AI集成与提示词
06	06-前端架构
07	07-前端数据层与Hooks
08	08-前端核心组件
09	09-前端特性与流程
10	10-部署与扩展

01 — 项目总览

本章用 10 分钟带你看懂 Resume Matcher：它是什么、解决什么问题、用什么技术、各部分怎么协作。

一、项目定位

Resume Matcher 是一个开源的 **AI 简历定制平台**，核心场景：

用户上传「主简历（Master Resume）」→ 粘贴「职位描述（JD）」→ AI 分析差距 → 生成定制化简历建议 → 用户编辑 → 导出 PDF + 求职信。

与之类似的工具有 Resume Worded、Jobscan、Teal 等，但 Resume Matcher 的差异化是 **本地可跑**（用 Ollama）+ **完全开源** + **支持多家 LLM**。

参考：[README.zh-CN.md:11](#)

二、核心能力一览

能力	描述	入口页面
主简历管理	一份完整简历作为「单一事实源」	apps/frontend/app/(default)/dashboard
JD 匹配	AI 分析 JD 与简历差距，输出定制建议	components/tailor/
简历改进	量化、可操作的简历内容改写	components/builder/
AI 富集	多轮对话式补全简历缺失信息	components/enrichment/
求职信生成	基于简历 + JD 生成求职信	apps/backend/app/services/cover_letter.py
PDF 导出	Playwright + Chromium 渲染多种模板	apps/backend/app/pdf.py
国际化	界面支持 5 种语言	apps/frontend/messages/

参考：[README.zh-CN.md:101-148](#)

三、用户 workflow

整个产品的用户旅程（参考 [docs/agent/architecture/frontend-workflow.md](#)）：

- Settings: 配置 AI 提供商 (Ollama / OpenAI / Anthropic / Gemini ...)
- Upload: 上传主简历 (PDF / DOCX)
- Tailor: 粘贴 JD → AI 输出改进建议
- Builder: 增删章节、拖拽排序、切换模板
- Enrichment: AI 多轮补全薄弱章节
- Cover Letter: 生成求职信
- Export: 选模板 → 下载 PDF

四、技术栈全景

后端 — apps/backend/

技术	用途	关键文件
FastAPI	HTTP 框架，自动 OpenAPI 文档	app/main.py
Python 3.13+	现代类型系统、模式匹配	pyproject.toml
LiteLLM	统一多 LLM provider 接口	app/llm.py
TinyDB	JSON 文件型数据库（无外部依赖）	app/database.py
Pydantic v2	数据校验与序列化	app/schemas/
Playwright	无头 Chromium 渲染 PDF	app/pdf.py
uv	极速 Python 依赖管理	pyproject.toml

前端 — apps/frontend/

技术	用途	关键文件
Next.js 16	App Router + Turbopack	next.config.ts
React 19	Server Components + Actions	app/
TypeScript 5	类型安全	tsconfig.json
Tailwind CSS v4	原子化样式	postcss.config.mjs
Tiptap	富文本编辑（简历内容）	components/builder/
dnd-kit	拖拽排序（章节调整）	package.json
Lucide Icons	图标库	package.json
Vitest	单元测试	vitest.config.ts
Swiss Design	设计系统（硬边、衬线、橙红强调）	docs/portable/swiss-design-system/

参考：[apps/frontend/package.json:1-40](#)

五、目录结构速览

```
Resume-Matcher/
├── apps/
│   ├── backend/                # FastAPI + Python
│   │   ├── app/
│   │   │   ├── main.py        # 入口
│   │   │   ├── config.py      # 环境配置
│   │   │   ├── database.py    # TinyDB
│   │   │   ├── llm.py         # LiteLLM
│   │   │   ├── pdf.py         # PDF 渲染
│   │   │   ├── routers/       # 5 个路由模块
│   │   │   ├── services/      # 4 个业务服务
│   │   │   ├── schemas/       # Pydantic 模型
│   │   │   └── prompts/       # LLM 提示词
│   │   └── data/              # 持久化数据
│   └── frontend/              # Next.js + React
│       ├── app/               # 页面（路由）
│       ├── components/        # 10 个功能子目录
│       ├── lib/               # API / config / context / i18n
│       ├── hooks/             # 3 个核心 hook
│       ├── messages/          # 5 国语言
│       └── i18n/              # 国际化配置
├── docker/                   # Docker 构建
├── docs/
│   ├── agent/                 # 给 AI 看的项目文档
│   └── portable/              # 可移植的设计/性能 pack
├── Dockerfile
└── docker-compose.yml
```

参考：[.claude/CLAUDE.md:48-90](#)

六、AI 集成亮点

Resume Matcher 通过 [LiteLLM](#) 抽象了 6 家 LLM provider:

Provider	用途	配置方式
Ollama	本地免费	<code>OPENAI_BASE_URL=http://localhost:11434/v1</code>
OpenAI	GPT-4o / GPT-4o-mini	<code>OPENAI_API_KEY=sk-...</code>
Anthropic	Claude 3.5 Sonnet	<code>ANTHROPIC_API_KEY=sk-ant-...</code>
Google Gemini	Gemini 1.5 Flash/Pro	<code>GEMINI_API_KEY=...</code>
OpenRouter	多模型聚合	<code>OPENROUTER_API_KEY=...</code>
DeepSeek	DeepSeek Chat	<code>DEEPSEEK_API_KEY=...</code>

参考：[README.zh-CN.md:200-212](#)

七、设计哲学 — Swiss International Style

整个 UI 严格遵循 [瑞士国际主义平面设计](#)（参考 [docs/portable/swiss-design-system/README.md](#)）：

- 画布: `#F0F0E8`（米白）
- 墨色: `#000000`（纯黑文字）
- 超链接蓝: `#1D4ED8`
- 信号绿: `#15803D`（成功）
- 警告橙: `#F97316`
- 错误红: `#DC2626`
- 标题: `font-serif`（衬线）
- 正文: `font-sans`（无衬线）
- 元数据: `font-mono`（等宽）
- 边框: `rounded-none` 硬边 + 1px 黑色 + 硬阴影

参考：[.claude/CLAUDE.md:108-117](#)

八、关键非协商规则（来自项目 CLAUDE.md）

1. 前端 UI 变更必须遵循 Swiss 设计系统
2. 所有 Python 函数必须有类型提示
3. 提交前端代码前必须运行 `npm run lint` 和 `npm run format`
4. 详细错误日志在服务端，客户端只返回通用消息
5. **不修改** `.github/workflows/` 文件（除非显式要求）

参考：[.claude/CLAUDE.md:17-22](#)

九、本章小结

你现在应该了解：

- Resume Matcher 是什么、解决什么问题
- 7 大核心能力
- 7 步用户工作流
- 后端 / 前端各自使用的技术
- 项目目录结构
- 6 个 LLM provider 集成
- Swiss 设计系统基础

十、下一步

- 想跑起来？→ [02-环境搭建](#)
- 想看后端代码？→ [03-后端架构](#)
- 想看前端代码？→ [06-前端架构](#)

02 — 环境搭建

本章手把手带你把 Resume Matcher 在本地跑起来，覆盖 uv 依赖、Node 模块、AI provider 配置、Docker 三种方式。

一、前置条件

工具	最低版本	检查命令	安装
Python	3.13+	<code>python --version</code>	python.org
Node.js	22+	<code>node --version</code>	nodejs.org
npm	10+	<code>npm --version</code>	随 Node 一起
uv	最新	<code>uv --version</code>	astral.sh/uv
Git	任意	<code>git --version</code>	git-scm.com

参考：[SETUP.zh-CN.md:36-40](#)

安装 uv（Python 包管理器）

项目使用 `uv`（Astral 出品的 Rust 写成的极速 pip 替代品）：

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# 或者用 pip
pip install uv
```

参考：[SETUP.zh-CN.md:44-55](#)

二、快速开始（最快 5 分钟）

如果你只关心能不能跑：

```
# 1. 克隆
git clone https://github.com/srbhr/Resume-Matcher.git
cd Resume-Matcher

# 2. 后端（终端 1）
cd apps/backend
cp .env.example .env          # 必须复制，否则 LLM 调用会失败
uv sync                      # 安装 Python 依赖（首次约 30s）
uv run app                    # 启动 FastAPI，默认 :8000

# 3. 前端（终端 2）
cd apps/frontend
npm install                  # 安装 Node 依赖（首次约 1-2 min）
npm run dev                  # 启动 Next.js，默认 :3000
```

打开 <http://localhost:3000> 即可。但你还需要在 Settings 里配置 AI provider 才能让 AI 工作。

参考：[SETUP.zh-CN.md:62-83](#)

三、详细步骤

步骤 1：克隆仓库

```
git clone https://github.com/srbhr/Resume-Matcher.git
cd Resume-Matcher
```

步骤 2：后端配置

2.1 进入后端目录

```
cd apps/backend
```

2.2 创建环境变量文件

```
cp .env.example .env
```

这一步不可省略！ 没有 `.env` 文件，LiteLLM 不知道用哪个 provider。

2.3 编辑 `.env`

最少需要配置一个 provider（详见下节「AI provider 配置」）。

步骤 3：前端配置

```
cd apps/frontend
npm install
npm run dev
```

`npm install` 会读取 `package.json` 安装所有依赖（参考 [apps/frontend/package.json:1-40](#)）。

`npm run dev` 实际执行 `next dev --turbo`（参考 [package.json:6](#)），用 Turbopack 启动开发服务器。

四、配置 AI Provider

选项 A：本地免费 — Ollama（推荐新手）

1. 安装 Ollama: ollama.com
2. 拉取一个模型（首次约 5GB）：

```
ollama pull llama3.2
ollama run llama3.2 # 测试一下
```

3. 启动 Ollama 服务（默认监听 `11434`）
4. 配置 `.env`：

```
LLM_PROVIDER=ollama
LLM_MODEL=llama3.2
LLM_BASE_URL=http://localhost:11434
```

或者用 OpenAI 兼容模式：

```
LLM_PROVIDER=openai
LLM_MODEL=llama3.2
OPENAI_API_KEY=ollama
OPENAI_BASE_URL=http://localhost:11434/v1
```

参考： [SETUP.zh-CN.md:160-200](#)

选项 B: 云端 OpenAI

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o-mini
OPENAI_API_KEY=sk-...
OPENAI_BASE_URL=https://api.openai.com/v1
```

选项 C: Anthropic Claude

```
LLM_PROVIDER=anthropic
LLM_MODEL=claude-3-5-sonnet-20241022
ANTHROPIC_API_KEY=sk-ant-...
```

选项 D: Google Gemini

```
LLM_PROVIDER=gemini
LLM_MODEL=gemini-1.5-flash
GEMINI_API_KEY=...
```

选项 E: OpenRouter / DeepSeek

类似配置，详细参数参考 [SETUP.zh-CN.md](#)。

五、Docker 部署

方式 1: 拉取官方镜像

```
docker pull srbhr/resume-matcher:latest
docker run -p 3000:3000 -p 8000:8000 srbhr/resume-matcher:latest
```

参考: [README.zh-CN.md:215-220](#)

方式 2: 本地 docker-compose

```
# 在仓库根目录
docker-compose up
```

`docker-compose.yml` 会同时启动 backend 和 frontend，共享网络。

Docker 中用 Ollama

如果 Ollama 跑在宿主机，Docker 容器内要访问它：

```
LLM_BASE_URL=http://host.docker.internal:11434
```

不要用 `localhost`，容器内 `localhost` 指向容器自己。

参考: [README.zh-CN.md:222](#)

六、常用命令速查

后端

命令	作用
<code>uv sync</code>	安装 / 同步 Python 依赖
<code>uv run app</code>	启动 FastAPI（默认 :8000）
<code>uv run uvicorn app.main:app --reload</code>	启动并热重载
<code>uv run pytest</code>	运行测试（如有）
<code>uv add <pkg></code>	添加新依赖

参考：[.claude/CLAUDE.md:30-35](#)

前端

命令	作用
<code>npm install</code>	安装 Node 依赖
<code>npm run dev</code>	启动开发服务器（:3000，Turbopack）
<code>npm run build</code>	生产构建
<code>npm run start</code>	启动生产服务器
<code>npm run lint</code>	ESLint 检查
<code>npm run format</code>	Prettier 格式化
<code>npm run test</code>	Vitest 单元测试

参考：[apps/frontend/package.json:5-12](#)

七、访问应用

URL	用途
<code>http://localhost:3000</code>	前端主入口
<code>http://localhost:8000/docs</code>	FastAPI 自动生成的 Swagger UI
<code>http://localhost:8000/redoc</code>	ReDoc 风格的 API 文档
<code>http://localhost:8000/api/health</code>	健康检查端点

参考：[.claude/CLAUDE.md:38-40](#)

八、首次使用流程

1. 打开 `http://localhost:3000`
2. 进入 **Settings** 页面
3. 选择 AI provider（如果 `.env` 没配好，前端会读取空配置）
4. 上传主简历（PDF / DOCX）
5. 进入 **Tailor**，粘贴 JD，点击「生成建议」
6. 在 **Builder** 编辑章节、选择模板
7. 导出 PDF

九、故障排查

后端启动报错：ModuleNotFoundError

```
# 解决：重新同步依赖
cd apps/backend
uv sync
```

前端 npm install 慢 / 失败

```
# 换镜像源
npm config set registry https://registry.npmirror.com
npm install
```

LLM 调用失败：401 Unauthorized

- 检查 `.env` 里的 `*_API_KEY` 是否正确
- 检查 `LLM_PROVIDER` 与 key 是否匹配

Ollama 连接不上

```
# 确认 Ollama 在运行
curl http://localhost:11434/api/tags

# Docker 场景：用 host.docker.internal
LLM_BASE_URL=http://host.docker.internal:11434
```

PDF 导出失败：Playwright 报错

```
# 安装 Chromium
cd apps/backend
uv run playwright install chromium
```

参考：[apps/backend/app/pdf.py](#) 中 Playwright 初始化逻辑。

十、验证安装是否成功

```
# 后端健康检查
curl http://localhost:8000/api/health
# 期望返回: {"status":"ok"}

# 前端检查
curl -I http://localhost:3000
# 期望返回: 200 OK
```

十一、下一步

- 跑起来了，想看后端怎么实现的？ → [03-后端架构](#)
- 想了解 AI 是怎么集成的？ → [05-AI集成与提示词](#)
- 准备生产部署？ → [10-部署与扩展](#)

03 · Resume Matcher 后端架构总览

适用版本：`apps/backend/pyproject.toml:3` (`version = "1.2.0"`)。本文面向学习 FastAPI 项目的中级开发者。

一、目标与定位

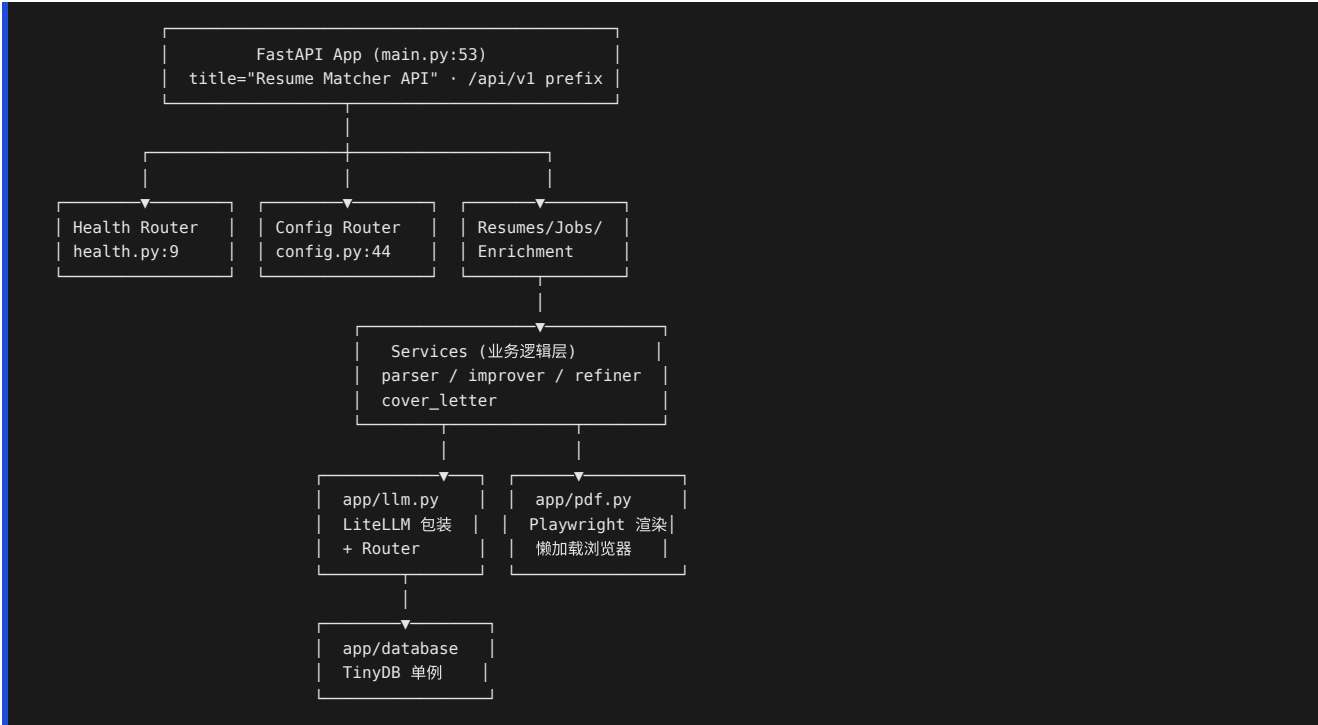
Resume Matcher 后端是一个**本地优先 (local-first)** 的 FastAPI 服务，核心职责是把用户的简历与岗位描述 (JD) 交给 LLM 进行"定向改写"，并把改写结果以结构化 JSON、Markdown 文本、PDF 三种形式返回。整个后端不依赖外部数据库，所有持久化数据存放在 `apps/backend/data/` 目录下的 JSON 文件里，可一键重置。

主要技术栈 (见 `apps/backend/pyproject.toml:6-19`)：

组件	选型	作用
Web 框架	<code>fastapi==0.128.4</code>	路由、参数校验、OpenAPI 文档
ASGI 服务器	<code>uvicorn==0.40.0</code>	启动 HTTP 服务
配置	<code>pydantic==2.12.5</code> + <code>pydantic-settings==2.12.0</code>	环境变量与 <code>.env</code> 解析
数据库	<code>tinydb==4.8.2</code>	JSON 文件存储
AI 抽象层	<code>litellm==1.86.2</code>	多 Provider LLM 统一接口
文档解析	<code>markdown[docx]==0.1.4</code>	PDF/DOCX → Markdown
文档输出	<code>pdfminer.six==20260107</code>	PDF 文本抽取
浏览器	<code>playwright==1.58.0</code>	无头 Chromium 渲染 PDF
依赖管理	<code>uv</code>	Python 包与虚拟环境

二、整体架构

后端由 **入口 / 配置 / 持久化 / LLM 抽象 / PDF 渲染 / 路由 / 服务 / 提示词 / Schema** 九个层次构成。下图展示了请求在系统中的流向。



2.1 模块职责一览

模块路径	行数（参考）	职责
<code>apps/backend/app/main.py</code>	100	入口、生命周期、CORS、路由挂载
<code>apps/backend/app/config.py</code>	226	环境变量、 <code>.env</code> 、API key 持久化
<code>apps/backend/app/database.py</code>	297	TinyDB 封装、Master 简历锁
<code>apps/backend/app/llm.py</code>	1157	LiteLLM 包装、Router、JSON 解析、超时
<code>apps/backend/app/pdf.py</code>	311	Playwright 浏览器、PDF 渲染
<code>apps/backend/app/routers/*.py</code>	3200+	5 个 HTTP 路由模块
<code>apps/backend/app/services/*.py</code>	2270+	业务逻辑（解析、改写、精修、求职信）
<code>apps/backend/app/schemas/*.py</code>	998+	Pydantic 模型
<code>apps/backend/app/prompts/*.py</code>	961	LLM 提示词模板

三、应用入口与生命周期

入口文件 `apps/backend/app/main.py` 承担三件事：配置日志、构造 `FastAPI` 实例、在 `lifespan` 钩子里初始化/释放资源。

3.1 日志配置

```
# apps/backend/app/main.py:24-30
def _configure_application_logging() -> None:
    """Set application log level from configuration."""
    numeric_level = getattr(logging, settings.log_level, logging.INFO)
    logging.getLogger("app").setLevel(numeric_level)

_configure_application_logging()
```

日志级别由 `settings.log_level`（默认 `INFO`）控制；LiteLLM 自身的日志则在 `apps/backend/app/llm.py:19-26` 单独设置。

3.2 lifespan

`main.py:33-50` 使用 `@asynccontextmanager` 实现 FastAPI 的现代 `lifespan` 协议：

- **启动**：确保 `settings.data_dir` 存在（`main.py:37`）。注意：PDF 渲染器采用**懒加载**，首次调用 `render_resume_pdf` 才会启动 Chromium，注释行 `main.py:39` 明确写出了这一点。
- **关闭**：分两个 `try/except` 关闭 PDF 渲染器与数据库（`main.py:42-50`），保证任何一个失败都不会影响另一个的清理。

3.3 CORS 与路由挂载

```
# apps/backend/app/main.py:61-74
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.effective_cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(health_router, prefix="/api/v1")
app.include_router(config_router, prefix="/api/v1")
app.include_router(resumes_router, prefix="/api/v1")
app.include_router(jobs_router, prefix="/api/v1")
app.include_router(enrichment_router, prefix="/api/v1")
```

`effective_cors_origins`（`config.py:193-200`）会把 `cors_origins` 与 `frontend_base_url` 合并，避免开发期忘记把 `http://localhost:3000` 加进白名单。所有路由统一以 `/api/v1` 为前缀。

3.4 console_script

`pyproject.toml:21-22` 注册了 `app = "app.main:main"`，因此 `uv run app` 即可启动服务（实际执行 `main.py:87-96` 的 `uvicorn.run`）。

四、配置与运行时参数

4.1 配置加载

apps/backend/app/config.py 实现了"环境变量 → JSON 配置文件"的二级回退。

```
# apps/backend/app/config.py:115-122
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )
```

主要配置项见 config.py:125-213，下表给出关键字段（除特殊注明，列名同时是环境变量名）：

字段	类型	默认值	用途
llm_provider	Literal	openai	支持 openai / openai_compatible / anthropic / openrouter / gemini / deepseek / groq / ollama
llm_model	str	gpt-5-nano-2025-08-07	模型名
llm_api_key	str	""	兜底 API key，UI 设置会优先
llm_api_base	str None	None	自定义 Base URL（Ollama、聚合代理）
log_llm	Literal	WARNING	LiteLLM 日志级别
host	str	0.0.0.0	uvicorn 监听地址
port	int	8000	uvicorn 监听端口
reload	bool	False	开发期代码热重载
frontend_base_url	str	http://localhost:3000	PDF 渲染时打开的 Next.js 页面
reasoning_effort	Literal None	None	推理模型参数 (minimal / low / medium / high)
cors_origins	list[str]	[http://localhost:3000, http://127.0.0.1:3000]	CORS 白名单
data_dir	Path	<repo>/apps/backend/data	数据库与上传文件目录

验证器 config.py:140-185 会把空字符串、未知级别规整成默认值，避免 INFO 写成 info 时崩溃。

4.2 API Key 优先级链

config.py:41-112 与 llm.py:323-349 共同实现了"API Key 单一真源"：

```
Settings.llm_api_key (环境变量 / .env)
└ 若为空
config.json 顶层 api_key
└ 若为空
config.json api_keys[provider] (多 provider 形态)
└ 若为空
settings.llm_api_key ← 关键: openai_compatible / ollama 会跳过这一步
```

openai_compatible 与 ollama 被刻意排除在"env 兜底"之外（llm.py:318-320），以免付费的 OPENAI_API_KEY 泄漏到本地 LLM。

五、数据库层

5.1 TinyDB 表结构

apps/backend/app/database.py:36-49 暴露三张表：

表名	关键字段	说明
resumes	resume_id、content、content_type、 is_master、parent_id、processed_data、 processing_status、cover_letter、 outreach_message、title、 original_markdown、created_at、updated_at	上传/生成的简历
jobs	job_id、content、resume_id、created_at	岗位描述
improvements	request_id、original_resume_id、 tailored_resume_id、job_id、 improvements、created_at	改写历史

`processing_status` 取值集合在 `database.py:73` 注释中给出：`pending` / `processing` / `ready` / `failed`。

5.2 单例与懒加载

```
# apps/backend/app/database.py:295-296
# Global database instance
db = Database()
```

`Database` 类自身并不在 `__init__` 里打开文件，而是在 `db` 属性里懒加载（`database.py:28-33`）。这意味着只要不调用数据库方法，启动服务时就不会持有文件句柄。

5.3 Master 简历并发控制

`database.py:21` 定义了一个类级 `asyncio.Lock`：

```
_master_resume_lock = asyncio.Lock()
```

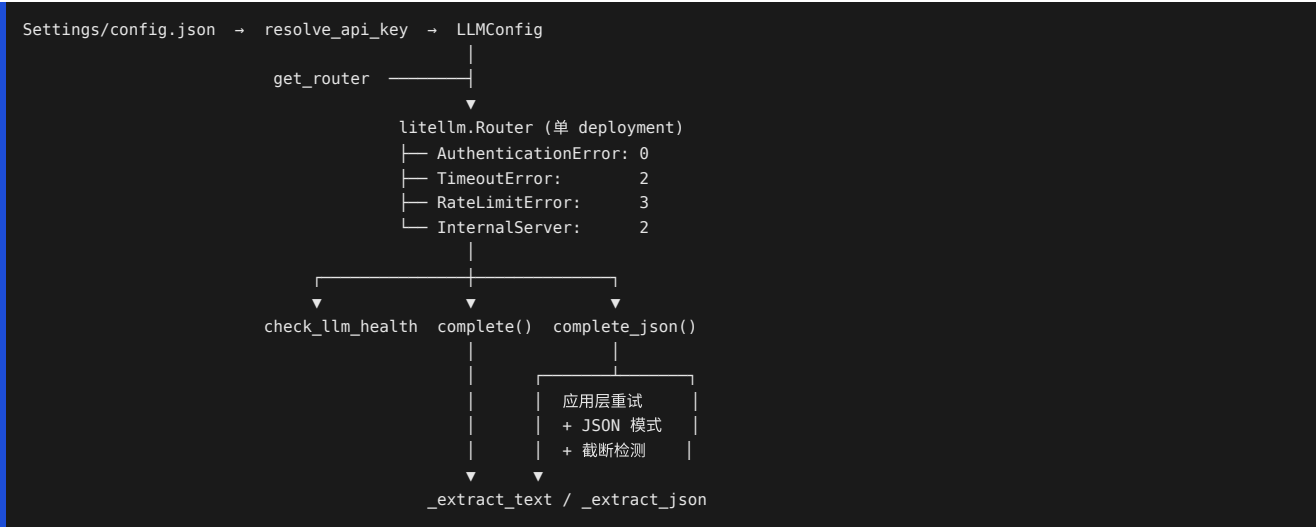
`create_resume_atomic_master`（`database.py:98-139`）在锁内完成：

- 1. 读取当前 master。
- 2. 若 master 处于 `failed` / `processing`，先降级再把新简历晋升为 master。
- 3. 调用 `create_resume` 写入。

`apps/backend/CLAUDE.md` 明确指出锁使用 `asyncio.Lock` 而非 `threading.Lock`，避免阻塞事件循环。

六、LLM 抽象层

`apps/backend/app/llm.py` 是整个后端最厚的一个文件（1157 行）。它的核心组件可以总结为下图：



6.1 Router 与配置指纹

llm.py:455-526 用 `litellm.Router` 把单模型封装成可重试的传输层，键控条件是 `_config_fingerprint` (llm.py:460-469) —— 只要 `provider | model | hash(api_key) | api_base` 不变就复用同一个 Router。`num_retries=3` 的 `RetryPolicy` (llm.py:492-499) 区分错误类型：对鉴权失败和 4xx 不重试，对超时/限流/5xx 才重试。

6.2 三个公开异步函数

函数	位置	用途
<code>check_llm_health</code>	llm.py:529-648	用一句 "Hi" 探活，并把错误信息脱敏后再返回
<code>complete</code>	llm.py:651-699	纯文本补全，自动剥离 <code><think></code> 标签
<code>complete_json</code>	llm.py:1026-1156	结构化补全，JSON 模式 → prompt-only 回退、3 次重试、截断感知

6.3 自适应超时

llm.py:897-924 的 `_calculate_timeout` 同时考虑"操作类型"与"Provider 因子"：

```
provider_factors = {
    "openai": 1.0,
    "anthropic": 1.2,
    "openrouter": 1.5,
    "groq": 1.0,
    "ollama": 2.0, # 本地模型最慢
}
```

基础值在 llm.py:39-41： `health_check=30s`、`completion=120s`、`json=180s`，再按 `max_tokens/4096` 比例放大。

6.4 JSON 提取与截断检测

`_extract_json` (llm.py:941-1023) 用大括号配对算法从 Markdown 代码块 / 原始 JSON 中切出字符串；`_appears_truncated` (llm.py:781-824) 对 `resume` / `enrichment` / `diff` / `keywords` 四类 schema 各自判断是否可能被截断，命中时 `complete_json` 会把 prompt 追加提示并重试。

七、PDF 渲染

apps/backend/app/pdf.py 通过 Playwright 启动无头 Chromium，把前端 `/print/*` 页面渲染成 PDF。

7.1 懒加载与平台兼容

```
# apps/backend/app/pdf.py:26-30
_playwright = None
_browser: Optional[Browser] = None
_init_lock = asyncio.Lock()
_subprocess_lock = asyncio.Lock()
_subprocess_supported = True
```

`init_pdf_renderer` (pdf.py:33-51) 使用 `asyncio.Lock` 防止并发初始化。`render_resume_pdf` (pdf.py:251-310) 在事件循环不支持子进程 (Windows 的 `SelectorEventLoop`) 时自动退化到 `_render_resume_pdf_in_thread`，因此在 `main.py:11-12` 还设置了 `WindowsProactorEventLoopPolicy`。

7.2 浏览器查找

`find_chromium_executable` (pdf.py:73-111) 在 Windows / macOS / Linux 上分别扫描 Chrome / Edge / Chromium / Snap / Flatpak 的常见安装路径；找不到则用 `PDFRenderError` 提示用户执行 `python -m playwright install chromium` (pdf.py:209-227)。

7.3 渲染管线

```
# apps/backend/app/pdf.py:129-143
async def _render_page_to_pdf(page, url, selector, pdf_format, pdf_margins):
    await page.goto(url, wait_until="networkidle")
    await page.wait_for_selector(selector)
    await page.evaluate("document.fonts.ready")
    return await page.pdf(format=pdf_format, print_background=True, margin=pdf_margins)
```


等待 CSS 选择器（默认 `.resume-print`）出现且字体加载完成后再调用 `page.pdf`，避免出现空白页或回退字体。

八、路由层

所有路由前缀 `/api/v1`，由 `apps/backend/app/routers/__init__.py:1-15` 统一导出。

路由文件	主要端点	行号
<code>health.py</code>	<code>GET /health</code> 、 <code>GET /status</code>	<code>health.py:12</code> 、 <code>health.py:21</code>
<code>config.py</code>	17 个端点，覆盖 LLM 配置/功能/语言/提示词/Key/重置	<code>config.py:98,114,171,214,225,249,264,302,318,35</code>
<code>resumes.py</code>	16 个端点，最大模块（1796 行）	<code>resumes.py:511,597,647,673,963,1098,1340,1389,1</code>
<code>jobs.py</code>	<code>POST /upload</code> 、 <code>GET /{id}</code>	<code>jobs.py:11,42</code>
<code>enrichment.py</code>	5 个端点，AI 增强模块	<code>enrichment.py:87,172,328,487,547</code>

完整端点表见 `docs/agent/architecture/backend-architecture.md:20-50` 与 `apps/backend/CLAUDE.md` 的 "Routers" 章节。

九、服务层（业务逻辑）

`apps/backend/app/services/__init__.py:1-14` 显式导出了五个对外服务：

服务	主要职责
<code>parser.py</code>	PDF/DOCX → Markdown（ <code>parse_document</code> ， <code>parser.py:119</code> ）；Markdown → 结构化 JSON（ <code>parse_resume_to_json</code> ， <code>parser.py:144</code> ）并自动还原月份（ <code>restore_dates_from_markdown</code> ， <code>parser.py:40</code> ）
<code>improver.py</code>	关键词抽取、改写（diff 模式 + 全量回退）、路径白名单校验、master 简历对齐
<code>refiner.py</code>	多轮润色：关键词注入、AI 套话清理、master 对齐校验
<code>cover_letter.py</code>	求职信、冷邮件话术、简历标题生成；支持用户自定义 prompt 回退（ <code>cover_letter.py:17-33</code> ）

服务层只依赖 `app.llm`、`app.prompts`、`app.schemas`，不直接操作 `app.database`（持久化由路由层统一处理）。

十、提示词与 Schema

10.1 提示词位置

文件	关键常量
<code>prompts/templates.py</code>	<code>PARSE_RESUME_PROMPT</code> 、 <code>EXTRACT_KEYWORDS_PROMPT</code> 、 <code>IMPROVE_RESUME_PROMPTS</code> （nudge/keywords/full 三种）、 <code>DIFF_IMPROVE_PROMPT</code> 、 <code>COVER_LETTER_PROMPT</code> 、 <code>OUTREACH_MESSAGE_PROMPT</code> 、 <code>GENERATE_TITLE_PROMPT</code> 、 <code>RESUME_SCHEMA_EXAMPLE</code> 、 <code>CRITICAL_TRUTHFULNESS_RULES</code> 、 <code>LANGUAGE_NAMES</code>
<code>prompts/enrichment.py</code>	<code>ANALYZE_RESUME_PROMPT</code> 、 <code>ENHANCE_DESCRIPTION_PROMPT</code> 、 <code>REGENERATE_ITEM_PROMPT</code> 、 <code>REGENERATE_SKILLS_PROMPT</code>
<code>prompts/refinement.py</code>	<code>KEYWORD_INJECTION_PROMPT</code> 、 <code>VALIDATION_POLISH_PROMPT</code> 、 <code>AI_PHRASE_BLACKLIST</code> 、 <code>AI_PHRASE_REPLACEMENTS</code>

`apps/backend/CLAUDE.md` 反复强调：模板里所有字面量 `{ }` 都必须写成 `{{ }}`，否则 `str.format()` 会抛 `KeyError`。

10.2 Schema

`apps/backend/app/schemas/__init__.py:1-109` 一并导出了 Pydantic v2 模型。`ResumeData`（`models.py:341`）是结构化简历的总入口，包含：

- `personalInfo: PersonalInfo`（`models.py:121-131`）

- `summary: str`
- `workExperience: list[Experience]` (`models.py:134-148`)
- `education: list[Education]` (`models.py:150-163`)
- `personalProjects: list[Project]` (`models.py:165-179`)
- `additional: AdditionalInfo` (`models.py:182-199`, 含 `technicalSkills` 等)
- `customSections: dict[str, CustomSection]` (`models.py:231-264`)
- `sectionMeta: list[SectionMeta]` (`models.py:203-212`), 决定渲染顺序

十一、典型请求/数据流

下面以 "上传简历 → 解析 → 针对 JD 改写" 为例, 串起全部模块:

```

1. 客户端 POST /api/v1/resumes/upload (multipart/form-data, file=*.pdf|*.docx)
   └─ resumes.py:511 upload_resume
      └─ parser.py:119 parse_document (markdown: PDF/DOCX → md)
         └─ database.py:98 create_resume_atomic_master (Master 锁)
            └─ parser.py:144 parse_resume_to_json (LLM JSON)
               └─ llm.py:1026 complete_json (litellm.Router)
                  └─ parser.py:40 restore_dates_from_markdown
                     └─ models.py:341 ResumeData.model_validate

2. 客户端 POST /api/v1/jobs/upload (JSON: {job_descriptions: [...], resume_id})
   └─ jobs.py:11 upload_job_descriptions
      └─ database.py:204 db.create_job

3. 客户端 POST /api/v1/resumes/improve/preview (JSON: {resume_id, job_id})
   └─ resumes.py:673 improve_resume_preview_endpoint
      └─ asyncio.wait_for 240 秒硬性超时 (resumes.py:703)
         └─ services/improver.py
            └─ extract_job_keywords (LLM, 缓存到 job)
               └─ generate_skill_target_plan + verify_skill_target_plan
                  └─ generate_resume_diffs (LLM diff)
                     └─ apply_diffs (路径白名单 + 字段黑名单)
                        └─ verify_diff_result (diff 真伪校验)
                           └─ services/refiner.py
                              └─ analyze_keyword_gaps
                                 └─ inject_keywords (LLM)
                                    └─ remove_ai_phrases (本地正则)
                                       └─ validate_master_alignment
                                          └─ 返回 preview_hash 给前端

4. 客户端 POST /api/v1/resumes/improve/confirm (带 preview_hash)
   └─ resumes.py:963 improve_resume_confirm
      └─ 校验 preview_hash 后写入 tailored resume + improvements 记录

5. 客户端 GET /api/v1/resumes/{id}/pdf
   └─ resumes.py:1389 render_resume_pdf
      └─ pdf.py:251 render_resume_pdf
         └─ Playwright 打开 FRONTEND_BASE_URL/print/{id}?token=...
            └─ page.pdf → 字节流

```

十二、关键约定

- **类型注解**: 所有 Python 函数 (包含私有 helper) 都必须带类型注解 (`apps/backend/CLAUDE.md` Non-Negotiable Rules #1)。
- **错误处理**: 服务端记详细日志, 客户端只回泛化消息 (参考 `main.py` / `resumes.py:120-127` 的 `_raise_improve_error`)。
- **不可变共享对象**: 所有 "可变量默认值" 与 "缓存读出" 必须 `copy.deepcopy()` (参考 `models.py:326-338` 的 `normalize_resume_data`)。
- **路由挂载**: 新端点必须挂在 `/api/v1` 前缀下, 并在 `routers/_init_.py` 暴露。
- **单 worker 假设**: 所有 `asyncio.Lock` / 全局缓存 (`config_cache`、`db`) 均假设只有一个 uvicorn worker; 多 worker 部署需要先重申这些并发原语。

十三、小结

- 后端只有 6 个核心文件 (`main.py` / `config.py` / `database.py` / `llm.py` / `pdf.py` + 一个 schema 总入口), 其余都是 "路由-服务" 两层扩展。
- **路由层** 只做参数校验、流程编排、超时控制; **服务层** 不直接碰数据库; **LLM 抽象层** 统一处理 Provider 差异、JSON 解析、重试、截断。
- 配置与 API key 走 "env → config.json → 多 provider 字典" 三级回退, 且对本地 Provider 刻意跳过 env 兜底。

- 数据全部本地化 ([apps/backend/data/](#)), 可一键 `POST /api/v1/config/reset` 销毁 ([config.py:568-594](#)), 但会要求传 `{"confirm": "RESET_ALL_DATA"}` 二次确认。

下一步建议阅读 [04-后端核心模块](#), 那里会逐文件拆解 [llm.py](#) / [database.py](#) / [pdf.py](#) / [services/improver.py](#) 的关键函数与数据流。

04 · Resume Matcher 后端核心模块精读

本文接续 [03-后端架构](#)，按"数据库 → LLM → PDF → 配置路由 → 简历服务 → Schema"的顺序，逐函数拆解后端最关键的 6 个文件。读者应是熟悉 FastAPI 与 `asyncio` 的中级开发者。

一、数据库层 `apps/backend/app/database.py`

1.1 类结构

整个文件围绕一个 `Database` 类（[database.py:18-296](#)）展开，使用 TinyDB 4.8 + JSON 文件存储。类内部维护：

- `self.db_path: Path` — 默认指向 `settings.db_path`（[config.py:206-208](#)），即 `apps/backend/data/database.json`。
- `self._db: TinyDB | None` — 懒加载，访问 `self.db` 时才打开。
- `_master_resume_lock: asyncio.Lock` — 类级锁，所有实例共享（[database.py:21](#)）。

1.2 三张表

```
# apps/backend/app/database.py:36-49
@property
def resumes(self) -> Table: ...      # 简历
@property
def jobs(self) -> Table: ...          # 岗位描述
@property
def improvements(self) -> Table: ...  # 改写历史
```

每张表都是 TinyDB 的 `Table`，可以直接用 `Query()` 进行字段过滤。

1.3 关键函数

函数	行号	关键点
<code>create_resume</code>	database.py:57-96	直接 <code>insert</code> 一行简历，状态默认 <code>pending</code> ；时间戳为 UTC ISO 字符串
<code>create_resume_atomic_master</code>	database.py:98-139	抢占 Master；旧 master 卡在 <code>failed</code> / <code>processing</code> 时自动降级
<code>get_master_resume</code>	database.py:147-151	查 <code>is_master == True</code> 的第一条
<code>set_master_resume</code>	database.py:182-201	先清空所有 <code>is_master=True</code> ，再设置新主
<code>create_job</code>	database.py:204-216	仅存 <code>content</code> 和可选的 <code>resume_id</code>
<code>create_improvement</code>	database.py:233-253	把 <code>original_resume_id</code> / <code>tailored_resume_id</code> / <code>job_id</code> 关联起来
<code>get_improvement_by_tailored_resume</code>	database.py:255-267	用于"按需生成求职信"时回查 JD
<code>reset_database</code>	database.py:279-292	截断三张表 + 删除 <code>data/uploads/</code> 目录

1.4 全局单例

```
# apps/backend/app/database.py:295-296
# Global database instance
db = Database()
```

整个后端只用这一个 `db`。所有路由都 `from app.database import db` 直接拿。

二、LLM 抽象层 `apps/backend/app/llm.py`

2.1 文件骨架

区段	行号	作用
日志与全局开关	<code>llm.py:1-51</code>	LiteLLM 静默、 <code>drop_params=True</code> 、 <code>modify_params=True</code> 、超时常量
<code>_normalize_api_base</code>	<code>llm.py:64-114</code>	修复 <code>anthropic</code> / <code>gemini</code> / <code>openrouter</code> 的 <code>/v1/v1</code> 重复
<code>_effective_api_key</code>	<code>llm.py:123-131</code>	<code>openai_compatible</code> 留空时填入 sentinel
文本提取	<code>llm.py:134-256</code>	<code>_extract_text_parts</code> / <code>_extract_message_text</code> / <code>_extract_choice_text</code> ，兼容 <code>reasoning_content</code> 与 <code>thinking</code>
密钥解析	<code>llm.py:302-349</code>	<code>resolve_api_key</code> （单一真源）+ <code>_scrub_secrets</code> （脱敏）
LLMConfig	<code>llm.py:54-61</code> 、获取	<code>get_llm_config</code> （ <code>llm.py:352-400</code> ）
模型命名	<code>llm.py:403-448</code>	<code>get_model_name</code> （添加 <code>openai/</code> 、 <code>anthropic/</code> 等前缀）
Router 构建	<code>llm.py:455-526</code>	<code>_build_router</code> 、 <code>get_router</code> （按指纹缓存）
<code>check_llm_health</code>	<code>llm.py:529-648</code>	单句 <code>"Hi"</code> 探活 + 错误归类
<code>complete</code>	<code>llm.py:651-699</code>	纯文本补全 + 思维链剥离
<code>complete_json</code>	<code>llm.py:1026-1156</code>	JSON 补全 + 应用层重试 + schema 感知截断检测

2.2 关键函数

`resolve_api_key`（`llm.py:323-349`）

```
def resolve_api_key(stored: dict, provider: str) -> str:
    api_key = stored.get("api_key", "")
    if not api_key:
        api_keys = stored.get("api_keys", {})
        config_provider = _PROVIDER_KEY_MAP.get(provider, provider)
        env_default = (
            ""
            if provider in _PROVIDERS_WITHOUT_ENV_KEY_FALLBACK
            else settings.llm_api_key
        )
        api_key = api_keys.get(config_provider, env_default)
    return api_key
```

四个细节：

- 1. 顶层 `api_key` 优先于 `api_keys[provider]`，保留向后兼容。
- 2. 本地 Provider 跳过 env 兜底，避免付费 key 泄漏到本地 LLM。
- 3. `_PROVIDER_KEY_MAP`（`llm.py:302-311`）把内部 `gemini` 映射到配置文件中的 `google`。
- 4. 所有获取 key 的代码路径都必须走它；README/CLAUDE 反复强调这是 "single source of truth"。

`get_llm_config`（`llm.py:352-400`）

读 `config.json` → 合并 Settings 默认值 → 处理 gpt-5 的 "一次性迁移"（首次 `reasoning_effort` 缺失时自动写 `"minimal"`，但用户显式清空时不会再恢复）→ 返回 `LLMConfig`（Pydantic 模型）。

`get_model_name`（`llm.py:403-448`）

把 "用户输入" 转成 LiteLLM 期望的格式：

Provider	最终前缀示例
openai	<code>gpt-5-nano-2025-08-07</code> (无前缀)
openai_compatible	<code>openai/<model></code> (使用 OpenAI 客户端)
ollama	<code>ollama_chat/<model></code> (走 <code>/api/chat</code>)
openrouter	<code>openrouter/<model></code> 或 <code><model></code> (已经带前缀时)

如果模型名已经包含 `anthropic/`、`gemini/` 等已知前缀，则不再添加。

`get_router` (`llm.py:507-526`)

```
def get_router(config: LLMConfig | None = None) -> tuple[Router, LLMConfig]:
    global _router, _router_config_key
    if config is None:
        config = get_llm_config()
    key = _config_fingerprint(config)
    with _router_lock:
        if _router is None or _router_config_key != key:
            _router = _build_router(config)
            _router_config_key = key
    return _router, config
```

线程锁 + 指纹缓存：只有当 `provider|model|hash(key)|api_base` 真正变化时才重建 Router。`_build_router` (`llm.py:472-504`) 使用 `num_retries=3` 的 `RetryPolicy`，并显式 `disable_cooldowns=True` (单 deployment 时不需要冷却，否则一次失败会让整个后端停摆)。

`check_llm_health` (`llm.py:529-648`)

- `ollama` 与 `openai_compatible` 允许 key 为空；其他 Provider 必须有 key，否则返回 `api_key_missing`。
- 用一句话 "Hi" 调 `litellm.acompletion`，`max_tokens=64`、`timeout=30s`。
- 失败时把异常信息按规则归类：`duplicate_vl_path` (`/v1/v1/`)、`not_found_404`、`html_response`、`health_check_failed`。
- 调用 `_scrub_secrets` (`llm.py:286-299`) 屏蔽可能出现在异常文本里的 `sk-...` / `AIza...` / `Bearer ...` 模式，避免在 Settings 页面被回读。
- 探测时把 `test_prompt` 与 `model_output` 包装成 ````text ... ````，方便前端调试面板渲染。

`complete` (`llm.py:651-699`)

- 优先判断 `_supports_temperature` (`llm.py:827-876`) 决定是否下发 `temperature`。
- 推理模型 (deepseek-r1、qwq) 的 `<think>...</think>` 由 `_strip_thinking_tags` (`llm.py:927-938`) 剥离。
- 失败时记录详细日志，重新包装为通用 `ValueError` 返回给调用方。

`complete_json` (`llm.py:1026-1156`)

`complete_json` 是最复杂的对外函数。核心循环：

```
for attempt in range(retries + 1):
    # 1) 调整 temperature (_get_retry_temperature 递增: 0.1 → 0.3 → 0.5 → 0.7)
    # 2) 第一次失败后回退到 prompt-only JSON 模式 (JSON-012)
    # 3) 调用 router.acompletion
    # 4) _extract_json 切出 JSON 字符串
    # 5) _appears_truncated(schema_type) 判断截断，命中则改写 prompt 重试
    # 6) 成功直接 return
```

错误分类：

- `json.JSONDecodeError` → 内容质量问题，重试 + 关闭 JSON 模式。
- `ValueError` (空响应 / 切不出 JSON) → 重试。
- 其他 `Exception` → 不重试 (Router 已经做过传输重试)。

配套工具

工具	行号	作用
<code>_supports_json_mode</code>	<code>llm.py:702-731</code>	查 <code>litellm.get_model_info</code> 判断是否支持 <code>response_format</code> ；Ollama 永远返回 True
<code>get_safe_max_tokens</code>	<code>llm.py:736-778</code>	把 <code>DEFAULT_JSON_MAX_TOKENS=8192</code> 截到模型上限
<code>_appears_truncated</code>	<code>llm.py:781-824</code>	schema 感知：resume 看核心数组是否空、enrichment 看 keys 是否齐全
<code>_calculate_timeout</code>	<code>llm.py:897-924</code>	基础超时 × token 比例 × provider 因子
<code>_scrub_secrets</code>	<code>llm.py:286-299</code>	正则屏蔽 <code>sk-...</code> 、 <code>Aiza...</code> 、 <code>Bearer ...</code>

三、PDF 渲染 `apps/backend/app/pdf.py`

3.1 状态机

模块维护 4 个模块级变量（`pdf.py:26-30`）：

```
_playwright = None
_browser: Optional[Browser] = None
_init_lock = asyncio.Lock() # 防并发初始化
_subprocess_lock = asyncio.Lock()
_subprocess_supported = True # 标记当前事件循环能否跑子进程
```

3.2 入口

```
# apps/backend/app/pdf.py:251-310
async def render_resume_pdf(
    url: str,
    page_size: str = "A4",
    selector: str = ".resume-print",
    margins: Optional[dict] = None,
) -> bytes:
    ...
```

执行步骤：

- `_resolve_pdf_format`（`pdf.py:54-59`）把 `"A4"` / `"LETTER"` 映射到 Playwright 接受的字符串。
- `_resolve_pdf_margins`（`pdf.py:62-70`）统一为 `"{N}mm"`，避免 HTML padding 只影响首页。
- 已有浏览器时直接渲染（`pdf.py:274-278`）。
- 否则根据 `_subprocess_supported` 选择主循环（`async_playwright`）或子线程（`asyncio.to_thread` + 新事件循环）路径。
- 最终调用 `_render_with_browser`（`pdf.py:146-157`），它会在新 page 上 `goto` → `wait_for_selector` → `document.fonts.ready` → `page.pdf`。

3.3 容错与错误分类

`_raise_playwright_error`（`pdf.py:209-227`）把 Playwright 异常映射成 `PDFRenderError`：

- `"Executable doesn't exist"` → 提示执行 `python -m playwright install chromium`。
- `"net::ERR_CONNECTION_REFUSED"` → 提示前端未启动 / `FRONTEND_BASE_URL` 不对。

`_find_chromium_executable`（`pdf.py:73-111`）依次扫描系统级 Chrome / Edge / Chromium / Snap / Flatpak，落到 Linux 桌面发行版的常见路径。

3.4 Windows 兼容

`main.py:11-12` 在 Windows 下设置 `WindowsProactorEventLoopPolicy`，让 Playwright 的子进程能在 `asyncio` 循环里运行。若循环仍不支持（`_loop_supports_subprocess` 返回 `False`），`render_resume_pdf` 会自动把渲染任务丢到 `asyncio.to_thread`（`pdf.py:198-206`）用 `ProactorEventLoop` 跑。

四、配置路由 `apps/backend/app/routers/config.py`

`config.py` 是路由模块里第二大文件（594 行），包含 16 个端点，按职责分为 5 组。

4.1 端点速查

分组	方法 + 路径	行号	备注
LLM 配置	<code>GET /llm-api-key</code>	<code>config.py:98</code>	返回脱敏 key
LLM 配置	<code>PUT /llm-api-key</code>	<code>config.py:114</code>	保存并触发后台 health check
LLM 配置	<code>POST /llm-test</code>	<code>config.py:171</code>	支持用请求体临时覆盖存储值
Feature Flag	<code>GET/PUT /features</code>	<code>config.py:214,225</code>	是否生成求职信 / 冷邮件
Language	<code>GET/PUT /language</code>	<code>config.py:249,264</code>	校验 <code>en/es/zh/ja/pt</code>
Prompt	<code>GET/PUT /prompts</code>	<code>config.py:302,318</code>	切换 <code>nudge/keywords/full</code> 三种改写 prompt
Feature Prompt	<code>GET/PUT /feature-prompts</code>	<code>config.py:350,367</code>	校验必须含 <code>{job_description} {resume_data} {ou</code>
API Key	<code>GET /api-keys</code>	<code>config.py:435</code>	返回 6 个 Provider 的脱敏状态
API Key	<code>POST /api-keys</code>	<code>config.py:458</code>	增量更新
API Key	<code>DELETE /api-keys?confirm=CLEAR_ALL_K</code>	<code>config.py:520</code>	危险操作，查询参数确认
API Key	<code>DELETE /api-keys/{provider}</code>	<code>config.py:546</code>	单 Provider 删除
Reset	<code>POST /reset</code>	<code>config.py:568</code>	销毁数据，body <code>{"confirm": "RESET_ALL_DATA"}</code>

4.2 关键模式

- **Key 脱敏** (`config.py:68-74`)：保留首 4 + 末 4，中间用 `*`。
- **配置保存后失效缓存**： `_save_config` 末尾调用 `invalidate_config_cache()` (`config.py:60-65`)，让下一次 `load_config` 重新读盘。
- **背景健康检查**： `PUT /llm-api-key` 用 `BackgroundTasks` (`config.py:160`) 调用 `_log_llm_health_check` (`config.py:82-95`)，不阻塞响应。
- **Prompt 校验**： `PUT /feature-prompts` 使用 `validate_prompt_placeholders` (`config.py:382-410`) 确保三个占位符都在；缺则返回 422 + 详细 `missing` 列表。
- **确认 token 模式**： `delete_all_api_keys` 与 `reset_database_endpoint` 都强制要求字符串 token，避免误操作 (`config.py:536-541,588-592`)。

五、简历路由 `apps/backend/app/routers/resumes.py`

`resumes.py` 是后端最复杂的文件（1796 行，16 个端点），承担了完整的"上传 → 解析 → 改写 → 确认 → 导出 PDF"流程。

5.1 端点

行号	方法 + 路径	作用
<code>resumes.py:511</code>	<code>POST /upload</code>	PDF/DOCX 上传, 调用 <code>parse_document</code> + <code>parse_resume_to_json</code>
<code>resumes.py:597</code>	<code>GET /</code>	通过 <code>?resume_id=</code> 查单条
<code>resumes.py:647</code>	<code>GET /list</code>	列出所有 (可排除 master)
<code>resumes.py:673</code>	<code>POST /improve/preview</code>	预览改写, 不入库; 返回 <code>preview_hash</code>
<code>resumes.py:963</code>	<code>POST /improve/confirm</code>	确认改写, 校验 <code>preview_hash</code> 后入库
<code>resumes.py:1098</code>	<code>POST /improve</code>	一站式改写 (旧版接口); 返回 <code>resume_id</code>
<code>resumes.py:1340</code>	<code>PATCH /{id}</code>	修改结构化字段
<code>resumes.py:1389</code>	<code>GET /{id}/pdf</code>	渲染 PDF
<code>resumes.py:1472</code>	<code>DELETE /{id}</code>	删除
<code>resumes.py:1481</code>	<code>POST /{id}/retry-processing</code>	重新跑 LLM 解析
<code>resumes.py:1533</code>	<code>PATCH /{id}/cover-letter</code>	直接保存求职信
<code>resumes.py:1546</code>	<code>PATCH /{id}/outreach-message</code>	直接保存冷邮件
<code>resumes.py:1559</code>	<code>PATCH /{id}/title</code>	修改标题
<code>resumes.py:1571</code>	<code>POST /{id}/generate-cover-letter</code>	按需生成求职信
<code>resumes.py:1644</code>	<code>POST /{id}/generate-outreach</code>	按需生成冷邮件
<code>resumes.py:1715</code>	<code>GET /{id}/job-description</code>	取出关联的 JD
<code>resumes.py:1757</code>	<code>GET /{id}/cover-letter/pdf</code>	把求职信渲染为 PDF

5.2 改写管线的三步

- 预览 (`improve_resume_preview_endpoint`, `resumes.py:673`): 调用 `_improve_preview_flow`, 用 `asyncio.wait_for(..., timeout=240)` 包住整条管线。
- 确认 (`improve_resume_confirm`, `resumes.py:963`): 重新计算 preview 的 SHA-256 (`_hash_improved_data`, `resumes.py:94-104`) 与请求体里带的 hash 比对, 再 `db.create_resume` 写入。
- 直接改写 (`improve_resume_endpoint`, `resumes.py:1098`): 老接口, 直接持久化。

预览/确认的好处: 避免 LLM 抽风时污染 master 简历, 且能给用户一个"diff 视图"。

5.3 内部辅助函数

辅助	行号	用途
<code>_get_default_prompt_id</code>	<code>resumes.py:66-71</code>	从 config.json 读 <code>default_prompt_id</code> , 缺失回退到 <code>DEFAULT_IMPROVE_PROMPT_ID</code>
<code>_hash_job_content</code>	<code>resumes.py:74-75</code>	JD 文本的 SHA-256 (用于关键词缓存 key)
<code>_normalize_payload</code>	<code>resumes.py:78-91</code>	Unicode NFC 归一化, 确保哈希稳定
<code>_hash_improved_data</code>	<code>resumes.py:94-104</code>	序列化 + <code>sort_keys</code> + SHA-256, 用于 preview 校验
<code>_get_original_resume_data</code>	<code>resumes.py:130-137</code>	兼容旧 <code>content_type=="json"</code> 的记录
<code>_get_original_markdown</code>	<code>resumes.py:140-153</code>	优先 <code>original_markdown</code> , 回退到 <code>content</code>
<code>_restore_original_dates</code>	<code>resumes.py:161-205</code>	把 LLM 丢掉的月份从原 markdown 还原
<code>_raise_improve_error</code>	<code>resumes.py:120-127</code>	统一日志+异常

六、改写服务 `apps/backend/app/services/improver.py`

6.1 概览

`improver.py` (1374 行) 是后端最大的服务文件, 包含"diff 模式 + 全量回退 + 字段/路径白名单"的全部逻辑。

6.2 关键函数

函数	行号	作用
<code>_sanitize_user_input</code>	<code>improver.py:48-56</code>	用 <code>_INJECTION_PATTERNS</code> 把"忽略以上指令"等关键词替换成 <code>[REDACTED]</code>
<code>_is_path_allowed</code>	<code>improver.py:112-114</code>	diff 路径白名单, 只允许 <code>summary</code> / <code>workExperience[i].description</code> / <code>personalProjects[i].description</code> / <code>additional.technicalSkills</code>
<code>_is_path_blocked</code>	<code>improver.py:117-...</code>	<code>personalInfo</code> / <code>customSections</code> / <code>sectionMeta</code> 永远拒绝; <code>years</code> / <code>company</code> / <code>title</code> 等"事实字段"也拒绝
<code>apply_diffs</code>	<code>improver.py:214-...</code>	把 LLM 返回的 diff 应用到原数据, 路径非法/字段黑名单就跳过并记录
<code>verify_diff_result</code>	<code>improver.py:381-...</code>	校验改写后的事实字段 (公司、年份、岗位) 是否被改动
<code>generate_resume_diffs</code>	<code>improver.py:457-...</code>	主 LLM 调用: 构造 <code>DIFF_IMPROVE_PROMPT</code> 、调 <code>complete_json</code> 、解析 <code>ImproveDiffResult</code>
<code>extract_job_keywords</code>	<code>improver.py:555-579</code>	用 <code>EXTRACT_KEYWORDS_PROMPT</code> 抽 JD 的 required / preferred skills
<code>improve_resume</code>	旧接口	整篇改写回退, LLM 一次性返回完整 <code>ResumeData</code>

6.3 提示词与变量

文件顶部 `improver.py:12-23` 集中导入 `EXTRACT_KEYWORDS_PROMPT`、`DIFF_IMPROVE_PROMPT`、`DIFF_STRATEGY_INSTRUCTIONS`、`SKILL_TARGET_PLAN_PROMPT`、`IMPROVE_RESUME_PROMPTS`、`CRITICAL_TRUTHFULNESS_RULES`、`get_language_name`, 并用 `IMPROVE_SCHEMA_EXAMPLE` 嵌入 JSON 模板。

6.4 改写流程 (在 `resumes.py:1138-1170` 调用)

```
extract_job_keywords(job)
↓
generate_skill_target_plan + verify_skill_target_plan
↓
generate_resume_diffs (LLM)
↓
apply_diffs (路径白名单 + 字段黑名单 + 事实校验)
↓
verify_diff_result (再校验一次)
↓
refine_resume (refiner.py: keyword 注入 / AI 套话清理 / master 对齐)
↓
返回 ImproveResumeData
```

七、精修服务 `apps/backend/app/services/refiner.py`

`refiner.py` (702 行) 实现"先关键词注入、再 AI 套话清理、最后 master 对齐"的多轮润色。核心入口是 `refine_resume` (`refiner.py:76-179`):

- 深度拷贝 `initial_tailored` 作为 `current`。
- `analyze_keyword_gaps` (`refiner.py:181-231`) 计算 `KeywordGapAnalysis`: JD 缺的关键词里只有"master 简历里也有"的才是 `injectable_keywords`, 其余标记为 `non_injectable_keywords` (防止编造技能)。
- `inject_keywords` (`refiner.py:452-...`) 让 LLM 把可注入关键词自然嵌入当前简历。

4. `remove_ai_phrases` (`refiner.py:233-288`) 纯本地操作: 用 `AI_PHRASE_BLACKLIST` 替换 `AI_PHRASE_REPLACEMENTS` (来自 `prompts/refinement.py`)。
 5. `validate_master_alignment` (`refiner.py:290-409`) 跑 `AlignmentReport` 检查, 必要时调 `fix_alignment_violations` (`refiner.py:515-563`)。
 6. 循环直到 `passes_completed == max_refinement_passes` 或没有可改进之处。
- `calculate_keyword_match` (`refiner.py:565-...`) 计算首末两端与 JD 的匹配率, 作为 API 响应里的 `RefinementStats` (`schemas/refinement.py:66-87`)。

八、Schema 层 `apps/backend/app/schemas/`

8.1 模块划分

文件	行数	关键模型
<code>models.py</code>	763	<code>SectionType</code> 、 <code>PersonalInfo</code> 、 <code>Experience</code> 、 <code>Education</code> 、 <code>Project</code> 、 <code>AdditionalInfo</code> 、 <code>SectionMeta</code> 、 <code>CustomSectionItem</code> 、 <code>CustomSection</code> 、 <code>ResumeData</code> 、 <code>normalize_resume_data</code> 、 <code>ResumeUploadResponse</code> 、 <code>RawResume</code> 、 <code>ResumeFetchData</code> 、 <code>ResumeFetchResponse</code> 、 <code>ResumeSummary</code> 、 <code>ResumeListResponse</code> 、 <code>JobUploadRequest/Response</code> 、 <code>ImproveResumeRequest/Response/ConfirmRequest</code> 、 <code>ImprovementSuggestion</code> 、 <code>ResumeFieldDiff</code> 、 <code>ResumeDiffSummary</code> 、 <code>RefinementStats</code> 、 <code>ImproveResumeData</code> 、 <code>ImproveDiffResult</code> 、 <code>LLMConfigRequest/Response</code> 、 <code>FeatureConfigRequest/Response</code> 、 <code>LanguageConfigRequest/Response</code> 、 <code>PromptOption</code> 、 <code>PromptConfigRequest/Response</code> 、 <code>FeaturePromptsRequest/Response</code> 、 <code>ApiKeyProviderStatus</code> 、 <code>ApiKeyStatusResponse</code> 、 <code>ApiKeysUpdateRequest/Response</code> 、 <code>ResetDatabaseRequest</code> 、 <code>UpdateCoverLetterRequest</code> 、 <code>UpdateOutreachMessageRequest</code> 、 <code>UpdateTitleRequest</code> 、 <code>GenerateContentResponse</code> 、 <code>HealthResponse</code> 、 <code>StatusResponse</code> 、 <code>ResumeChange</code>
<code>enrichment.py</code>	126	<code>EnrichmentItem</code> 、 <code>EnrichmentQuestion</code> 、 <code>AnalysisResponse</code> 、 <code>AnswerInput</code> 、 <code>EnhanceRequest</code> 、 <code>EnhancedDescription</code> 、 <code>EnhancementPreview</code> 、 <code>ApplyEnhancementsRequest</code> 、 <code>RegenerateRequest</code> 、 <code>RegeneratedItem</code> 、 <code>RegenerateItemError</code> 、 <code>RegenerateResponse</code>
<code>refinement.py</code>	125	<code>RefinementConfig</code> 、 <code>KeywordGapAnalysis</code> 、 <code>AlignmentViolation</code> 、 <code>AlignmentReport</code> 、 <code>RefinementStats</code> 、 <code>RefinementResult.to_stats</code>
<code>__init__.py</code>	109	统一 re-export

8.2 关键模型字段

ResumeData ([models.py:341-...](#))

```
class ResumeData(BaseModel):
    personalInfo: PersonalInfo = Field(default_factory=PersonalInfo)
    summary: str = ""
    workExperience: list[Experience] = Field(default_factory=list)
    education: list[Education] = Field(default_factory=list)
    personalProjects: list[Project] = Field(default_factory=list)
    additional: AdditionalInfo = Field(default_factory=AdditionalInfo)
    customSections: dict[str, CustomSection] = Field(default_factory=dict)
    sectionMeta: list[SectionMeta] = Field(default_factory=list)
```

每个嵌套模型都自带 `field_validator` (见 `Experience.normalize_description`、`AdditionalInfo.normalize_string_fields`)，保证从前端 / LLM 接收的脏数据进入业务逻辑前已经被归一化。

JobUploadRequest ([models.py:423-...](#))

```
class JobUploadRequest(BaseModel):
    job_descriptions: list[str]
    resume_id: str | None = None
```

[jobs.py:18-29](#) 还会做“每条 JD 都不能为空”的二次校验。

ImproveResumeRequest ([models.py:439-445](#))

```
class ImproveResumeRequest(BaseModel):
    resume_id: str
    job_id: str
    prompt_id: str | None = None
```

`prompt_id` 为空时由 `_get_default_prompt_id` ([resumes.py:66](#)) 回退到 `config.json` 里的 `default_prompt_id`。

LLMConfigRequest ([models.py:557-...](#))

```
class LLMConfigRequest(BaseModel):
    provider: str | None = None
    model: str | None = None
    api_key: str | None = None
    api_base: str | None = None
    reasoning_effort: str | None = None
```

所有字段都允许 `None`，让 PUT `/llm-api-key` 实现“只更新提供的字段”。

ApiKeysUpdateRequest ([models.py:675-...](#))

支持 6 个 Provider 的可选字段 (`openai` / `anthropic` / `google` / `openrouter` / `deepseek` / `groq`)。空字符串表示删除该 Provider 的 key ([config.py:469-509](#))。

RefinementConfig ([schemas/refinement.py:6-13](#))

```
class RefinementConfig(BaseModel):
    enable_keyword_injection: bool = True
    enable_ai_phrase_removal: bool = True
    enable_master_alignment_check: bool = True
    max_refinement_passes: int = Field(default=2, ge=1, le=5)
```

可作为未来 `POST /improve` 的请求体扩展点，让用户按需关闭某一项润色。

九、API 调用示例

下面给出三个最小可运行的示例（假设服务监听在 `http://localhost:8000`，`uvicorn` 已启动）。

9.1 健康检查 + LLM 探活

```
# 进程是否在跑 (不调 LLM)
curl -s http://localhost:8000/api/v1/health

# 完整状态: LLM + DB
curl -s http://localhost:8000/api/v1/status
```

预期响应 (参考 [health.py:13-18](#) 与 [health.py:21-40](#)) :

```
// /health
{ "status": "healthy" }
// /status
{ "status": "ready", "llm_configured": true, "llm_healthy": true,
  "has_master_resume": true, "database_stats": { "total_resumes": 1, ... } }
```

9.2 上传简历 (Python requests 版)

```
import requests
API = "http://localhost:8000/api/v1"

with open("resume.pdf", "rb") as f:
    r = requests.post(
        f"{API}/resumes/upload",
        files={"file": ("resume.pdf", f, "application/pdf")},
        timeout=60,
    )
r.raise_for_status()
data = r.json()
resume_id = data["resume_id"]
print("uploaded:", resume_id, "status:", data["processing_status"])
```

后端处理流程 ([resumes.py:511-594](#)) : [markdown](#) 抽文本 → [db.create_resume_atomic_master](#) → [parse_resume_to_json](#) (LLM JSON 解析) → 更新状态为 [ready](#) 或 [failed](#)。

9.3 改写预览 + 确认 (curl 版)

```
# 1) 上传 JD
JOB_ID=$(curl -s -X POST http://localhost:8000/api/v1/jobs/upload \
  -H 'Content-Type: application/json' \
  -d '{"job_descriptions":["We are looking for a senior Python developer..."]}' \
  | jq -r '.job_id[0]')

# 2) 预览改写 (不写入)
PREVIEW=$(curl -s -X POST http://localhost:8000/api/v1/resumes/improve/preview \
  -H 'Content-Type: application/json' \
  -d '{"resume_id\":\"$RESUME_ID\",\"job_id\":\"$JOB_ID\"}')
echo "$PREVIEW" | jq '.data.preview_hash, .data.resume_preview.personalInfo.name'

# 3) 确认写入
curl -s -X POST http://localhost:8000/api/v1/resumes/improve/confirm \
  -H 'Content-Type: application/json' \
  -d "$(jq -n --arg rid $RESUME_ID --arg jid $JOB_ID --arg ph "$PREVIEW_HASH" \
    '{resume_id:$rid, job_id:$jid, improved_data:<从 preview 复制>, preview_hash:$ph}')
```

[preview_hash](#) 是 [_hash_improved_data](#) ([resumes.py:94-104](#)) 算出的 SHA-256, 后端在 confirm 时会重新计算并比对, 校验失败则返回 400。

十、调试小贴士

- **LiteLLM 静默**: 默认 [log_llm=WARNING](#) ([config.py:138](#))。改成 [DEBUG](#) 并重启, 可以看到每条请求的 provider/model/usage。
- **Router 重建日志**: 切换 Provider/模型/Key 时会打印 ["LiteLLM Router rebuilt for ..."](#) ([llm.py:523](#)), 方便确认缓存确实失效。
- **PDF 渲染失败**: [PDFRenderError](#) 异常信息里会直接给出 [python -m playwright install chromium](#) 命令, 照抄即可。
- **Master 简历异常**: 当 master 状态卡在 [failed](#) 或 [processing](#), 下次上传会自动晋升新 master ([database.py:119-127](#))。
- **Schema 懒迁移**: 旧数据若缺 [SectionMeta](#), [GET /resumes?resume_id=...](#) 会自动用 [DEFAULT_SECTION_META](#) ([models.py:268-323](#)) 填充, 无需手动迁移。

十一、小结

- `database.py` 用类级 `asyncio.Lock` 守住"恰好一个 master 简历"的不变量。
- `llm.py` 是整个后端的"复杂度中心": Router 缓存、应用层重试、schema 感知截断、provider 能力探测都在它里面。
- `pdf.py` 通过懒加载 + 平台分支处理让 Chromium 在所有桌面平台都能跑。
- `config.py` 的 16 个端点可以"在线换 Provider / 改 Key / 自定义 Feature Prompt", 并且所有写操作都通过 `invalidate_config_cache()` 让其他模块感知。
- `improver.py` + `refiner.py` 共同构成"diff 改写 + 多轮润色"双层防护; `resumes.py` 的 preview/confirm 流程则把 LLM 不可控因素挡在 master 之外。

到此后端核心模块的逐函数拆解就完成了。下一步可结合 <docs/agent/architecture/backend-guide.md> 与 `apps/backend/tests/` 跑通端到端测试, 进一步体会各接口的边界条件。

Resume Matcher 的 AI 集成与提示词工程实战

本教程面向希望把 LLM 接入真实生产系统的 Python 后端开发者。我们以 Resume Matcher（一个面向求职场景的 AI 改写简历服务）为蓝本，从 LiteLLM 封装、提示词模板设计、端到端工作流、容错策略、新增 provider 的步骤等角度逐项拆解。所有结论都基于 `apps/backend/app/` 下的实际源码，并标注 `文件:行号` 以便复核。

1. 项目中的 LLM 集成位置一览

Resume Matcher 并不把 LLM 调用散落在路由层，而是统一收敛到三层：

层	文件	职责
传输/容错层	<code>apps/backend/app/llm.py</code>	适配 LiteLLM，处理多 provider、重试、超时、JSON 抽取
提示词层	<code>apps/backend/app/prompts/__init__.py</code> 、 <code>templates.py</code> 、 <code>enrichment.py</code> 、 <code>refinement.py</code>	全部提示词模板 + 占位符校验
业务编排层	<code>apps/backend/app/services/</code>	<code>parser.py</code> （解析）、 <code>improver.py</code> （diff 改写）、 <code>refiner.py</code> （多轮打磨）、 <code>cover_letter.py</code> （求职信/外联/标题）

路由层（`apps/backend/app/routers/`）只负责：接收请求 → 调服务 → 返回 Pydantic 模型。这种分层让 LLM 相关能力（切换 provider、调整 prompt、加重试）可以在不动路由的情况下集中迭代。

2. LiteLLM 封装分析（`apps/backend/app/llm.py`）

2.1 支持的 provider

`llm.py:302-311` 通过 `_PROVIDER_KEY_MAP` 显式列举了 8 个 provider：

provider 字符串	LiteLLM 实际路由	典型模型
<code>openai</code>	不加前缀	gpt-5、gpt-4o
<code>openai_compatible</code>	<code>openai/</code> 前缀	llama.cpp / vLLM / LM Studio
<code>anthropic</code>	<code>anthropic/</code> 前缀	Claude Haiku 4.5、Claude Opus 4.x
<code>gemini</code>	<code>gemini/</code> 前缀	Gemini 3 Flash
<code>openrouter</code>	<code>openrouter/</code> 前缀	任何 OpenRouter 列表里的模型
<code>deepseek</code>	<code>deepseek/</code> 前缀	DeepSeek Chat、DeepSeek R1
<code>groq</code>	<code>groq/</code> 前缀	Llama 3 在 Groq 上的部署
<code>ollama</code>	<code>ollama_chat/</code> 前缀	本地 ollama 服务

注意 `get_model_name`（`llm.py:403-448`）做了两件事：先按 `provider` 表加前缀，再识别 `openrouter/anthropic/...` 之类的“已经带前缀”的形式避免重复拼接。`ollama` 特殊用 `ollama_chat/` 而不是 `ollama/`，因为 chat 端点支持多轮消息数组（`llm.py:421`）。

2.2 `api_base` 的归一化

本地自托管（llama.cpp、vLLM、Ollama）和 OpenAI 兼容代理经常出现 `http://host:8080/v1` 这种 URL。`llm.py:64-114` 的 `_normalize_api_base` 做了三件事：

- `strip + rstrip('/')`；
- 对 Anthropic / Gemini / OpenRouter / Ollama，把末尾的 `/v1` 去掉（LiteLLM 内部还会拼一次，否则会出现 `/v1/v1/...` 的 404，参见 `llm.py:91-112` 注释）；

3. 对 OpenAI / OpenAI 兼容端点，原样保留 URL（包括 `/v1`），因为 OpenAI 客户端自己会正确解析。

`llm.py:617-620` 还用 `_scrub_secrets` 在向上游报告错误时遮蔽 `sk-...`、`AIza...`、`Bearer ...` 等敏感串，避免 Settings UI 间接读回 API key。

2.3 API key 的解析与防泄漏

`llm.py:323-349` 的 `resolve_api_key` 实现了"单一真实来源"，所有需要 key 的代码路径都必须走它。优先级是：

- `stored["api_key"]`（顶层字段，UI 写）；
- `stored["api_keys"][provider]`（按 provider 分桶）；
- `settings.llm_api_key`（环境变量）。

但对 `openai_compatible` 与 `ollama` 这两个本地 provider，`llm.py:318-320` 的 `_PROVIDERS_WITHOUT_ENV_KEY_FALLBACK` 集合强制跳过环境变量——因为 `LLM_API_KEY` 通常是付费 OpenAI 密钥，绝不能误传到本地明文服务。`openai_compatible` 用户留空时还会注入哨兵值 `sk-no-key` 满足 OpenAI 客户端的"非空字符串"校验（`llm.py:120-131`）。

2.4 Router、RetryPolicy 与 cooldown

`llm.py:472-504` 用 `litellm.Router` 包装一个单 deployment 的模型，配置如下：

- `num_retries=3`：LiteLLM 在 transport 层最多自动重试 3 次；
- `RetryPolicy` 按错误类型差异化：401/400/内容策略违规 → 0 次（重试无意义），超时/500 → 2 次，429 → 3 次；
- `disable_cooldowns=True`：因为只有一个 deployment，cooldown 会让后端在瞬时故障后整体黑屏。

`get_router`（`llm.py:507-526`）用 `provider|model|hash(api_key)|api_base` 指纹缓存 Router，仅在配置变化时重建，配合 `threading.Lock` 处理并发请求。

2.5 能力探测：能力来自 Registry 而不是硬编码

避免在代码里写 `if provider == "openai"` 这类硬编码——`llm.py` 通过 `litellm.get_model_info` 查询注册表：

- `_supports_json_mode`（`llm.py:702-731`）：判断模型是否在 `supported_openai_params` 里含 `response_format`。Ollama 走 `format="json"`，所以特判返回 True。
- `get_safe_max_tokens`（`llm.py:736-778`）：把调用方请求的 token 数 clamp 到 `max_output_tokens`，未知模型 fallback 4096。
- `_supports_temperature`（`llm.py:827-876`）：返回 False 时调用方就不会传 `temperature`，避免 `BadRequestError`。同时为 Anthropic Claude Opus 4.x（已弃用 temperature）、Moonshot Kimi K2.6（只允许 1.0）做硬编码特判。
- 顶层 `litellm.drop_params = True`（`llm.py:31`）：让 LiteLLM 主动丢弃目标 provider 不识别的参数（如 `reasoning_effort`、非默认 `temperature`），不再抛 `UnsupportedParamsError`。

2.6 两条主调用 API

`complete`（`llm.py:651-699`）与 `complete_json`（`llm.py:1026-1156`）是服务的唯二对外函数。区别：

维度	<code>complete</code>	<code>complete_json</code>
输出	纯文本	解析后的 <code>dict</code>
JSON 模式	否	自动启用 <code>response_format={"type": "json_object"}</code>
应用层重试	0	2 次，针对 malformed JSON / truncation
重试策略	全部由 Router 处理	解析失败时改温度（0.1→0.3→0.5→0.7，参见 <code>_get_retry_temperature</code> <code>llm.py:879-894</code> ）
失败兜底	抛 <code>ValueError</code>	失败时关闭 JSON 模式，改用 prompt 强制（ <code>json_mode_failed</code> 标志， <code>llm.py:1126-1133</code> ）

`complete_json` 还有 schema 感知的截断检测 `appears_truncated`（`llm.py:781-824`）：当 schema 是 `resume` 且 `workExperience/education/skills` 出现空数组、`enrichment` 缺 `items_to_enrich/questions` 时判为截断并重试。

2.7 错误处理与安全

`llm.py:693-699` 是 LLM 调用的"安全降级"模板：


```
except Exception as e:
    logging.error(f"LLM completion failed: {e}", extra={"model": model_name})
    raise ValueError(
        "LLM completion failed. Please check your API configuration and try again."
    ) from e
```

详情在服务端日志，客户端只拿到一段通用提示。`_scrub_secrets` (`llm.py:286-299`) 在 `error_detail` 露出前再次擦除 key。

`check_llm_health` (`llm.py:529-648`) 把 `404 /v1/v1/`、`404` 通用、HTML 响应等常见错误映射到独立 `error_code`，前端可以直接用来定位"用户填的 base URL 多带了一个 `/v1`"。

3. 提示词模板设计

3.1 文件分布

文件	主要常量	备注
<code>apps/backend/app/prompts/_init__.py:1-57</code>	集中 re-export 与 <code>validate_prompt_placeholders</code>	业务代码只 <code>from app.prompts import ...</code>
<code>apps/backend/app/prompts/templates.py</code>	解析、关键词、3 个改写、diff、skill plan、求职信、外联、标题	大本营
<code>apps/backend/app/prompts/enrichment.py</code>	<code>ANALYZE_RESUME_PROMPT</code> 、 <code>ENHANCE_DESCRIPTION_PROMPT</code> 、 <code>REGENERATE_ITEM_PROMPT</code> 、 <code>REGENERATE_SKILLS_PROMPT</code>	与弱描述改写相关
<code>apps/backend/app/prompts/refinement.py</code>	<code>KEYWORD_INJECTION_PROMPT</code> 、 <code>VALIDATION_POLISH_PROMPT</code> 、AI 短语黑/白名单	多轮打磨

3.2 模板编写约束

`apps/backend/app/prompts/templates.py` 几乎所有 prompt 都遵守同一规则——JSON 输出模板中的 `{}` 必须写成 `{{}}`，否则 `str.format()` 会抛 `KeyError`。`EXTRACT_KEYWORDS_PROMPT` (`templates.py:179-196`)、`DIFF_IMPROVE_PROMPT` (`templates.py:457-527`)、所有 `enrichment.py` 里的 prompt 都是如此；`PARSE_RESUME_PROMPT` (`templates.py:154-177`) 是例外，它把 JSON 模板通过 `{schema}` 整体注入，因此不需要转义。

`templates.py:4-15` 维护 `LANGUAGE_NAMES` 字典，把 `en / es / zh / ja / pt` 映射到完整语言名，并提供 `get_language_name(code)` (`templates.py:13-15`)。`{output_language}` 占位符在所有 prompt 中都接收完整语言名（如 "Brazilian Portuguese"），而不是语言码。

3.3 三种改写策略

`templates.py:320-342` 定义 `IMPROVE_RESUME_PROMPTS = {nudge, keywords, full}`，每种对应不同 `CRITICAL_TRUTHFULNESS_RULES` (`templates.py:198-227`)：

prompt_id	风格	truthfulness 第 7 条
<code>nudge</code>	最小改动，只重述现有内容	"DO NOT add new bullet points or content"
<code>keywords</code>	在重述里织入关键词	"You may rephrase existing bullet points to include keywords"
<code>full</code>	可扩展/补充	"You may expand existing bullet points or add new ones that elaborate"

`DEFAULT_IMPROVE_PROMPT_ID = "keywords"` (`templates.py:344`)。`apps/backend/app/routers/resumes.py:67-71` 的 `_get_default_prompt_id` 从 `config.json` 读用户偏好，未设置时回落到 `keywords`。

3.4 diff 改写：让 LLM 返回"动作"而非全文

`DIFF_IMPROVE_PROMPT` (`templates.py:457-527`) 与三选一全文改写相比，关键差异是它只允许 LLM 输出 `changes` 数组，每条 change 都带 `path / action / original / value / reason`。`path` 仅限 4 类白名单 (`apps/backend/app/services/improver.py:80-85`)：

```

_ALLOWED_PATH_PATHS = [
    re.compile(r"^summary$"),
    re.compile(r"^workExperience\[d+\]\.description\[d+\]?"),
    re.compile(r"^personalProjects\[d+\]\.description\[d+\]?"),
    re.compile(r"^additional\.technicalSkills$"),
]

```

而 `personalInfo / customSections / years / company / institution` 等敏感字段被列入黑名单 (`improver.py:88-135`)。 `apply_diffs` 在落库前要过 4 道门:

1. path 在白名单;
2. path 不在黑名单;
3. path 实际能 resolve 到原值;
4. `replace` 类动作必须 `original` 文本与实际内容一致 (大小写不敏感比较, 见 `_verify_original_matches`, `improver.py:205-211`)。

这就把 LLM "幻觉式重写" 风险压到最低——就算模型给出 "把 Apple 改成 Google" 这种伪造信息, 也会被原始匹配失败这一关挡掉。

3.5 富集 (enrichment) 四件套

`apps/backend/app/prompts/enrichment.py` 维护 4 个 prompt, 结构非常工整:

- `ANALYZE_RESUME_PROMPT` (`enrichment.py:3-82`): 让模型审视简历, 输出 "待补强条目 + 6 个以内的引导问题", 每题附 `placeholder` 提示。
- `ENHANCE_DESCRIPTION_PROMPT` (`enrichment.py:84-125`): 基于用户回答, 追加新 bullet, 而不是替换; prompt 显式写 "ADD new bullet points ... DO NOT rewrite or replace existing bullets"。
- `REGENERATE_ITEM_PROMPT` (`enrichment.py:133-172`): 整段重写, 约束 "绝不引入新事实, 没有数字就强调定性影响"。
- `REGENERATE_SKILLS_PROMPT` (`enrichment.py:175-195`): 针对技能列表的重写。

四份 prompt 都强制 "输出 JSON only", 并通过 `system_prompt` 二次强调。 `enrichment.py:5`、`86`、`135`、`177` 都用 `{output_language}` 引导所有生成文本走多语言。

3.6 自定义 prompt: 占位符必须存在

`apps/backend/app/prompts/_init_.py:22-26` 定义了求职信/外链类 prompt 的强制占位符:

```

REQUIRED_FEATURE_PROMPT_PLACEHOLDERS = (
    "{job_description}", "{resume_data}", "{output_language}",
)

```

`validate_prompt_placeholders` (`_init_.py:29-39`) 在 `PUT /config/feature-prompts` 写入前校验, 缺失就 422。
`apps/backend/app/services/cover_letter.py:17-33` 的 `_resolve_feature_prompt` 在运行时再兜一次: 自定义 prompt `.format()` 失败就回落默认模板并 `logging.warning`, 避免磁盘被手工改坏导致整个生成崩溃。

4. 端到端工作流

下面把 `POST /api/v1/resumes/improve/preview` 走通, 看 LLM 在每个环节如何被调用。

4.1 上传与解析

`apps/backend/app/routers/resumes.py:511-594` 的 `upload_resume`:

1. 校验类型 (PDF/DOC/DOCX, <=4 MB, `resumes.py:503-509`);
2. `parse_document` (`apps/backend/app/services/parser.py:119-141`) 用 `markdown` 把文件转 Markdown;
3. `parse_resume_to_json` (`parser.py:144-176`) 用 `complete_json` + `PARSE_RESUME_PROMPT` 让 LLM 抽结构化 JSON;
4. `restore_dates_from_markdown` (`parser.py:40-116`) 把 markdown 里出现的 `Jan 2020 - Dec 2023` 反向补回被 LLM 误删月份的 year-only 字段;
5. `ResumeData.model_validate` 做 Pydantic 二次校验。

4.2 关键词抽取

`apps/backend/app/services/improver.py:555-572` 的 `extract_job_keywords` 用 `EXTRACT_KEYWORDS_PROMPT`, 产出 `required_skills / preferred_skills / experience_requirements / education_requirements / key_responsibilities / keywords / experience` 字典。
`routers/resumes.py:728-749` 用 `job_id` + `job_keywords_hash` 缓存, 重复 tailor 不必二次 LLM 调用。

4.3 改写主流程 (diff 模式)

`routers/resumes.py:719-960` 的 `_improve_preview_flow` 是核心:

- 1. **Skill 计划** (`improver.py:785-835`): 先 `generate_skill_target_plan` 问 LLM "这份简历要强调哪些技能", 再用 `verify_skill_target_plan` (`improver.py:705-782`) 把建议归为 accepted/rejected; 只有 `existing` / `jd_added` / `supported_by_resume` 三个 source 才允许注入。
- 2. **生成 diff** (`improver.py:457-552`): 用 `DIFF_IMPROVE_PROMPT` 让 LLM 只输出 `changes[]`。
- 3. **应用 diff** (`improver.py:214-361`): 四道门 + `apply_diffs`; 不合规的 change 全部进 `rejected_changes` 列表返回给前端。
- 4. **质量校验** (`improver.py:381-454`): `verify_diff_result` 检查 section 数量、身份字段、字数膨胀率、新增 metrics 是否在原文中存在。
- 5. **本地安全网** (`routers/resumes.py:832-844`): `_preserve_personal_info` / `_restore_original_dates` / `restore_dates_from_markdown` / `_preserve_original_skills` / `_protect_custom_sections` ——即便 diff 通过, 也强制把"个人姓名 / 关键日期 / 原始技能"按原值回填。

4.4 多轮打磨

`refine_resume` (`apps/backend/app/services/refiner.py:76-178`) 串联三步:

- 1. **关键词注入** (`refiner.py:452-512`): 把 `injectable_keywords` 通过 `KEYWORD_INJECTION_PROMPT` (`apps/backend/app/prompts/refinement.py:138-159`) 织入现有 bullet。 `_validate_resume_structure` (`refiner.py:429-449`) 在落库前再做一次结构校验。
- 2. **AI 短语清除** (`refiner.py:233-287`): 本地 `re.sub` 把 `AI_PHRASE_BLACKLIST` (如 "spearheaded"、"synergy"、"utilized") 替换为 `AI_PHRASE_REPLACEMENTS` (如 "led"、"collaboration"、"used"), 但跳过在 JD 文本中出现的短语以免误伤行业术语。
- 3. **Master 对齐校验** (`refiner.py:290-408`): 检查 `technicalSkills` / `certificationsTraining` / `company` 三个字段在 master 中存在, 否则按 `critical` violation 处理; `fix_alignment_violations` (`refiner.py:515-562`) 删除虚构内容。

4.5 求职信/外联/标题

`apps/backend/app/services/cover_letter.py`:

- `generate_cover_letter` (`cover_letter.py:36-87`): 用 `COVER_LETTER_PROMPT` (`templates.py:349-371`), 要求 100-150 词、3-4 段、自定义 prompt 可覆盖。
- `generate_outreach_message` (`cover_letter.py:90-136`): 70-100 词, 更口语化。
- `generate_resume_title` (`cover_letter.py:139-168`): 用 `temperature=0.3, max_tokens=60` 让模型严格按 "Role @ Company" 格式输出。

`routers/resumes.py:446-498` 的 `_generate_auxiliary_messages` 用 `asyncio.gather` 并行触发, 单点失败不影响其它 (异常被收集进 `warnings`)。

4.6 富集 (enrichment) 支线

`apps/backend/app/routers/enrichment.py:87-169` 的 `analyze_resume` 是入口:

- 拉简历 → `ANALYZE_RESUME_PROMPT` 拿 `items_to_enrich` / `questions` / `analysis_summary`;
- 单题 180 秒超时 (`enrichment.py:118-121`), 失败分别映射 504 / 422 / 500。
- 用户回答后 `/enhance` (`enrichment.py:172-325`) 按 `item_id` 聚合答案, **fast path** 直接从 `processed_data` 取条目信息 (`enrichment.py:43-84`), 避免重新跑 LLM 分析。
- `/regenerate` (`enrichment.py:487-544`) 用 `asyncio.gather` 并行处理多个条目, 单条失败只记入 `errors` 列表, 不阻塞其它。

4.7 时间线总结

阶段	入口	LLM 角色	输出校验
上传	<code>POST /resumes/upload</code>	<code>parse_resume_to_json</code>	<code>ResumeData</code> + markdown 日期回填
富集	<code>POST /enrichment/analyze/{id}</code>	<code>ANALYZE_RESUME_PROMPT</code>	Pydantic
改写	<code>POST /resumes/improve/preview</code>	<code>EXTRACT_KEYWORDS_PROMPT</code> → <code>SKILL_TARGET_PLAN_PROMPT</code> → <code>DIFF_IMPROVE_PROMPT</code>	diff 4 道门 + 安全网 + <code>refine_resume</code>
确认	<code>POST /resumes/improve/confirm</code>	不直接调 LLM, 但调用 <code>_generate_auxiliary_messages</code>	hash 一致性 + <code>personalInfo</code> 不变
收尾	<code>POST /resumes/{id}/generate-cover-letter</code> 等	<code>COVER_LETTER_PROMPT</code> / <code>OUTREACH_MESSAGE_PROMPT</code> / <code>GENERATE_TITLE_PROMPT</code>	prompt 校验 + 异常降级

5. 错误处理与重试机制

层	机制	触发条件	行为
传输层	<code>Router.num_retries=3</code> + <code>RetryPolicy</code>	429 / 500 / Timeout	LiteLLM 内部指数退避
应用层	<code>complete_json(retries=2)</code>	JSON 解析失败 / 截断	改温度重试 + 关闭 JSON mode 兜底
业务层	<code>asyncio.wait_for(timeout=...)</code>	LLM 拖死	路由层返回 504
业务层	异常日志 + 通用 500	未知	<code>logger.error(...)</code> + <code>raise HTTPException(500, "... Please</code>
安全层	<code>_sanitize_user_input</code>	JD 文本含 <code>ignore previous instructions</code> 等	替换为 <code>[REDACTED]</code> (<code>apps/backend/app/services/improver.</code>)
提示层	<code>{critical_truthfulness_rules}</code>	全文改写 prompt	强制 "DO NOT add new skill ..."
数据层	<code>_validate_resume_structure</code> / <code>_preserve_personal_info</code> / <code>_restore_original_dates</code>	LLM 返回值结构错误	局部回退到原值

容错策略的核心思路是：把"内容质量"问题留在应用层，把"传输"问题交给 Router。`llm.py:1150-1154` 的注释明确写 "Transport errors — Router already retried with backoff. Cooldowns are disabled ... no additional retry is attempted here"。

`apps/backend/app/services/refiner.py:411-426` 的 `_prepare_job_description` 把超过 2000 字符的 JD 截断并打 warning，防止 prompt 过长被 provider 直接拒收。`MAX_JSON_CONTENT_SIZE = 1 MB` (`llm.py:45`) 防止 JSON 抽取时无限递归或被巨大响应拖死。

6. 如何添加新的 LLM provider

假设要加一个 `mistral` provider，步骤如下：

- 环境变量白名单：`apps/backend/app/config.py:125-134` 把 `mistral` 加入 `llm_provider` 的 `Literal`。
- Provider 路由：`apps/backend/app/llm.py:302-311` 的 `_PROVIDER_KEY_MAP` 增加 `"mistral": "mistral"`。
- 模型名前缀：`apps/backend/app/llm.py:410-422` 的 `provider_prefixes` 加 `"mistral": "mistral/"`。
- base URL 归一化：`apps/backend/app/llm.py:64-114` 的 `_normalize_api_base` 检查 mistral 是否需要去 `/v1` (LiteLLM 内部会拼)。
- 环境回退：`apps/backend/app/llm.py:318-320` 的 `_PROVIDERS_WITHOUT_ENV_KEY_FALLBACK` 视情况加入 (mistral 是云端，不应跳过环境回退)。
- 健康检查特判：若 mistral 的本地代理存在空 key 场景，可仿照 ollama 在 `llm.py:543-549` 增加豁免。
- 超时系数：`apps/backend/app/llm.py:915-921` 的 `provider_factors` 加入 mistral 经验延迟系数。
- 文档：`docs/agent/llm-integration.md:5-17` 的 provider 表补一行。

完成后调用 `await get_llm_config()` 即可返回新的 `LLMConfig`，不需要改任何业务代码。

7. 关键 Python 调用示例

7.1 健康检查 + 文本补全

```
# app/routers/config.py 风格的用法
from app.llm import check_llm_health, complete

# 1) 验证当前 provider 可用
result = await check_llm_health(include_details=True)
assert result["healthy"], result

# 2) 让模型生成一段 200 字内的自我介绍
text = await complete(
    prompt="用两句话介绍 FastAPI 与 LiteLLM 的关系。",
    system_prompt="你是一名技术布道师，输出简体中文。",
    max_tokens=256,
    temperature=0.5,
)
print(text)
```

7.2 结构化 JSON 抽取 + 自动重试

```
# 与 improver.extract_job_keywords 等价的最小用法
import json
from app.llm import complete_json
from app.prompts import EXTRACT_KEYWORDS_PROMPT

prompt = EXTRACT_KEYWORDS_PROMPT.format(
    job_description="我们正在招聘 5 年以上经验的 Python 后端，要求熟悉 FastAPI、PostgreSQL、Kubernetes。"
)
keywords = await complete_json(
    prompt=prompt,
    system_prompt="你是职位描述分析助手。",
    max_tokens=2048,
    schema_type="keywords", # 关闭 _appears_truncated 启发式
)
print(json.dumps(keywords, ensure_ascii=False, indent=2))
# 期望输出形如:
# {
#   "required_skills": ["Python", "FastAPI", "PostgreSQL", "Kubernetes"],
#   "preferred_skills": [...],
#   "experience_years": 5,
#   "seniority_level": "senior",
#   ...
# }
```

7.3 用 Router 做带退避的批量调用

```
import asyncio
from app.llm import get_router, get_llm_config, get_model_name

router, config = get_router()
model_name = get_model_name(config)

async def one_call(prompt: str) -> str:
    resp = await router.acompletion(
        model="primary",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=512,
        timeout=120,
    )
    return resp.choices[0].message.content

# 5 个并发任务由 Router 统一做 429/500/timeout 重试
results = await asyncio.gather(*[one_call(f"解释 #{i} 号概念") for i in range(5)])
```

7.4 自定义求职信 prompt 校验

```
# app/prompts/_init_.py:validate_prompt_placeholders
from app.prompts import validate_prompt_placeholders

user_prompt = """
Write a friendly cover letter.
Job: {job_description}
Me: {resume_data}
Language: {output_language}
"""

missing = validate_prompt_placeholders(user_prompt)
assert not missing, f"占位符缺失: {missing}"
# 若 user_prompt 留空 (仅含空白), validate_prompt_placeholders 返回 [],
# 表示 "use default", 无需报错。
```

8. 提示词工程最佳实践（基于本项目代码事实）

- JSON 模板中的 `{}` 必须转义为 `{{}}`，否则 `str.format()` 直接抛 `KeyError`（参考 `enrichment.py` 全文件）。
- 把“绝不能做的事”放在 prompt 顶部，并且用 `CRITICAL` 字样强化：例如 `templates.py:198-227` 的 `CRITICAL_TRUTHFULNESS_RULES`，被注入到全文改写 prompt 的开头。
- 提供完整的 JSON Schema 示例：`RESUME_SCHEMA_EXAMPLE` / `IMPROVE_SCHEMA_EXAMPLE`（`templates.py:19-152`）让模型对字段名、层级、ID 编号规则有显式参考。
- 用 enum-like 字符串约束动作：`DIFF_IMPROVE_PROMPT` 把 `action` 限定为 `replace / append / reorder / add_skill`，便于服务端 `apply_diffs` 走 switch 校验。
- 对用户输入做防注入：`improver._sanitize_user_input`（`improver.py:48-56`）用 8 条正则替换 `ignore previous instructions` 等典型注入串；这块要写进 `extract_job_keywords` 和 `generate_resume_diffs` 入口。
- 应用层与传输层重试分离：transport 重试交给 LiteLLM Router，内容质量（JSON 解析、截断）由 `complete_json` 自己重试，调用方不重复做。`llm.py:1150-1154` 是这一原则的注释化总结。
- 能力探测走 Registry 而非硬编码：`_supports_json_mode` / `_supports_temperature` / `get_safe_max_tokens` 都调用 `litellm.get_model_info`，新增模型时只要 LiteLLM 注册表更新，本项目即可自动跟上。
- 本地安全网（defense in depth）：即便 diff 通过，仍跑 `_preserve_personal_info` / `_restore_original_dates` / `_preserve_original_skills` / `_protect_custom_sections` 五个本地回填函数（`routers/resumes.py:832-844`），把“个人姓名 / 日期 / 技能 / 自定义章节”硬性恢复。
- schema 感知的截断判断：`_appears_truncated(data, schema_type)` 根据是 `resume / enrichment / diff / keywords` 走不同启发式，避免把合法空 diff 误判。
- 多语言通过 `{output_language}` 注入完整语言名（`templates.py:13-15`），让模型翻译自己的“措辞风格”，比加一句“respond in English”更稳定。
- 温度随重试递增（`_get_retry_temperature`，`llm.py:879-894`）以 0.1→0.3→0.5→0.7 的梯度跳变，让二次抽样的输出与首轮尽量不一样，提升从 malformed 状态逃出的概率。
- 错误日志先详细、再返回通用消息：`llm.py:693-699`、`routers/resumes.py:126-127` 注释化的“Log details server-side, return generic messages to clients”是必须遵守的契约。
- prompt 中显式禁止 em-dash：从 `IMPROVE_RESUME_PROMPT_NUDGE`（`templates.py:246`）到 `OUTREACH_MESSAGE_PROMPT`（`templates.py:391`）都在末尾强调“Do NOT use em dash”。`AI_PHRASE_REPLACEMENTS`（`refinement.py:131-133`）还把 em-dash 在本地替换成逗号。
- 多轮打磨用本地 + LLM 组合：能本地 `re.sub` 的（AI 短语替换、master 对齐校验）就别再调 LLM，节省 token；必须 LLM 的（关键词注入）才走 `complete_json`。
- preview + confirm 两阶段协议：`POST /resumes/improve/preview` 生成 `preview_hash` 写回 job；`POST /resumes/improve/confirm` 校验 hash + `personalInfo` 不变才允许落库（`routers/resumes.py:963-1025`）。这是防止客户端篡改 LLM 输出的一道防线。

9. 关键文件速查

关注点	文件	关键行
Provider 路由	apps/backend/app/llm.py	302-311 (map)、403-448 (get_model_name)
Router 与重试	apps/backend/app/llm.py	472-526
JSON 抽取	apps/backend/app/llm.py	941-1023
JSON 补全 + 截断检测	apps/backend/app/llm.py	1026-1156
API key 防泄漏	apps/backend/app/llm.py	286-349、617-648
改写 prompt	apps/backend/app/prompts/templates.py	198-348
Diff prompt + 策略	apps/backend/app/prompts/templates.py	416-527
富集 prompt	apps/backend/app/prompts/enrichment.py	3-195
关键词注入 / 短语黑名单	apps/backend/app/prompts/refinement.py	4-159
解析服务	apps/backend/app/services/parser.py	119-176
改写主流程	apps/backend/app/services/improver.py	457-552、705-835
Diff 应用	apps/backend/app/services/improver.py	80-361
多轮打磨	apps/backend/app/services/refiner.py	76-562
求职信 / 外联 / 标题	apps/backend/app/services/cover_letter.py	36-168
富集路由	apps/backend/app/routers/enrichment.py	87-544
改写路由	apps/backend/app/routers/resumes.py	719-1331
配置 schema	apps/backend/app/config.py	84-225

10. 小结

Resume Matcher 在 LLM 集成上展示了几个值得借鉴的工程原则：**统一传输层**（LiteLLM Router + 指纹缓存）、**能力探测走 Registry**、**内容质量与应用层重试解耦**、**JSON-mode→prompt-only 兜底**、**diff 改写 + 路径白/黑名单 + 本地安全网的多层防护**、**多轮打磨用本地 + LLM 组合**、**preview/confirm 两阶段协议**。把这些"基础设施"做到 `llm.py` 一个文件里，业务侧只需关心 prompt 模板与编排逻辑，迭代速度非常快。

下一步建议：阅读 `apps/backend/app/services/improver.py` 的 diff 4 道门与 `apps/backend/app/services/refiner.py` 的三段打磨，对照本教程第 4 节画时序图，会对整个 AI 集成链路有更直观的掌握。

06 - 前端架构：Resume Matcher 的 Next.js 16 + React 19 实现

本篇聚焦于 `apps/frontend` 目录,系统化梳理其技术栈选型、目录结构、路由组织、配置文件、状态管理与样式系统,帮助读者理解整个前端工程的骨架。

一、整体技术栈

Resume Matcher 前端是单仓多包 (monorepo) 中独立运行的 Next.js 应用,版本与工具链如下:

维度	选型	版本	关键作用
框架	Next.js	16.2.6	App Router + Turbopack
视图层	React	19.2.4	Server Components、use Hooks
语言	TypeScript	5 (strict)	类型安全
样式	Tailwind CSS	v4	CSS-first 配置,无 <code>tailwind.config.js</code>
富文本	Tiptap	3.20	简历编辑
拖拽	@dnd-kit	6/10/3	章节排序
测试	Vitest + Testing Library	4.0	jsdom 环境
图标	lucide-react	0.575	通过 deep path 直接 import 避免 barrel

`apps/frontend/package.json:13-29` 列出完整依赖。 `apps/frontend/package.json:5-12` 定义脚本:

```
"dev": "next dev --turbopack",
"build": "next build",
"lint": "eslint .",
"format": "prettier --write .",
"test": "vitest run"
```

注意 `dev` 使用 Turbopack 启动, `build` 仍然使用 `next build` (默认 Webpack),二者都启用 App Router。

二、目录结构

`apps/frontend/CLAUDE.md:30-59` 描述了完整目录布局,关键路径:

```
app/                # 路由(全部由 App Router 接管)
  (default)/        # 路由组,包主应用 + Provider
  print/            # 服务端渲染的 PDF 入口
components/         # UI 组件,按功能域拆分
  ui/ builder/ dashboard/ tailor/ enrichment/ resume/ preview/ home/ settings/ common/
lib/                # 数据层与基础设施
  api/              # 后端 HTTP 客户端
  context/          # 全局 React Context
  i18n/             # 翻译引擎
  utils/ utils.ts   # 工具函数
  types/            # 共享类型
  config/ constants/ # 配置与常量
hooks/              # 自定义 React Hook
i18n/config.ts      # locale 元数据(注意:与 lib/i18n 不同目录)
messages/           # 各语言 JSON 翻译
tests/              # vitest 单测
```

`apps/frontend/tsconfig.json:21-23` 设置路径别名:


```
"paths": {
  "@/*": ["./*"]
}
```

任意 `import { x } from '@lib/api/client'` 都解析为 `apps/frontend/lib/api/client.ts`。

三、路由组织

3.1 路由组 (default)

`app/(default)/` 是 Next.js 16 App Router 的路由组 (括号包裹的目录不会出现在 URL 中)。`apps/frontend/app/(default)/layout.tsx:1-19` 将所有受 Provider 包裹的页面收纳在同一棵布局树下:

```
<StatusCacheProvider>
  <LanguageProvider>
    <ResumePreviewProvider>
      <LocalizedErrorBoundary>
        <main className="min-h-screen flex flex-col">{children}</main>
      </LocalizedErrorBoundary>
    </ResumePreviewProvider>
  </LanguageProvider>
</StatusCacheProvider>
```

`StatusCacheProvider` 负责缓存 `/status` (`apps/frontend/lib/context/status-cache.tsx:39-216`), `LanguageProvider` 维护 UI/ 内容语言 (`apps/frontend/lib/context/language-context.tsx:26-113`), `ResumePreviewProvider` 与 `LocalizedErrorBoundary` 分别来自 `components/common/`。`apps/frontend/app/layout.tsx:5-15` 在根布局加载 Geist + Space Grotesk 字体并设置 `<html lang="en-US">`, 是整棵布局树的最外层。

3.2 页面清单

路由	文件	渲染方式	用途
<code>/</code>	<code>app/(default)/page.tsx:3-5</code>	Server	落地页, 直接渲染 <code><Hero/></code>
<code>/dashboard</code>	<code>app/(default)/dashboard/page.tsx:33-42</code>	Client	主面板, resume 列表、上传、删除、重试、状态卡
<code>/builder</code>	<code>app/(default)/builder/page.tsx:3-5</code>	Server 包装	委托给 <code>components/builder/resume-builder.t</code>
<code>/tailor</code>	<code>app/(default)/tailor/page.tsx:24-504</code>	Client	粘贴 JD → 触发 improve preview → 弹 diff modal
<code>/settings</code>	<code>app/(default)/settings/page.tsx:104-112</code>	Client	LLM 配置、特性开关、提示词、Danger Zone
<code>/resumes/[id]</code>	<code>app/(default)/resumes/[id]/page.tsx</code>	Client	单个简历查看、下载、改名、AI 增强

3.3 动态路由 [id]

`/resumes/[id]` 与 `/print/*` 都使用 Next.js 16 的 `Promise<{...}>` 形态参数。`apps/frontend/app/print/resumes/[id]/page.tsx:17-38` 给出了示例:

```
type PageProps = {
  params: Promise<{ id: string }>;
  searchParams?: Promise<{ template?: string; pageSize?: string; ... }>;
};
```

Next.js 15+ 改为异步 `params` / `searchParams`, 所有页面必须用 `await` 解析。

3.4 服务端 `print/*` 子树

`apps/frontend/CLAUDE.md:26` 明确指出: 打印页是有意保留的 Server Component。它们没有 Provider 包裹, 通过 `API_BASE` + `lib/i18n/server.ts:translate` 直接在后端渲染并交给 headless Chromium 转 PDF。这种"分叉的 App Router 子树"是 Next.js 16 一个非常实用的模式。

四、关键配置文件解析

4.1 next.config.ts

apps/frontend/next.config.ts:1-45 的核心要点:

1. 环境变量决定后端地址: `BACKEND_ORIGIN` 默认为 `http://127.0.0.1:8000` (`next.config.ts:3`)。
2. standalone 输出: `output: 'standalone'` (`next.config.ts:6`) 启用独立部署模式,适合 Docker。
3. 超时与 tree-shaking: `experimental.proxyTimeout: 240_000` (`next.config.ts:8`) 与 `optimizePackageImports` (`next.config.ts:9-19`) 联合把 lucide/typewriter/dnd-kit 拆解为按需 import,缩短冷启动 200~800ms。
4. rewrites 代理: `next.config.ts:21-42` 将 `/api/*`、`/docs`、`/redoc`、`/openapi.json` 全部代理到 FastAPI 后端。注释 `next.config.ts:22-23` 警告:"filesystem 路由优先",禁止创建 `app/api/*` 路由,否则会遮蔽代理。

4.2 tsconfig.json

apps/frontend/tsconfig.json:1-33 启用 `strict: true`、`moduleResolution: "bundler"`、`jsx: "react-jsx"`、路径别名 `@/*`。`include` 字段 (`tsconfig.json:25-31`) 涵盖 `.next/dev/types/**/*.ts`,这意味着 Next.js 16 的 dev 类型(基于 `typedRoutes`)会自动被纳入编译。

4.3 Tailwind v4 与 PostCSS

Tailwind v4 取消了 `tailwind.config.js`,改为 CSS-first 配置。`apps/frontend/postcss.config.mjs:1-7` 仅注册 `@tailwindcss/postcss` 插件:

```
export default { plugins: { '@tailwindcss/postcss': {} } };
```

真正的 token 来自 `app/(default)/css/globals.css` 的 `@theme inline {}` 块(由 `app/layout.tsx:3` import)。`apps/frontend/CLAUDE.md:117-129` 列出 Swiss Design token:

Token	值	Tailwind 类
Canvas	#F0F0E8	bg-background bg-canvas
Ink	#000000	text-ink text-ink-soft
Hyper Blue	#1D4ED8	text-primary bg-primary
Signal Green	#15803D	text-success
Alert Orange	#F97316	text-warning
Alert Red	#DC2626	text-destructive

规范要求 `rounded-none` 全局生效、1px 黑边、硬阴影 `shadow-sw-xs..xl` (实心墨色无模糊)。`apps/frontend/CLAUDE.md:128` 强调"硬阴影是 Swiss 设计的特征"。

4.4 ESLint 与 Prettier

apps/frontend/eslint.config.mjs:1-35 合并了 `eslint-config-next/core-web-vitals`、`/typescript`、`eslint-config-prettier`,并在 `rules` (`eslint.config.mjs:25-29`) 中关闭 `react-hooks/set-state-in-effect` —— 注释说明现有代码依赖 effect 中 `setState` 同步 props 与 DOM 测量。Prettier 通过 ESLint 的 `prettier/prettier: 'error'` 强制。

4.5 Vitest

apps/frontend/vitest.config.ts:1-21 配置 jsdom 环境、`@vitejs/plugin-react`、`./vitest.setup.ts`,并复用 `@/*` 别名。`vitest.setup.ts` 注入 `@testing-library/jest-dom`。

五、状态管理方案

项目没有引入 Redux/Zustand/Jotai,只使用 React Context + 自定义 Hooks,具体分层如下:

层	来源	关注点
全局只读缓存	<code>StatusCacheProvider</code> (<code>lib/context/status-cache.tsx:39-216</code>)	30 min 自动刷新 <code>/status</code> 、乐观计数器 <code>incrementResumes</code> 等
双向同步	<code>LanguageProvider</code> (<code>lib/context/language-context.tsx:26-113</code>)	UI 语言走 localStorage,内容语言走 backend
预览数据	<code>ResumePreviewProvider</code> (<code>components/common/resume_previewer_context.tsx</code>)	<code>/tailor</code> 写入、 <code>/builder</code> 读取
流程状态	自定义 Hook(<code>hooks/use-enrichment-wizard.ts</code> 、 <code>hooks/use-regenerate-wizard.ts</code>)	useReducer 或 useState 局部管理
本地存储	<code>localStorage</code> (<code>master_resume_id</code> 、 <code>resume_builder_draft</code> 等)	客户端持久化

`apps/frontend/CLAUDE.md:77-79` 写明:"StatusCache is React Context, not a fetch lib",这是项目有意选择 Context 的原因 —— Context 是稳定订阅通道,而 React Query/SWR 等数据层库会引入额外的失效/重试策略,与 30min 缓存粒度不匹配。

六、样式与设计系统

`apps/frontend/CLAUDE.md:112-130` 强制要求所有 UI 走 Swiss International Style:

- **字体**:标题用 `font-serif`,正文用 `font-sans` (Geist),元数据用 `font-mono` (Space Grotesk)。`app/layout.tsx:5-15` 通过 `next/font/google` 注入 CSS 变量。
- **形状**:`rounded-none` 全局生效,无圆角。
- **边框与阴影**:1px 黑色实边 + 硬阴影(`shadow-sw-*`),无模糊、无渐变。
- **图标**:lucide-react,深路径 import(如 `lucide-react/dist/esm/icons/loader-2`),避免 barrel re-export 拖慢打包。`next.config.ts:9-19` 同步配置 `optimizePackageImports` 作为兜底 tree-shake。
- **打印**:`components/resume/styles/*.module.css` 自带 `_tokens.css`、`swiss-single`、`swiss-two-column`、`modern` 等 CSS Module,专供 print 路由使用。

七、构建与开发命令

命令	行为	备注
<code>npm install</code>	安装前端依赖	必须先于其它命令
<code>npm run dev</code>	<code>next dev --turbo</code> 启动:3000	后端必须独立在:8000
<code>npm run build</code>	触发 <code>tsc</code> + <code>next build</code>	i18n 结构漂移会在此 fail
<code>npm run start</code>	生产模式启动	需要先 build
<code>npm run lint</code>	<code>eslint .</code>	含 Prettier 规则
<code>npm run format</code>	<code>prettier --write .</code>	提交前必跑
<code>npm run test</code>	<code>vitest run</code>	jsdom + Testing Library

后端代理逻辑见 `next.config.ts:21-42`。前端**不会**暴露 8000 端口给浏览器,所有 `/api/*` 通过 Next.js 自身服务器(3000)代理,降低 CORS 复杂度。

八、整体架构图

```
flowchart TB
    Browser[浏览器 :3000] --> NextApp[Next.js App Router]
    NextApp --> Layout1[app/layout.tsx<br/>fonts + globals.css]
    NextApp --> GroupLayout[app/(default)/layout.tsx]
    GroupLayout --> PrintLayout[app/print/.../page.tsx<br/>Server Component]

    GroupLayout --> StatusCtx[StatusCacheProvider]
    GroupLayout --> LangCtx[LanguageProvider]
    GroupLayout --> ResumeCtx[ResumePreviewProvider]
    GroupLayout --> ErrorBoundary[LocalizedErrorBoundary]

    subgraph Pages["Client 页面"]
        Dashboard[/dashboard/]
        Builder[/builder/]
        Tailor[/tailor/]
        Settings[/settings/]
        ResumesId[/resumes/[id]/]
    end

    GroupLayout --> Pages
    PrintLayout --> PrintPages[/print/resumes/[id]/<br/>/print/cover-letter/[id]/]

    Pages --> Hooks[hooks/<br/>useFileUpload / useEnrichmentWizard / useRegenerateWizard]
    Pages --> API[lib/api/*<br/>client.ts / resume.ts / config.ts / enrichment.ts]
    Hooks --> API

    API -->|fetch + AbortController| Rewrite[next.config.ts rewrites]
    Rewrite -->|proxy| FastAPI[FastAPI :8000]

    FastAPI --> TinyDB[(TinyDB<br/>data/storage.json)]
    FastAPI --> LiteLLM[LiteLLM<br/>OpenAI/Anthropic/...]
```

九、关键设计取舍

1. **Turbopack dev + Webpack build**: Turbo 提升 HMR 速度,但 `next build` 仍由 Webpack 执行,这意味着线上打包路径与开发路径略有不同,遇到 issue 时要分头排查。
2. **Client 优先、Print 例外**: `(default)/layout.tsx` 用 Provider 树包装,而 `print/*` 全部保持 Server 渲染,便于 headless Chromium 准确抓取。这是一种"局部 Server、局部 Client"的分叉。
3. **i18n 必须结构对齐**: `lib/i18n/messages.ts:11-17` 把所有 locale 强制 cast 为 `Messages = typeof en`,任何对 `en.json` 的增删都必须镜像到其他 4 个文件,否则 `npm run build` 失败。
4. **路径别名 + 单一 client.ts**: `lib/api/client.ts:43-74` 的 `apiFetch` 是 240s 超时 + 友好错误的统一封装,所有页面与 Hook 都通过它访问后端,避免在组件里直接 `fetch('/api/...')`。
5. **本地存储是 Source of Truth 的一部分**: `master_resume_id`、`resume_builder_draft` 等 localStorage 字段(`apps/frontend/CLAUDE.md:97-102`)是约定俗成的"客户端主键",删除/重置时需主动清理(见 `app/(default)/settings/page.tsx:536-541` 的 `handleResetDatabase`)。

十、阅读路线建议

- 先看 `app/layout.tsx` 与 `app/(default)/layout.tsx`,理解 Provider 顺序。
- 再读 `lib/api/client.ts`,把 240s 超时 + 内部 origin 切换的逻辑搞清楚。
- 然后读 `app/(default)/dashboard/page.tsx:33-180`,掌握 `useStatusCache` + 列表加载模式。
- 最后看 `app/print/resumes/[id]/page.tsx` 的 server-side 渲染流程,理解 print 子树的特殊之处。

理解以上六层结构后,继续阅读 [07-前端数据层与Hooks](#) 即可掌握具体实现细节。

07 - 前端数据层与 Hooks:API 客户端、Context 与自定义 Hook 详解

本篇深入 `apps/frontend/lib` 与 `apps/frontend/hooks` 两个目录,系统讲解 API 客户端封装、Context 状态管理、三个核心自定义 Hook,并辅助介绍类型、常量与 `i18n` 模块。所有引用均给出 `文件路径:行号` 便于回查。

一、API 客户端:统一 fetch 封装

1.1 `lib/api/client.ts` 的设计要点

`apps/frontend/lib/api/client.ts:7-33` 定义两个运行时常量:

```
const DEFAULT_PUBLIC_API_URL = '/';
const INTERNAL_API_ORIGIN = 'http://127.0.0.1:8000';

export const API_URL = normalizeApiUrl(process.env.NEXT_PUBLIC_API_URL ?? DEFAULT_PUBLIC_API_URL);
export const API_BASE = resolveRuntimeApiBase(toApiBase(API_URL));
```

`resolveRuntimeApiBase` (`client.ts:25-30`) 是核心 trick:当运行在浏览器端时,`API_BASE` 保持相对路径 (`/api/v1`),由 `next.config.ts:21-42` 的 `rewrites` 转发;运行在服务端(Server Component / `print/*`)时,把相对路径改为 `http://127.0.0.1:8000/api/v1`,绕过 Next.js 代理直接打后端。这就是 `print` 页面不依赖浏览器 cookie 也能拉取数据的机制。

`apps/frontend/lib/api/client.ts:43-74` 的 `apiFetch` 是统一的 fetch wrapper:

```
export async function apiFetch(endpoint, options?, timeoutMs?): Promise<Response> {
  // 1. 规范化 URL (补前导斜杠, 识别绝对 URL 与 /api 路径)
  // 2. 默认 timeout = 240_000ms (与后端 resumes.py wait_for 保持一致)
  // 3. 使用 AbortController 取消超时请求
  // 4. AbortError 转译为 'Request timed out. Please try again...'
}
```

`client.ts:79-118` 在此基础上封装了 `apiPost` / `apiPatch` / `apiPut` / `apiDelete`, 这些 helper 不自动 JSON 化返回值,调用方需自行 `await res.json()` —— 这是项目刻意保留的"低级但灵活"的 API,例如 `lib/api/resume.ts:122-135` 在 `improveResume` 中用 `response.text()` 自己做 JSON 解析,避免空响应导致 `res.json()` 抛错。

1.2 业务 API 模块

文件	范围	关键函数
<code>lib/api/resume.ts</code>	简历与岗位 CRUD、AI 改善、PDF 下载	<code>uploadJobDescriptions</code> 、 <code>previewImproveResume</code> 、 <code>confirmImproveResume</code> 、 <code>fetchResume</code> 、 <code>updateResume</code> 、 <code>downloadResumePdf</code> 、 <code>deleteResume</code> 、 <code>renameResume</code> 、 <code>retryProcessing</code> 、 <code>fetchJobDescription</code>
<code>lib/api/config.ts</code>	LLM / 功能 / 提示词 / 语言配置	<code>fetchLlmConfig</code> 、 <code>updateLlmConfig</code> 、 <code>testLlmConnection</code> 、 <code>fetchSystemStatus</code> 、 <code>PROVIDER_INFO</code> 、 <code>fetchFeaturePrompts</code> 、 <code>updateFeaturePrompts</code> (422 抛 <code>FeaturePromptsError</code> , <code>config.ts:310-318</code>)、 <code>clearAllApiKeys</code> 、 <code>resetDatabase</code>
<code>lib/api/enrichment.ts</code>	AI 增强 + AI 重新生成	<code>analyzeResume</code> 、 <code>generateEnhancements</code> 、 <code>applyEnhancements</code> 、 <code>regenerateItems</code> 、 <code>applyRegeneratedItems</code>
<code>lib/api/index.ts</code>	barrel re-export	注意只 re-export 公共部分,部分函数仍需直接 import 子模块

`lib/api/resume.ts:111-135` 的 `postImprove` 是个特殊 wrapper:它在 `response.text()` 拿到原文后再 `JSON.parse`,这样既能拿到非 2xx 的服务端详细错误,也能容错处理空响应。`lib/api/resume.ts:218-252` 的 `getResumePdfUrl` 把 `TemplateSettings` 序列化为 URL query string,后端 Playwright 读取后渲染 PDF。

1.3 类型安全与错误处理

每个业务模块都在文件顶部定义与后端 Pydantic 一一对应的 interface(例如 `lib/api/resume.ts:8-50` 的 `ProcessedResume`、`lib/api/config.ts:42-48` 的 `SystemStatus`)。错误路径采用一致的 `data.detail || 'Fallback (status X).'` 模式,如 `lib/api/config.ts:91-93`:

```
if (!res.ok) {
  const data = await res.json().catch(() => ({}));
  throw new Error(data.detail || `Failed to update LLM config (status ${res.status}).`);
}
```

`lib/api/config.ts:328-369` 的 `updateFeaturePrompts` 是更精细的例子:它在 422 + `code: 'missing_placeholders'` 时抛 `FeaturePromptsError`,让 UI 能精确定位到哪个字段缺了哪个占位符。

二、Context 提供者

2.1 StatusCacheProvider

`apps/frontend/lib/context/status-cache.tsx:39-216` 是项目里最重的 Context:

- 缓存 `/status` 响应(`SystemStatus`),首屏自动 fetch。
- 启动一个 30 min 周期的 `setInterval` 重新拉取 LLM 健康(`status-cache.tsx:189-199`),不打扰数据库统计。
- 暴露 `incrementResumes` / `decrementResumes` / `incrementJobs` / `incrementImprovements` / `setHasMasterResume` (`status-cache.tsx:97-176`) 用于乐观更新——例子见 `app/(default)/dashboard/page.tsx:178-186` 的 `handleUploadComplete`。
- 提供 `useIsStatusStale` (`status-cache.tsx:229-251`)辅助判断数据陈旧度。

所有更新都走 immutable spread(`status-cache.tsx:99-110`),从不直接修改 `prev.status`,符合函数式风格。

2.2 LanguageProvider

`apps/frontend/lib/context/language-context.tsx:26-113` 维护两个独立状态:

字段	来源	持久化
<code>uiLanguage</code>	<code>localStorage(resume_matcher_ui_language)</code>	纯客户端
<code>contentLanguage</code>	<code>localStorage → backend /config/language</code>	同步到后端

`language-context.tsx:64-87` 的 `setContentLanguage` 采用"乐观更新 + 失败回滚"模式:先更新本地 state,再 PUT 后端,失败时回滚。`setUiLanguage` (`language-context.tsx:89-96`)是纯客户端 setter,只写 `localStorage`,无网络往返。

2.3 ResumePreviewProvider

`apps/frontend/components/common/resume_previewer_context.tsx` (由 `app/(default)/layout.tsx:1` 引入)承担 `/tailor` → `/builder` 的数据接力:`tailwind/page.tsx:154` 在 `confirm` 之后 `setImprovedData(confirmed)`,builder 通过 `useResumePreview()` 读取并填充表单。详见 `apps/frontend/app/(default)/tailor/page.tsx:69-162`。

三、三个核心自定义 Hook

3.1 useEnrichmentWizard —— 多步向导状态机

`apps/frontend/hooks/use-enrichment-wizard.ts` 用 `useReducer` 管理一个 9 步向导: `idle → analyzing → questions → generating → preview → applying → complete → error | no-improvements`。

Reducer 设计

`use-enrichment-wizard.ts:75-165` 的 `wizardReducer` 接收 13 种 action,典型模式:

```
case 'SET_ANSWER': {
  return {
    ...state,
    answers: { ...state.answers, [action.questionId]: action.answer },
  };
}
```

所有 case 都返回新对象,符合不可变约束。

对外 API

`use-enrichment-wizard.ts:302-321` 暴露的字段:

类别	字段
State	<code>state</code> (整棵 WizardState)
动作	<code>startAnalysis</code> 、 <code>setAnswer</code> 、 <code>nextQuestion</code> 、 <code>prevQuestion</code> 、 <code>goToQuestion</code> 、 <code>generateEnhancements</code> 、 <code>applyChanges</code> 、 <code>reset</code> 、 <code>retry</code>
派生	<code>currentQuestion</code> 、 <code>currentItem</code> 、 <code>isLastQuestion</code> 、 <code>isFirstQuestion</code> 、 <code>answeredCount</code> 、 <code>totalQuestions</code>

使用示例

```
import { useEnrichmentWizard } from '@hooks/use-enrichment-wizard';

function EnrichmentModal({ resumeId }: { resumeId: string }) {
  const wizard = useEnrichmentWizard(resumeId);

  useEffect(() => {
    if (wizard.state.step === 'idle') wizard.startAnalysis();
  }, [wizard.state.step, wizard]);

  if (wizard.state.step === 'questions') {
    return (
      <QuestionStep
        question={wizard.currentQuestion!}
        value={wizard.state.answers[wizard.currentQuestion!.question_id] ?? ''}
        onChange={(v) => wizard.setAnswer(wizard.currentQuestion!.question_id, v)}
        onNext={wizard.nextQuestion}
        onPrev={wizard.prevQuestion}
        isFirst={wizard.isFirstQuestion}
        isLast={wizard.isLastQuestion}
      />
    );
  }
  if (wizard.state.step === 'preview') {
    return <PreviewStep items={wizard.state.preview} onApply={wizard.applyChanges} />;
  }
  // ... 其他 step
}
```

`wizard.retry` (`use-enrichment-wizard.ts:267-284`) 是个非常贴心的设计:它根据当前 state 决定回退到哪一步 —— 已有 preview 则停在 preview,只有 answers 则回到 questions,否则从头开始。

3.2 useFileUpload —— 拖拽 + 异步上传

`apps/frontend/hooks/use-file-upload.ts:76-629` 是一个完全自包含的文件上传 hook,处理:

能力	位置
拖拽事件(dragenter/dragover/drop)	use-file-upload.ts:522-578
点击触发	openFileDialog (use-file-upload.ts:590-596)
getInputProps 注入 <code><input type="file"></code>	use-file-upload.ts:598-612
类型/大小校验	validateFile (use-file-upload.ts:125-158)
唯一 ID	generateUniqueId (use-file-upload.ts:167-169)
重复检测	use-file-upload.ts:367-378
图片预览 URL.createObjectURL	use-file-upload.ts:160-165
上传过程(fetch + FormData)	uploadFileInternal (use-file-upload.ts:171-320)
并发计数	inFlightUploadsRef (use-file-upload.ts:105)
卸载时清理	clearFiles (use-file-upload.ts:501-516)

返回值是 `[state, actions]` tuple,方便解构。 [isUploadingGlobal](#) 由引用计数维护,只有当所有文件都上传完毕才变 false,避免界面闪烁。

使用示例

```
import { useFileUpload, formatBytes } from '@/hooks/use-file-upload';
import { getUploadUrl } from '@/lib/api';

function ResumeUploader({ onUploaded }: { onUploaded: (resumeId: string) => void }) {
  const [{ files, isDragging, errors, isUploadingGlobal }, { getInputProps, openFileDialog, removeFile }] =
    useFileUpload({
      maxSize: 10 * 1024 * 1024, // 10MB
      accept: 'application/pdf', // 仅 PDF
      multiple: false,
      uploadUrl: getUploadUrl(),
      onUploadSuccess: (file, response) => {
        if (typeof response.resume_id === 'string') onUploaded(response.resume_id);
      },
    });

  return (
    <div onDragEnter={...} onDragOver={...} onDrop={...}>
      <input {...getInputProps()} />
      <button onClick={openFileDialog} disabled={isUploadingGlobal}>
        {isUploadingGlobal ? 'Uploading...' : 'Select PDF'}
      </button>
      {files.map((f) => (
        <div key={f.id}>
          {f.file.name} — {formatBytes(f.file.size)}
          <button onClick={() => removeFile(f.id)}>✕</button>
        </div>
      ))}
      {errors.map((e, i) => <p key={i} className="text-red-600">{e}</p>)}
    </div>
  );
}
```

注意 [use-file-upload.ts:309](#) 会过滤与该文件同名的通用错误,避免重复提示。

3.3 useRegenerateWizard — 章节重写向导

[apps/frontend/hooks/use-regenerate-wizard.ts:59-194](#) 是 AI "regenerate" 流程的状态机,与 `enrichment` 类似但更轻:它直接用多个 `useState` 而非 `reducer`,因为状态字段数量少且彼此独立。

[use-regenerate-wizard.ts:95-125](#) 的 `generate` :

1. 校验至少选中一项([use-regenerate-wizard.ts:96-99](#))。
2. 构造 `RegenerateRequest` (含 `output_language`, [use-regenerate-wizard.ts:106-111](#))。
3. 调用 `regenerateItemsApi` ([lib/api/enrichment.ts:151-160](#))。
4. 成功后切到 `previewing` step,失败时回到 `instructing` 让用户修改 prompt。

`acceptChanges` ([use-regenerate-wizard.ts:140-166](#)) 在 `apply` 成功后 `await new Promise(r => setTimeout(r, 0))` —— 一行非常精妙,给 React 机会 flush UI 后再 `reset`,避免 wizard 关闭时的视觉抖动。

使用示例

```
import { useRegenerateWizard } from '@hooks/use-regenerate-wizard';

function RegenerateButton({ resumeId, onSuccess }: { resumeId: string; onSuccess: () => void }) {
  const w = useRegenerateWizard({
    resumeId,
    outputLanguage: 'en',
    onSuccess,
  });

  return (
    <
      <Button onClick={w.startRegenerate}>Regenerate Selected</Button>
      {w.step === 'selecting' && (
        <ItemPicker
          items={allItems}
          selected={w.selectedItems}
          onChange={w.setSelectedItems}
        />
      )}
      {w.step === 'instructing' && (
        <InstructionDialog
          value={w.instruction}
          onChange={w.setInstruction}
          onConfirm={w.generate}
        />
      )}
      {w.step === 'previewing' && (
        <DiffView items={w.regeneratedItems} onAccept={w.acceptChanges} onReject={w.rejectAndRegenerate} />
      )}
    </>
  );
}
```

四、工具函数

4.1 lib/utils.ts — cn + 日期归一化

`apps/frontend/lib/utils.ts:11-13` 提供 `cn()`, 基于 `clsx` + `tailwind-merge`, 是处理条件 `className` 的标准工具。

`utils.ts:28-51` 的 `formatDateRange` 把 en-dash/em-dash 归一化为 ASCII hyphen-minus, 并补全 `Jun 2025 Aug 2025` → `Jun 2025 - Aug 2025`。注释 `utils.ts:17` 说明这能保证 PDF 文本提取的 ATS 兼容性。

4.2 lib/utils/download.ts

- `downloadBlobAsFile` (`download.ts:1-15`): 触发浏览器下载并 revoke object URL。
- `sanitizeFilename` (`download.ts:33-62`): 把任意标题转成安全的 PDF 文件名, 使用 `Array.from()` 处理多字节字符避免截断 emoji/中文。
- `buildResumeFilename` (`download.ts:68-92`): 构造 `{Name} - {Type} - {Company}.pdf` 模式, 各字段缺失时优雅降级。

4.3 lib/utils/section-helpers.ts

围绕 `SectionMeta` (`components/dashboard/resume-component.tsx` 导出) 操作:

- `DEFAULT_SECTION_META` (`section-helpers.ts:16-71`): 6 个内建章节的元数据。
- `localizeDefaultSectionMeta` (`section-helpers.ts:93-110`): 只翻译用户没改过的英文默认名, 避免覆盖自定义。
- `withLocalizedDefaultSections` (`section-helpers.ts:118-125`): 用 `i18n` 包裹整个 `ResumeData`, 用于 print 路由注入。
- `getSortedSections` / `getAllSections` (`section-helpers.ts:137-148`): 按 `order` 排序, 前者过滤隐藏。
- `createCustomSection` (`section-helpers.ts:165-182`): 自动生成 `custom-<n>` 唯一 ID。

4.4 lib/utils/keyword-matcher.ts

- 200+ 词停用词表 (`keyword-matcher.ts:6-200`)。
- `extractKeywords` (`keyword-matcher.ts:209-223`): 切词 + 过滤。
- `segmentTextByKeywords` (`keyword-matcher.ts:229-257`): 返回 `{text, isMatch}[]`, 前端高亮匹配。
- `calculateMatchStats` (`keyword-matcher.ts:262-285`): 计算匹配百分比。

4.5 lib/utils/html-sanitizer.ts

apps/frontend/lib/utils/html-sanitizer.ts:21-27 用 isomorphic-dompurify 提供白名单:

```
const ALLOWED_TAGS = ['strong', 'em', 'u', 'a'];
const ALLOWED_ATTR = ['href', 'target', 'rel'];
```

FORCE_BODY: true 让 sanitize 输出 body 元素而非 DocumentFragment,以便直接用于 dangerouslySetInnerHTML。

五、类型定义

文件	内容
lib/types/template-settings.ts	TemplateType / PageSize / AccentColor / MarginSettings / TemplateSettings 等; DEFAULT_TEMPLATE_SETTINGS、settingsToCssVars (template-settings.ts:159-196)、 TEMPLATE_OPTIONS
lib/types/lucide.d.ts	给 lucide-react/dist/esm/icons/* 加 module declaration,支持 deep path import 的类型推断

lib/types/template-settings.ts:159-196 的 settingsToCssVars 把所有 spacing/font 配置转成 CSS 变量,在 print/resumes/[id]/page.tsx 里 <Resume style={settingsToCssVars(settings)} /> 直接消费,避免手写 style={{ '--section-gap': ... }}。

六、常量管理

文件	常量
lib/config/version.ts	APP_VERSION = '1.2'、 APP_CODENAME = 'Nightvision'、 APP_NAME 与 getVersionString
lib/constants/page-dimensions.ts	PAGE_DIMENSIONS (A4/LETTER mm)、 mmToPx、 getContentArea、 calculatePreviewScale (page-dimensions.ts:14-73)

calculatePreviewScale (page-dimensions.ts:59-62)把页面 mm 换算成 96 DPI 像素,供 components/preview/ 缩放预览。

七、国际化

7.1 双目录结构

路径	角色
i18n/config.ts	locale 元数据(5 种语言 + 名称 + flag)
lib/i18n/	翻译引擎:客户端 hook、server util、messages 索引、嵌套查找
messages/*.json	翻译文本,文件名 pt-BR.json 被 import 为 pt

i18n/config.ts:5 的 locales = ['en','es','zh','ja','pt'] 是 source of truth;注意 docs/agent/features/i18n.md 表格已过期,实际项目支持葡萄牙语(apps/frontend/CLAUDE.md:90)。

7.2 Messages 索引与类型对齐

apps/frontend/lib/i18n/messages.ts:9-17 是项目里最容易出 bug 的地方:

```
export type Messages = typeof en;
const allMessages: Record<Locale, Messages> = { en, es, zh, ja, pt };
```

因为 5 个 locale 都被强类型化为 Messages (= en.json 的精确形状),只要给 en.json 加一个 key, tsc / next build 就会在 es/zh/ja/pt-BR 中寻找同样的路径并失败。这是项目主动选择的"结构漂移即刻 fail"的策略,见 apps/frontend/CLAUDE.md:97-105。

7.3 客户端 hook 与 server util

- `lib/i18n/translations.ts:16-37` 的 `useTranslations` 返回 `{ t, messages, locale }`, `t(key, params)` 支持 `dot.path` 与 `{placeholder}` 替换。
- `lib/i18n/translations.ts:47-55` 的 `translate` 是 server 端等价物,供 print 路由使用。
- `lib/i18n/utils.ts:1-24` 提供 `getNestedValue` 与 `applyParams`, `getNestedValue` 在 key 缺失时返回原 key 字符串,不抛错 —— 这是一种降级策略,UI 会显示 "common.save" 而不是崩。
- `lib/i18n/locale.ts:3-8` 的 `resolveLocale` 把 `searchParams.lang` 限定到合法 locale,避免越界。

7.4 使用示例

```
'use client';
import { useTranslations } from '@lib/i18n';

export default function SaveButton() {
  const { t, locale } = useTranslations();
  return (
    <button aria-label={t('common.saveAria')}>
      {t('common.save', { count: 3 })}
    </button>
  );
}
```

`print/resumes/[id]/page.tsx:13` 则是 server-side 用法:

```
import { translate } from '@lib/i18n/server';
const localized = translate(locale, 'resume.sections.summary');
```

八、Context 消费模式速查

```
import { useStatusCache } from '@lib/context/status-cache';
import { useLanguage } from '@lib/context/language-context';

function AnyClientComponent() {
  const { status, isLoading, refreshStatus, incrementResumes } = useStatusCache();
  const { contentLanguage, setContentLanguage, uiLanguage, setUiLanguage } = useLanguage();

  if (isLoading) return <Spinner />;
  return <div>{status?.database_stats.total_resumes} resumes</div>;
}
```

每个 hook 都会在 Provider 外抛错(`status-cache.tsx:218-224`、`language-context.tsx:115-121`),所以在测试或迁移时缺少 Provider 会立即被定位。

九、最佳实践总结

1. 后端访问 100% 走 `lib/api/*`: 组件层永远不要 `fetch('/api/...')`, 否则会绕过 240s 超时和 AbortError 翻译。`apps/frontend/CLAUDE.md:65` 明确禁止。
2. i18n 增删 key 必须同步 5 个 JSON: `en.json` 是其他 4 个文件的形状模板, build 阶段会严格校验。
3. Context 暴露的 setter 保持稳定: 在 `useCallback` 包装并放进 Provider 的 `value` 中, 避免消费组件无效重渲染。
4. 不可变更新: Reducer 和 Context 内部都使用 spread, 例如 `status-cache.tsx:99-110`。
5. 乐观更新 + 失败回滚: `LanguageProvider.setContentLanguage` (`language-context.tsx:64-87`) 是范例 —— 立即更新 UI, 再同步后端, 失败时回滚。
6. 错误路径携带后端 detail: `data.detail || 'Fallback (status X).'` (`lib/api/config.ts:91-93` 等) 是统一模式, 既保留后端语义又不会因 `res.json()` 失败丢失上下文。

十、阅读顺序建议

1. 先读 `lib/api/client.ts:1-75`, 理解 240s 超时 + 内部 origin 切换。
2. 接着看 `lib/api/resume.ts:111-135` 的 `postImprove`, 体会 "先 text 再 parse" 的容错思路。

3. 再看 `lib/context/status-cache.tsx`, 掌握 Context + 乐观更新的写法。

4. 然后按顺序阅读三个 Hook, 关注 Reducer 与 `useState` 的取舍。

5. 最后阅读 `lib/i18n/*`, 理解 build-time 结构校验的设计动机。

完成本篇后, 你已经掌握前端"客户端状态 + 网络层"的全部主干, 可以继续阅读 [前端核心模块] 与 [前端特性与流程] 了解具体业务页面与端到端流程。

Resume Matcher 前端核心组件深度解析

适用版本： `apps/frontend` (Next.js 16 + React 19 + TypeScript + Tailwind v4) 本教程面向具备 React 基础、希望深入理解真实业务组件设计的中级开发者。

一、组件目录总览

`apps/frontend/components/` 按职责切分为 10 个目录，共约 70 个 `.tsx` 文件。组织原则为"按业务领域 + 按抽象层级"二维划分。

目录	文件数	抽象层级	业务领域
<code>ui/</code>	14	原子组件	跨页面通用 (Button、Dialog、Input)
<code>common/</code>	2	跨领域工具	错误边界、跨页面 Context
<code>home/</code>	2	落地页	Hero 横幅、SwissGrid 网格
<code>dashboard/</code>	2	仪表盘	上传对话框、Resume 类型定义
<code>builder/</code>	21 + 9 forms	编辑器	主简历表单、模板、AI 重生成、JD 对比
<code>enrichment/</code>	4	AI 增强向导	多步问答、AI 改进预览
<code>taylor/</code>	1	差异化定制	简历前后对比弹窗
<code>resume/</code>	6 + 5 CSS	渲染模板	4 种排版样式的输出组件
<code>preview/</code>	4	分页预览	WYSIWYG 页面分页、缩放
<code>settings/</code>	1	设置	LLM API Key 菜单

1.1 用户故事视角下的组件映射

整个应用的核心闭环：**上传** → **改进** → **定制** → **导出**。每一阶段由一组组件协作完成。

阶段	用户动作	关键组件	入口文件
上传	拖入 PDF/DOCX	<code>ResumeUploadDialog</code>	<code>components/dashboard/resume-upload-</code>
改进	AI 增强简历	<code>EnrichmentModal</code> + <code>QuestionStep</code>	<code>components/enrichment/enrichment-mc</code>
编辑	表单修改	<code>ResumeForm</code> + <code>SectionHeader</code>	<code>components/builder/resume-form.tsx:</code>
定制	粘贴 JD 生成	<code>TailorPage</code> + <code>DiffPreviewModal</code>	<code>app/(default)/tailor/page.tsx:24</code>
导出	下载 PDF	<code>PaginatedPreview</code> + <code>PageContainer</code>	<code>components/preview/paginated-previe</code>

二、UI 基础组件 (`components/ui/`)

UI 组件库采用"手写而非依赖 shadcn"的策略，每一组件都按 Swiss International Style 严格设计：`rounded-none` 方形硬边、1px 黑边框、`shadow-sw-sm` 硬阴影 (无模糊)、hover 时 `translate-[1px]` 制造"按下"反馈。

2.1 Button: `components/ui/button.tsx`

定义 8 个语义化变体 (`variant`)，区分主次操作：

变体	颜色	用途
<code>default</code>	Hyper Blue <code>#1D4ED8</code>	保存、提交、创建
<code>success</code>	Signal Green <code>#15803D</code>	下载、确认、完成
<code>destructive</code>	Alert Red <code>#DC2626</code>	删除、移除
<code>warning</code>	Alert Orange <code>#F97316</code>	重置、清空
<code>outline</code>	透明 + 黑边框	取消、返回
<code>ghost</code>	无背景	工具栏图标
<code>link</code>	文字下划线	内联链接

关键设计：icon 按钮使用 `before:-inset-1.5` 伪元素扩展点击区至 44×44（WCAG 2.5.8 要求），即使可见尺寸只有 32×32。源码注释 `components/ui/button.tsx:67-75` 解释了此设计动机。

```
// components/ui/button.tsx:46
const Button = React.forwardRef<HTMLButtonElement, ButtonProps>({
  { className, variant = 'default', size = 'default', ...props }, ref) => {
    // baseStyles: layout, transition, focus, disabled
    // variantClass: bg/border/shadow/hover/active
    // sizeClass: h-X w-X px-X
    return <button ref={ref} className={cn(baseStyles, variantClass, sizeClass, className)} {...props} />;
  }
});
```

2.2 Dialog: `components/ui/dialog.tsx`

无外部依赖的原生实现。使用 `React.createContext` 暴露 `open`、`onOpenChange`、`titleId` 三个字段。`DialogContent` 通过 `createPortal` 渲染到 `document.body` 并设置 `role="dialog"` + `aria-modal="true"` + `aria-labelledby` 三件套（`components/ui/dialog.tsx:144-156`）。`useId()` 保证多实例下 `aria-labelledby` 引用稳定（`components/ui/dialog.tsx:43`）。

支持的子组件：`DialogTrigger`、`DialogClose`、`DialogContent`、`DialogHeader`、`DialogFooter`、`DialogTitle`、`DialogDescription`。`DialogTrigger` 支持 `asChild` 模式通过 `React.cloneElement` 注入 `onClick`（`components/ui/dialog.tsx:56-71`），可与任意按钮组合。

2.3 ConfirmDialog: `components/ui/confirm-dialog.tsx`

基于 `Dialog` 二次封装，提供 4 种语义（`danger` / `warning` / `success` / `default`），每种对应不同图标和确认按钮变体（`components/ui/confirm-dialog.tsx:73-106`）。`closeOnConfirm` 默认 `true`，意味着点击确认后自动关闭弹窗。

2.4 RetroTabs: `components/ui/retro-tabs.tsx`

瑞士风格 Tabs：底部黑边、活动 Tab 白色背景 + 顶左硬阴影模拟凸起（`components/ui/retro-tabs.tsx:50-53`）。`disabled` 状态时背景变为 `bg-paper-tint` + 50% 透明度，鼠标光标变为 `not-allowed`。

2.5 其他原子组件

组件	文件	关键特性
<code>Input</code>	<code>components/ui/input.tsx:14</code>	黑色 1px 边框，Hyper Blue 聚焦环， <code>rounded-none</code>
<code>Textarea</code>	<code>components/ui/textarea.tsx:6</code>	同 Input 风格， <code>min-h-[60px]</code>
<code>Label</code>	<code>components/ui/label.tsx:4</code>	<code>font-mono uppercase tracking-wider text-ink-soft</code>
<code>Card</code>	<code>components/ui/card.tsx:9</code>	4 变体（default/interactive/outline/ghost）， <code>interactive</code> 启用悬停 <code>translate-y-[-2px]</code>
<code>Dropdown</code>	<code>components/ui/dropdown.tsx:23</code>	自实现菜单， <code>useId</code> 绑定 <code>aria-controls</code>
<code>ToggleSwitch</code>	<code>components/ui/toggle-switch.tsx:24</code>	角色 <code>role="switch"</code> ，开关时 <code>translate-x-6</code>
<code>LinkDialog</code>	<code>components/ui/link-dialog.tsx</code>	简历中超链接编辑弹窗
<code>RichTextEditor</code>	<code>components/ui/rich-text-editor.tsx</code>	基于 Tiptap，富文本编辑（用于 <code>ExperienceForm</code> ）
<code>RichTextToolbar</code>	<code>components/ui/rich-text-toolbar.tsx</code>	Tiptap 工具栏

三、通用组件（`components/common/`）

3.1 ResumePreviewProvider: `components/common/resume_previewer_context.tsx`

跨页面共享 AI 改进结果的 Context。提供 `improvedData` 与 `setImprovedData` 两个字段（`components/common/resume_previewer_context.tsx:107-110`）。`ImprovedResult.data` 包含 LLM 输出的所有字段：`resume_preview`、`cover_letter`、`outreach_message`、`diff_summary`、`detailed_changes`、`job_description`、`job_keywords`（`components/common/resume_previewer_context.tsx:73-101`）。在 `app/(default)/layout.tsx` 包裹整个 `(default)` 路由组，使 `tailor` 页写入的 AI 结果能被 `builder` 页直接读取。

3.2 ErrorBoundary: `components/common/error-boundary.tsx`

React 类组件，捕获子树异常并展示回退 UI。`getDerivedStateFromError` 切换 `hasError` 标志（`components/common/error-boundary.tsx:37`），`componentDidCatch` 打印到控制台（`components/common/error-boundary.tsx:41`）。暴露两个动作按钮：Try Again（重置 state）与 Reload Page（`window.location.reload()`）。

四、仪表盘（`components/dashboard/`）

4.1 ResumeUploadDialog: `components/dashboard/resume-upload-dialog.tsx`

支持受控/非受控两种 open 模式（`components/dashboard/resume-upload-dialog.tsx:54-61`）。内部使用 `useFileUpload` hook 管理文件状态（拖拽、验证、上传进度）。允许的 MIME：`application/pdf`、`application/vnd.openxmlformats-officedocument.wordprocessingml.document`、`application/msword`，单文件 4MB 上限（`components/dashboard/resume-upload-dialog.tsx:33-38`）。

成功上传后延迟 1500ms 关闭弹窗以便用户看到成功提示（`components/dashboard/resume-upload-dialog.tsx:83-90`）。后端返回 `processing_status: 'failed'` 时保留弹窗并暴露"重试处理"按钮（`components/dashboard/resume-upload-dialog.tsx:120-128`）。

4.2 Resume 类型与默认组件: `components/dashboard/resume-component.tsx`

文件导出所有共享类型：`PersonalInfo`、`Experience`、`Education`、`Project`、`AdditionalInfo`、`SectionMeta`、`CustomSection`、`ResumeData`。`SectionMeta` 是动态章节系统的核心，记录每个章节的 `id` / `key` / `displayName` / `sectionType` / `isDefault` / `isVisible` / `order`（`components/dashboard/resume-component.tsx:88-96`）。

默认导出的 `Resume` 是一个轻量路由组件，根据 `settings.template` 选择 `swiss-single` / `swiss-two-column` / `modern` / `modern-two-column` 四种模板（`components/dashboard/resume-component.tsx:179-207`）。所有模板接收相同的 `data: ResumeData`，通过 `settingsToCssVars` 将设置转换为 CSS 变量（`components/dashboard/resume-component.tsx:172`），实现"一套数据 + 一组变量 → 四种样式"。

五、简历构建器（`components/builder/`）

构建器是整个应用最复杂的模块，21 个文件 + 9 个 form 子组件。`resume-builder.tsx` 是总装入口。

5.1 ResumeBuilder: `components/builder/resume-builder.tsx`

主编辑器，1028 行。状态机包含 16 个 `useState`，管理 4 个 Tab（`resume` / `cover-letter` / `outreach` / `jd-match`）。数据加载三优先级逻辑（`components/builder/resume-builder.tsx:282-361`）：

- URL 中 `?id=` 存在 → 调用 `fetchResume(id)` 拉取后端
- Context 中的 `improvedPreview`（来自 `tailor` 页）→ 用作草稿
- `localStorage['resume_builder_draft']` → 浏览器刷新恢复

保存/下载逻辑：`handleSave` 调用 `updateResume`（`components/builder/resume-builder.tsx:411`），`handleDownload` 调用 `downloadResumePdf` 拿到 Blob 再用 `downloadBlobAsFile` 触发浏览器下载（`components/builder/resume-builder.tsx:443-449`）。`onbeforeunload` 事件防止误关丢失草稿（`components/builder/resume-builder.tsx:270-279`）。

AI 重新生成使用 `useRegenerateWizard` hook（`components/builder/resume-builder.tsx:160-204`），把 `resumeData.workExperience` / `personalProjects` / `additional.technicalSkills` 映射为 `RegenerateItemInput[]`（`components/builder/resume-builder.tsx:207-238`）。

5.2 ResumeForm: components/builder/resume-form.tsx

章节表单编排器。使用 @dnd-kit 实现拖拽排序，外部 DndContext + 内部 SortableContext 包裹所有章节 (components/builder/resume-form.tsx:379-405)。handleDragEnd 通过交换 order 字段实现重排 (components/builder/resume-form.tsx:173-207)，保留 personalInfo 始终在最先。

renderDefaultSection 根据 section.key 分发到 6 个子表单组件 (components/builder/resume-form.tsx:215-273)。renderCustomSection 根据 section.sectionType (text / itemList / stringList) 渲染对应的通用表单 (components/builder/resume-form.tsx:318-358)。所有默认/自定义章节都包裹在 SectionHeader 中，提供重命名/上下移/删除/显隐 4 个动作。

5.3 SectionHeader: components/builder/section-header.tsx

章节元数据操作面板。本地状态 isEditing / editedName / showDeleteConfirm 三个 (components/builder/section-header.tsx:46-48)。重命名支持 Enter 保存、Esc 取消 (components/builder/section-header.tsx:67-73)。删除内置二次确认 (ConfirmDialog)，默认章节的删除按钮实际触发"隐藏"动作 (components/builder/section-header.tsx:75-83)。

5.4 DraggableSectionWrapper & DraggableListItem

两个拖拽包装器。前者包装章节 (id: string)，后者包装列表项 (id: number ，如一条工作经历)。两者都用 @dnd-kit/sortable 的 useSortable hook 获取 attributes / listeners / setNodeRef / transform / transition / isDragging ，并将 transform 通过 CSS.Transform.toString 注入 style (components/builder/draggable-section-wrapper.tsx:29-39)。差异在于禁用条件：章节中 personalInfo 不可拖动 (resume-form.tsx:395)，列表项均可拖动。

5.5 章节表单: components/builder/forms/

9 个表单组件，每个都是受控组件。共同模式：父组件持有 data ，子组件通过 onChange(data) 整体回传，避免深拷贝子树。

表单	文件	特殊能力
PersonalInfoForm	forms/personal-info-form.tsx:14	8 个字段的网格布局
SummaryForm	forms/summary-form.tsx:13	纯 Textarea，Enter 阻止冒泡
ExperienceForm	forms/experience-form.tsx:45	动态加载 Tiptap (dynamic import 减少首屏 JS)，DraggableListItem 排序
EducationForm	forms/education-form.tsx:33	同 ExperienceForm 但更简单
ProjectsForm	forms/projects-form.tsx	类似 Experience
AdditionalForm	forms/additional-form.tsx:14	4 个 Textarea 数组字段 (每行一项)
GenericTextForm / GenericItemForm / GenericListForm	同目录	自定义章节三种 schema 类型的渲染器

5.6 FormattingControls: components/builder/formatting-controls.tsx

模板/排版设置面板 (576 行)。可折叠 (isExpanded state)，内含 6 大区块：模板选择、强调色 (仅 modern 系列)、页面尺寸、边距、间距、字号。每个区块都遵循相同模式：<h4> 标题 + 一组按钮。MarginSlider (components/builder/formatting-controls.tsx:512-538) 是 5-25mm 范围的 <input type="range">。SpacingSelector (components/builder/formatting-controls.tsx:551-574) 渲染 1-5 数字按钮组。

底部"Effective Output"区实时显示当前设置映射的实际 CSS 值 (rem 单位、行高、字号)，帮助用户预览。

5.7 模板与渲染: components/builder/template-selector.tsx + components/resume/

TemplateSelector (components/builder/template-selector.tsx:18) 渲染 4 个缩略图按钮。TemplateThumbnail 内部组件 (template-selector.tsx:80+) 用 SVG 画简单线框图代表各布局。ResumeSingleColumn (components/resume/resume-single-column.tsx:29) 是最常用的"瑞士单列"模板，使用 getSortedSections 工具获取按 order 排序的可见章节 (resume-single-column.tsx:37)，再对每个 SectionMeta 分发渲染 (resume-single-column.tsx:93-122)。DynamicResumeSection (components/resume/dynamic-resume-section.tsx:23) 处理用户自定义章节，根据 sectionType 渲染 text / itemList / stringList 三种内容。

SafeHtml (components/resume/safe-html.tsx:28) 是无 'use client' 指令的服务端组件，使用 sanitizeHtml (基于 isomorphic-dompurify) 清洗 LLM 输出的 HTML，仅保留 strong/em/u/a 四种标签。

5.8 AI 重生成向导：components/builder/regenerate-wizard.tsx + 三个子组件

三步向导的状态机由 `RegenerateWizardStep` 联合类型定义：`idle` / `selecting` / `instructing` / `generating` / `previewing` / `complete` (`components/builder/regenerate-wizard.tsx:13-19`)。`RegenerateWizard` 是空壳调度，根据 `step` 决定打开哪个 `Dialog` (`regenerate-wizard.tsx:122-159`)：

1. `RegenerateDialog`：勾选要重生成的工作经历/项目/技能条目 (`regenerate-dialog.tsx:36`)
2. `RegenerateInstructionDialog`：输入修改指令 (`regenerate-instruction-dialog.tsx:37`)
3. `RegenerateDiffPreview`：对比原内容与 LLM 输出，按条目 Accept/Reject (`regenerate-diff-preview.tsx:43`)

整个状态机由外部 `useRegenerateWizard` hook 驱动 (`resume-builder.tsx:160-204`)，Wizard 自身只负责"打开/关闭哪个 Dialog"。

5.9 JD 对比视图：components/builder/jd-comparison-view.tsx

二分栏布局：左侧 `JDDisplay` (`components/builder/jd-display.tsx:14`，只读 JD)，右侧 `HighlightedResumeView` (`components/builder/highlighted-resume-view.tsx:18`，简历中匹配 JD 关键词的部分用 `<mark className="bg-yellow-200">` 高亮)。关键词提取与统计逻辑在 `lib/utils/keyword-matcher.ts`，由父组件通过 `useMemo` 调用 (`components/builder/jd-comparison-view.tsx:25,60`)。

5.10 Cover Letter & Outreach

`CoverLetterEditor` (`components/builder/cover-letter-editor.tsx:22`) 与 `OutreachEditor` (`components/builder/outreach-editor.tsx:22`) 几乎对称，都提供字数统计、保存按钮（其中 Outreach 额外提供复制到剪贴板）。`CoverLetterPreview` (`components/builder/cover-letter-preview.tsx:29`) 按 `\n\n` 分段渲染信件正文 + 本地化日期。`OutreachPreview` (`components/builder/outreach-preview.tsx:15`) 则使用消息气泡样式。

`GeneratePrompt` (`components/builder/generate-prompt.tsx:22`) 是首次进入 Cover Letter / Outreach Tab 时的引导页。当 `isTailoredResume=false` 时提示用户先到 tailor 页生成 (`generate-prompt.tsx:35-58`)，否则显示"开始生成"按钮。

六、简历预览 (components/preview/)

6.1 PaginatedPreview: components/preview/paginated-preview.tsx

WYSIWYG 预览器 (228 行)。核心思想：用"隐形测量 + 缩放展示"分离——先把 `<Resume>` 渲染到一个不可见容器 (`left: -9999`) 以获得真实 `scrollHeight`，然后切分为多个 `<PageContainer>` 渲染 (`components/preview/paginated-preview.tsx:170-225`)。

- 缩放：0.4× 到 1.5×，默认 0.6× (`paginated-preview.tsx:18-20`)。`calculateAutoZoom` 根据容器宽度自动计算 (`paginated-preview.tsx:75-82`)，用户手动缩放后 `autoZoom` 置为 `false`。
- 边距辅助线：toggle `showMargins`，渲染蓝色虚线框 (`page-container.tsx:67-76`)。
- 页数显示：右上角 `FileText` 图标 + 单复数文案 (`paginated-preview.tsx:155-164`)。

6.2 PageContainer: components/preview/page-container.tsx

单页画布。`width: pageWidthPx`、`height: pageHeightPx`，`transform: scale(${scale})` 缩放 (`page-container.tsx:57-65`)。`contentOffset` / `contentEnd` 让每个 PageContainer 只显示属于本页的内容片段，避免与上下页重复 (`page-container.tsx:50-52`)。

6.3 usePagination: components/preview/use-pagination.ts

分页算法 hook。先 `document.fonts.ready` 等字体加载 (`use-pagination.ts:53`)，再 `getContentAreaPx` 算出可写区高度，遍历 `.resume-item` / `[data-no-break]` 元素确定分页点 (`use-pagination.ts:71-79`)，但不在 `.resume-section` 之间强制分页 (`use-pagination.ts:70` 注释解释：让章节自然跨页)。150ms debounce 防止 resize 时频繁计算 (`use-pagination.ts:34`)。

七、简历定制 (components/tailor/ 与 app/(default)/tailor/page.tsx)

7.1 DiffPreviewModal: components/tailor/diff-preview-modal.tsx

对比弹窗 (440 行)。结构：

- 顶部 5 个 `StatCard` 汇总变更数 (`diff-preview-modal.tsx:149-175`)：新增技能、移除技能、新增证书、修改描述、高风险变更。
- 中部按 `field_type` 分组的 `ChangeSection` (`diff-preview-modal.tsx:367-385`)，可折叠。`ChangeItem` (`diff-preview-modal.tsx:392-440`) 用 `+/-/~` 字符标识 `added` / `removed` / `modified`，删除线显示原值。
- 底部 "Reject" 和 "Confirm" 两个按钮，Confirm 提交后端 `confirmImproveResume`。

`elapsed` state 显示确认请求的等待秒数 (`diff-preview-modal.tsx:40-54`)。弹窗关闭逻辑: `isConfirming=true` 时禁用外部关闭, 避免中断正在进行的请求 (`diff-preview-modal.tsx:60-64`)。

7.2 TailorPage 流程: `app/(default)/tailor/page.tsx`

1. 用户粘贴 JD → `uploadJobDescriptions` 提交 (`tailor/page.tsx:11-15`)
2. `previewImproveResume` 异步生成预览, 把结果写入 `ResumePreviewContext` (`setImprovedData`)
3. 弹出 `DiffPreviewModal` 展示 diff
4. 用户确认 → `confirmImproveResume` 落库 → 跳转 `/builder?id=...`

八、简历增强 (`components/enrichment/`)

8.1 EnrichmentModal: `components/enrichment/enrichment-modal.tsx`

AI 增强向导 (203 行)。基于原生 `<dialog>` 元素 (`enrichment-modal.tsx:108-143`), 通过 `dialogRef.current?.showModal()` 打开, `close()` 关闭。
`document.body.style.overflow = 'hidden'` 防止背景滚动 (`enrichment-modal.tsx:50`)。

8 个步骤由 `state.step` 字段驱动: `idle` / `analyzing` / `questions` / `generating` / `preview` / `applying` / `complete` / `no-improvements` / `error` (`enrichment-modal.tsx:146-202`)。 `renderStep()` 内部 switch 把当前步骤映射到对应子组件。所有状态与动作 (`startAnalysis` / `setAnswer` / `generateEnhancements` / `applyChanges` / `retry`) 由 `useEnrichmentWizard` hook 暴露 (`enrichment-modal.tsx:29-44`)。

8.2 QuestionStep / PreviewStep / LoadingSteps

`QuestionStep` (`components/enrichment/question-step.tsx:24`): 多步问答的当前题目页。顶部进度条 (`question-step.tsx:81-101`), 中部当前 item 类型徽章 + 问题 + 答案 Textarea。 `Ctrl/Cmd+Enter` 跳到下一题 (`question-step.tsx:65-77`)。

`PreviewStep` (`components/enrichment/preview-step.tsx:14`): 改进前后的并排展示, 每个 `EnhancementCard` 显示"保留" (灰色列表) 与"新增" (绿色边框列表)。

`LoadingSteps` (`components/enrichment/loading-steps.tsx`): 6 个状态组件: `AnalyzingStep` / `GeneratingStep` / `ApplyingStep` / `CompleteStep` / `NoImprovementsStep` / `ErrorStep`。统一使用 `LoadingStep` 内部组件渲染圆形 `Loader2` + 标题 + 副标题 (`loading-steps.tsx:12-24`)。

九、设置 (`components/settings/`)

`ApiKeyMenu` (`components/settings/api-key-menu.tsx:12`): 折叠菜单, 显示脱敏后的 LLM API Key (前 6 位 + `....`), 点击展开输入框以更新。
`status` 状态机: `idle` / `loading` / `saving` / `saved` / `error`。 `fetchLlmApiKey` / `updateLlmApiKey` 均封装在 `lib/api/config.ts`。

十、落地页 (`components/home/`)

`Hero` (`components/home/hero.tsx:7`): 整屏 hero, 蓝色 40px 网格背景。3 个按钮: GitHub、文档、Launch App。

`SwissGrid` (`components/home/swiss-grid.tsx:8`): 仪表盘通用容器。Header + 内容网格 + Footer 三段。网格用 `@container` + `@2xl:grid-cols-2` 等 Tailwind v4 容器查询 (`swiss-grid.tsx:40`), 卡片随容器宽度自适应列数, 5 列封顶。

十一、完整使用示例

示例 1: 上传简历 → 进入 Builder 流程

调用链:

```

app/(default)/dashboard/page.tsx:33 DashboardPage
└─ <ResumeUploadDialog onUploadComplete={handleUploadComplete} /> (dashboard/page.tsx:353)
    └─ useFileUpload hook (上传文件 + 验证大小/类型)
        └─ 成功后: onUploadComplete(resumeId) (resume-upload-dialog.tsx:79)
            ↓
handleUploadComplete (dashboard/page.tsx:178)
  → localStorage.setItem('master_resume_id', resumeId)
  → checkResumeStatus(resumeId) 轮询后端直到 ready

用户点击 Master Resume Card
  → router.push(`/resumes/${masterResumeId}`) (dashboard/page.tsx:385)

```

示例 2：在 Builder 中编辑章节并保存

调用链：

```

app/(default)/builder/page.tsx:3 BuilderPage
└─ <ResumeBuilder /> (components/builder/resume-builder.tsx:1021)
    └─ <ResumeForm resumeData={resumeData} onUpdate={handleUpdate} />
        └─ <DraggableSectionWrapper id={section.id}> (resume-form.tsx:395)
            └─ <SectionHeader onRename={...} onDelete={...} onMoveUp/Down={...} />
                └─ <ExperienceForm onChange={({data}) => onUpdate({...resumeData, workExperience: data})} />
                    └─ <DraggableListItem id={exp.id}>
                        └─ <Input value={exp.title} onChange={...} />
                        └─ <RichTextEditor ...> (动态加载)

用户点 Save
  → handleSave() (resume-builder.tsx:404)
  → updateResume(resumeId, resumeData) // lib/api/resume.ts

```

示例 3：Tailor 流程生成定制简历

调用链：

```

app/(default)/tailor/page.tsx:24 TailorPage
└─ 用户粘贴 JD → 点击 "Improve"
    └─ previewImproveResume(jdText) (lib/api/resume.ts)
        返回 ImprovedResult { data: { resume_preview, diff_summary, ... } }
    └─ setImprovedData(result) (写入 ResumePreviewContext)
    └─ setShowDiffModal(true) → 渲染 <DiffPreviewModal>
        └─ <StatCard ... /> 汇总变更数
        └─ <ChangeSection> 按类型分组
            └─ <ChangeItem original_value / new_value />

用户点 Confirm
  → confirmImproveResume(...) (lib/api/resume.ts)
  → 成功后 router.push(`/builder?id=${resumeId}`)

Builder 页面加载时 (resume-builder.tsx:282-361)
  检测到 improvedPreview 存在 → 用作初始数据

```

十二、关键设计模式总结

模式	体现	好处
受控数据流	所有 form 组件 <code>onChange</code> 回传整个对象	单一数据源，调试简单
Context 跨页面	<code>ResumePreviewProvider</code> 串联 tailor ↔ builder	无需 URL 编码大对象
乐观 UI + 状态机	多个 <code>useState</code> 标志 loading/saving/error	反馈明确，避免重复点击
路由分发	<code>Resume</code> 组件根据 <code>template</code> 字段选择子模板	4 种样式共用同一份 <code>ResumeData</code>
弹窗内嵌弹窗	<code>ConfirmDialog</code> 基于 <code>Dialog</code> 二次封装	语义化 API + 复用基础能力
i18n shape 强约束	<code>Messages = typeof en</code> 强制 5 个 locale 同构	编译期捕获翻译漂移
a11y 优先	<code>aria-modal</code> 、 <code>aria-labelledby</code> 、44×44 触摸区	默认符合 WCAG 2.2 AA
CSS Modules for 打印	<code>styles/*.module.css</code> 隔离模板样式	打印 PDF 与屏幕预览共用组件

十三、学习路径建议

1. 先读 UI 层（`ui/button.tsx` → `ui/dialog.tsx`）理解 Swiss 设计 tokens 如何落到代码。
2. 再看 Dashboard（`dashboard/page.tsx` + `resume-upload-dialog.tsx`）理解数据获取与状态机。
3. 重点看 Builder（`resume-builder.tsx` → `resume-form.tsx`）理解大型组件的拆分艺术。
4. 最后看 Tailor & Enrichment（`tailor/page.tsx` + `enrichment-modal.tsx`）理解跨页面 Context 与多步向导模式。
5. 对照打印路径：同样的 `ResumeData` 经 `print/resumes/[id]/page.tsx`（服务端组件）渲染，被 Playwright headless Chrome 截图生成 PDF。

附：组件文件路径速查

路径	用途
apps/frontend/components/ui/button.tsx	8 变体按钮
apps/frontend/components/ui/dialog.tsx	原生 Dialog 实现
apps/frontend/components/ui/retro-tabs.tsx	瑞士风格 Tabs
apps/frontend/components/dashboard/resume-component.tsx	ResumeData 类型 + 模板路由
apps/frontend/components/dashboard/resume-upload-dialog.tsx	上传对话框
apps/frontend/components/builder/resume-builder.tsx	Builder 主组件
apps/frontend/components/builder/resume-form.tsx	章节表单编排
apps/frontend/components/builder/formatting-controls.tsx	模板/排版设置
apps/frontend/components/builder/regenerate-wizard.tsx	AI 重生成三步向导
apps/frontend/components/builder/jd-comparison-view.tsx	JD vs 简历对比
apps/frontend/components/enrichment/enrichment-modal.tsx	AI 增强向导
apps/frontend/components/tailor/diff-preview-modal.tsx	简历 diff 确认弹窗
apps/frontend/components/preview/paginated-preview.tsx	WYSIWYG 分页预览
apps/frontend/components/preview/use-pagination.ts	分页算法 hook
apps/frontend/components/resume/resume-single-column.tsx	瑞士单列模板
apps/frontend/components/common/resume_previewer_context.tsx	跨页面 AI 结果 Context
apps/frontend/components/common/error-boundary.tsx	错误边界
apps/frontend/components/home/swiss-grid.tsx	仪表盘网格容器
apps/frontend/components/settings/api-key-menu.tsx	LLM Key 菜单
apps/frontend/app/(default)/builder/page.tsx	<code>/builder</code> 路由
apps/frontend/app/(default)/tailor/page.tsx	<code>/tailor</code> 路由
apps/frontend/app/(default)/dashboard/page.tsx	<code>/dashboard</code> 路由

09 - 前端特性与流程

本章面向希望理解 Resume Matcher 前端特性实现与典型用户流程的开发者。我们将依次剖析国际化、自定义章节、简历模板、AI 富集、JD 匹配等核心模块，并梳理从打开站点到导出 PDF 的端到端流程。

1. 技术栈与目录约定

前端位于 `apps/frontend/`，技术栈为 **Next.js 16 (App Router + Turbopack)**、**React 19**、**TypeScript (strict)**、**Tailwind CSS v4**。所有路由入口位于 `app/`，UI 组件位于 `components/`，后端调用集中在 `lib/api/`，自定义 Hooks 位于 `hooks/`，消息翻译文件位于 `messages/`，i18n 引擎位于 `lib/i18n/` 与 `i18n/config.ts`。

详细的目录约定与文件分工可参考 `apps/frontend/CLAUDE.md:23-58`。

2. 国际化 (i18n)

2.1 两套语言设置

Resume Matcher 在 Settings 页面维护两套独立语言：

- **UI 语言 (uiLanguage)**：界面文案，仅在 localStorage 持久化，影响按钮、菜单、提示文本。
- **内容语言 (contentLanguage)**：LLM 生成内容（简历、求职信）使用的语言，既写 localStorage，也会同步到后端配置。

两套语言相互独立，例如界面用中文而生成英文简历是支持的（见 `docs/agent/features/i18n.md:17-23`）。

2.2 文件结构

```
apps/frontend/
├─ i18n/
│   └─ config.ts          # locales / localeNames / localeFlags (来源真相)
├─ lib/i18n/
│   ├── index.ts          # 桶导出
│   ├── messages.ts       # 静态 import 全部 JSON; 导出 Messages 类型
│   ├── translations.ts   # useTranslations hook (客户端)
│   ├── server.ts         # translate() (服务端)
│   ├── locale.ts         # resolveLocale
│   └─ utils.ts           # getNestedValue / applyParams
├─ messages/
│   ├── en.json           # 来源真相, Messages 类型从此推导
│   ├── es.json
│   ├── zh.json
│   ├── ja.json
│   └─ pt-BR.json        # 文件名是 pt-BR, 但 i18n/config.ts 把它注册为 pt
└─ lib/context/
    └─ language-context.tsx # LanguageProvider, UI + Content 两层状态
```

源码真相位于 `apps/frontend/i18n/config.ts:5-25`，五种支持的语言为 `en / es / zh / ja / pt`（其中 `pt` 的实际文件是 `messages/pt-BR.json`，由 `lib/i18n/messages.ts:7` 导入）。

2.3 翻译机制

客户端组件使用 `useTranslations()` Hook（`apps/frontend/lib/i18n/translations.ts:16-37`）：

```
const { t } = useTranslations();
<button>{t('common.save')}</button>
```

`t()` 接受点号分隔的 key，从 `messages/{locale}.json` 的嵌套结构中取值，并通过 `applyParams`（`apps/frontend/lib/i18n/utils.ts:16-24`）替换 `{placeholder}` 形式的占位符。

服务端组件（如 `app/print/resumes/[id]/page.tsx`）使用 `translate(locale, key, params)`（`apps/frontend/lib/i18n/server.ts:5-13`）。

类型推导的关键在于 `apps/frontend/lib/i18n/messages.ts:9`：

```
export type Messages = typeof en;
const allMessages: Record<Locale, Messages> = { en, es, zh, ja, pt };
```

⚠️ **构建中断陷阱：** 因为 `Messages` 是从 `en.json` 推导的严格结构类型，所有其它语言的 JSON 必须与 `en.json` 结构完全对齐。给 `en.json` 增删 key 后必须同步修改其余四个文件，否则 `npm run build` 会失败（见 `apps/frontend/CLAUDE.md:115-122`）。

2.4 添加新语言（实操步骤）

- 1. 复制 `en.json` 为新语言文件，例如 `apps/frontend/messages/de.json`，翻译每一项 value（保持 key 路径不变）。
- 2. 在 `apps/frontend/i18n/config.ts` 中追加 locale：

```
export const locales = ['en', 'es', 'zh', 'ja', 'pt', 'de'] as const;
export const localeNames: Record<Locale, string> = { ..., de: 'Deutsch' };
export const localeFlags: Record<Locale, string> = { ..., de: '🇩🇪' };
```

- 3. 在 `apps/frontend/lib/i18n/messages.ts` 中新增 import 与映射：

```
import de from '@messages/de.json';
const allMessages: Record<Locale, Messages> = { en, es, zh, ja, pt, de };
```

- 4. 在 `apps/backend/app/prompts/templates.py` 的 `LANGUAGE_NAMES` 与 `get_language_name()` 中追加语言代码到全称的映射，确保 LLM 能正确接收 `{output_language}`。
- 5. 运行 `npm run build` 验证所有 JSON 结构对齐（构建器会做类型检查）。

3. 自定义章节（Custom Sections）

3.1 章节类型

详见 `docs/agent/features/custom-sections.md:5-13`：

Type	描述	适用场景
<code>personalInfo</code>	特殊类型，固定在头部	姓名、联系方式
<code>text</code>	单一文本块	Summary、Objective、Statement
<code>itemList</code>	数组型条目	Experience、Projects、Publications
<code>stringList</code>	简单字符串数组	Skills、Languages、Hobbies

3.2 章节控制

每个非 `personalInfo` 章节的 header 处提供五个控件（`docs/agent/features/custom-sections.md:24-33`）：

- **可见性（Eye / EyeOff）：** 切换是否在 PDF 中显示。隐藏章节仍可在表单中编辑，仅在 PDF/预览中过滤（`getSortedSections` 与 `getAllSections` 的差别）。
- **上移/下移（ChevronUp/ChevronDown）：** 调整章节顺序。
- **重命名（Pencil）：** 就地编辑显示名。
- **删除（Trash）：** 默认隐藏，自定义章节可彻底删除。

3.3 数据结构

```
interface ResumeData {
  // ... 内建字段
  sectionMeta?: SectionMeta[]; // 顺序、显示名、可见性
  customSections?: Record<string, CustomSection>; // 用户自定义章节
}
```

章节元数据定义在 `apps/backend/app/schemas/models.py`（`SectionType / SectionMeta / CustomSection`），状态由 `apps/frontend/lib/utils/section-helpers.ts` 维护。

3.4 关键实现细节

- **隐藏视觉反馈**: 隐藏章节在表单里显示为虚线边框 + 60% 透明度, 并附带琥珀色 "Hidden from PDF" 徽章 (`docs/agent/features/custom-sections.md:36-41`)。
- **迁移兼容**: 旧数据通过懒加载归一化 (lazy normalization) 补齐 `SectionMeta`, 无需手工迁移。
- **不可变默认值**: 使用 `copy.deepcopy(DEFAULT_SECTION_META)` 避免共享可变引用 bug (见 `docs/agent/features/custom-sections.md:68`, 与 `apps/backend/CLAUDE.md:130-132` 强调一致)。

3.5 添加新章节 (步骤)

1. 在 `apps/frontend/components/builder/add-section-dialog.tsx` 的章节类型选项中暴露新类型 (如需)。
2. 编写对应的表单组件 (参考 `forms/` 目录下既有的 `generic-text-form.tsx` / `generic-item-form.tsx` / `generic-list-form.tsx`)。
3. 在 `apps/frontend/components/resume/dynamic-resume-section.tsx` 中处理该类型在 PDF 模板中的渲染。
4. 视情况扩展 `apps/backend/app/schemas/models.py` 的 `SectionType` 联合类型。

4. 简历模板系统

4.1 现有模板

模板类型定义在 `apps/frontend/lib/types/template-settings.ts` (`TemplateType`)。当前四种模板见 `docs/agent/features/resume-templates.md:5-13`:

Template	描述
<code>swiss-single</code>	单列, 最大内容密度, 1-2 页首选
<code>swiss-two-column</code>	65%/35% 双列, 左侧经历, 右侧技能
<code>modern</code>	单列 + 彩色 accent header, 可换主题色
<code>modern-two-column</code>	modern + 双列布局

四种模板文件位于 `apps/frontend/components/resume/`:

```
components/resume/
├─ index.ts
├─ resume-single-column.tsx
├─ resume-two-column.tsx
├─ resume-modern.tsx
├─ resume-modern-two-column.tsx
├─ dynamic-resume-section.tsx
├─ safe-html.tsx
├─ styles/
│   ├── _tokens.css
│   ├── _base.module.css
│   ├── swiss-single/...
│   ├── swiss-two-column/...
│   ├── modern/...
│   └─ modern-two-column/...
```

4.2 格式化控件

`TemplateSettings` 提供以下调参 (详见 `docs/agent/features/resume-templates.md:15-30`):

控件	范围	默认	作用
Margins	5-25mm	8mm	页面边距
Section Spacing	1-5	3	章节间距
Item Spacing	1-5	2	条目间距
Line Height	1-5	3	行高
Base Font Size	1-5	3	基础字号缩放（11-16px）
Header Scale	1-5	3	标题缩放
Header Font	serif / sans-serif / mono	serif	标题字体
Body Font	serif / sans-serif / mono	sans-serif	正文字体
Compact Mode	bool	false	0.6x 间距（仅间距）
Contact Icons	bool	false	联系信息加图标
Accent Color	blue/green/orange/red	blue	modern 系列 accent 颜色

4.3 CSS 变量机制

模板通过 CSS 自定义属性驱动排版，根样式位于 `apps/frontend/components/resume/styles/_tokens.css`：

```
--section-gap, --item-gap, --line-height
--font-size-base, --header-scale, --section-header-scale
--header-font, --body-font
--margin-top / bottom / left / right
--accent-primary, --accent-light
```

新模板必须使用 `baseStyles['resume-section']` / `baseStyles['resume-item-subtitle']` 等共享类（`docs/agent/features/resume-templates.md:54-55`）以响应设置变化。

4.4 添加新模板（步骤）

参考 `docs/agent/features/adding-resume-templates.md`：

1. 创建组件 `components/resume/resume-{name}.tsx`：

```
interface TemplateProps {
  resumeData: ResumeData;
  settings: TemplateSettings;
}
export function ResumeNewTemplate({ resumeData, settings }: TemplateProps) {
  return (
    <div className="resume-print">
      <header className="resume-section">
        <h1>{resumeData.personalInfo?.name}</h1>
      </header>
      {getSortedSections(resumeData).map(section => (
        <section key={section.id} className="resume-section">
          <h3 className="resume-section-title">{section.displayName}</h3>
          <div className="resume-items">...</div>
        </section>
      ))}
    </div>
  );
}
```

2. 导出：`components/resume/index.ts` 增补 `export { ResumeNewTemplate }`。
3. 注册到选择器：`components/builder/formatting-controls.tsx` 的 `TEMPLATES` 数组中追加新模板 id。
4. 添加缩略图：在选择器 UI 上提供预览。
5. 测试：跨多种 section 类型与 2 页以上内容验证 PDF 分页。

4.5 必需的 CSS 类

新模板必须使用以下类（Playwright 等待 `.resume-print`、`.resume-item` 不跨页，见 `docs/agent/design/pdf-template-guide.md:78-85`）：

```
.resume-print          /* 根容器, Playwright 等待 */
.resume-section        /* 章节容器 */
.resume-section-title  /* 章节标题 */
.resume-items          /* 条目容器 */
.resume-item           /* 单条目 (禁止分页) */
```

5. AI 富集 (Enrichment)

5.1 工作流

[docs/agent/features/enrichment.md:7-19](#) 描述了一个 5 步闭环：

1. 用户在 master resume 页面点击 **Enhance Resume**。
2. AI 分析简历，识别薄弱/泛化的描述。
3. 系统最多生成 6 个针对性强的问题（防止用户疲劳）。
4. 用户回答后，AI 生成新的 bullet points。
5. 新 bullet points **追加**到原内容之后（不替换原内容）。

5.2 API 接口

端点	说明
<code>POST /enrichment/analyze/{resume_id}</code>	分析简历并生成问题
<code>POST /enrichment/enhance</code>	根据用户答案生成增强描述
<code>POST /enrichment/apply/{resume_id}</code>	应用增强到简历

源码分布：

- 后端提示词：[apps/backend/app/prompts/enrichment.py](#) (`ANALYZE_RESUME_PROMPT` / `ENHANCE_DESCRIPTION_PROMPT` / `REGENERATE_ITEM_PROMPT` / `REGENERATE_SKILLS_PROMPT`)。
- 后端路由：[apps/backend/app/routers/enrichment.py](#)。
- 前端状态机：[apps/frontend/hooks/use-enrichment-wizard.ts](#)。
- 前端 UI：[apps/frontend/components/enrichment/](#)。

5.3 设计决策 ([docs/agent/features/enrichment.md:23-26](#))

- **最多 6 题**：权衡用户耐心与信息覆盖度。
- **叠加式增强**：原始 bullet 保留，新内容追加。
- **优先级**：AI 挑选 ROI 最高的问题。

6. JD 匹配 (JD Match)

6.1 触发条件

只有当一份简历是**针对特定 JD 量身定制**时，Builder 中才会出现 **JD MATCH** 标签。普通 master resume 不会暴露此视图 ([docs/agent/features/jd-match.md:9-13](#))。

6.2 视图结构

[docs/agent/features/jd-match.md:13-17](#) 描述的布局：

- **左面板**：原始 JD（只读）。
- **右面板**：简历视图，命中关键词高亮黄色。

6.3 匹配规则

[docs/agent/features/jd-match.md:21-25](#)：

- 关键词提取时过滤停用词。

- 大小写不敏感匹配。
- 统计总关键词、命中数、命中率。
- 命中率颜色编码：≥50% 绿、≥30% 黄、<30% 红。

6.4 关键实现

- 关键词提取：`apps/frontend/lib/utils/keyword-matcher.ts`。
- 视图组件：`apps/frontend/components/builder/jd-comparison-view.tsx`、`jd-display.tsx`、`highlighted-resume-view.tsx`。
- 后端接口：`apps/backend/app/routers/resumes.py` 中 `GET /{resume_id}/job-description` 用于取回当初 tailor 时使用的 JD。

7. Swiss International Style 设计系统

Resume Matcher 的所有 UI 必须遵循 Swiss International Style。完整规范位于 `docs/portable/swiss-design-system/`，本节摘录关键点。

7.1 核心原则

见 `docs/portable/swiss-design-system/README.md:21-28`：

1. **Grid-based layouts** — 数学精度而非直觉。
2. **Asymmetric balance** — 留白策略而非居中对称。
3. **Objective typography** — serif 标题、mono 元数据、sans 正文。
4. **Minimal ornamentation** — 硬边、无渐变、无圆角、无装饰图标。

一句话记忆：**square corners, hard shadows, pure colors, three-font hierarchy**。

7.2 设计 Token

颜色（`docs/portable/swiss-design-system/tokens.md:9-22`）

Name	Hex	用途
Canvas	<code>#F0F0E8</code>	页面背景（暖白，绝不用纯白）
Ink	<code>#000000</code>	文本、边框
Hyper Blue	<code>#1D4ED8</code>	链接、主操作、focus ring
Signal Green	<code>#15803D</code>	成功、确认、下载
Alert Orange	<code>#F97316</code>	警告
Alert Red	<code>#DC2626</code>	错误、销毁操作
Steel Grey	<code>#4B5563</code>	次级文字、禁用态

字体（`docs/portable/swiss-design-system/tokens.md:39-43`）

```
font-serif → 标题 (Georgia / Times)
font-sans  → 正文 (Inter / Helvetica / system-ui)
font-mono  → 元数据、标签 (SF Mono / Consolas)
```

字号（`docs/portable/swiss-design-system/tokens.md:60-66`）

```
xs(12)  sm(14)  base(16)  lg(18)  xl(20)  2xl(24)  3xl(30)  4xl(36)  5xl(48)
```

阴影（`docs/portable/swiss-design-system/tokens.md:96-106`）

只允许硬阴影（无 blur）：

```
shadow-[2px_2px_0px_0px_#000000] /* 按钮 */
shadow-[4px_4px_0px_0px_#000000] /* 卡片 */
shadow-[8px_8px_0px_0px_#000000] /* hero */
```

hover 行为：硬阴影元素 "按下去"，即 `hover:translate-y-[1px] hover:translate-x-[1px] hover:shadow-none` ([docs/portable/swiss-design-system/tokens.md:109-115](#))。

7.3 组件范例 ([docs/portable/swiss-design-system/components.md:9-26](#))

```
<button className="
  rounded-none
  border-2 border-black
  bg-blue-700 text-white
  px-4 py-2
  font-mono uppercase tracking-wider text-sm
  shadow-[2px_2px_0px_0px_#000000]
  hover:translate-y-[1px] hover:translate-x-[1px] hover:shadow-none
  transition-none
">
  Submit
</button>
```

按钮变体：Primary（蓝）/ Success（绿）/ Destructive（红）/ Warning（橙）/ Outline（透明）。每屏只有一个 Primary 按钮。

7.4 文本域 Enter 键 ([docs/portable/swiss-design-system/components.md:78-85](#))

模态框或拖拽卡片中嵌入的 textarea 必须阻止 Enter 冒泡：

```
const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
  if (e.key === 'Enter') e.stopPropagation();
};
```

8. 端到端用户流程图

下面是从打开站点到导出 PDF 的完整路径（结合 [apps/frontend/CLAUDE.md:14-32](#)）：

```

[浏览器] 打开 http://localhost:3000
|
▼
[/] (app/(default)/page.tsx, server) ← Landing
| Hero + 介绍
|
▼
用户点击 "Get Started"
|
▼
[/dashboard] (client) ← Resume 列表
| - 状态网格: master / tailored / failed
| - 操作: Upload, Delete, Rename, Retry
|
|
▼
[/builder] (client → components/builder/resume-builder.tsx)
| 1) 主简历编辑 (个人 / 摘要 / 经历 / 项目 / 教育 / 技能 ...)
| 2) 模板选择 + 格式化控件 (margin / font / accent)
| 3) AI Regenerate 单条目 / 单章节
| 4) 拖拽排序、自定义章节、增删可见性
|
| → AI Enrichment 向导 (点击 "Enhance Resume")
| analyze → 6 道题 → enhance → apply
|
|
▼
[/tailor] (client) ← 粘贴 JD
| 1) POST /resumes/improve/preview
| 2) 服务端执行: extract_job_keywords → skill_target_plan → diff → verify
| 3) 返回 diff, 用户在 diff modal 中确认
| 4) POST /resumes/improve/confirm (携带 preview_hash)
|
|
▼
[/resumes/{id}] (client) ← 单份简历查看
| - JD MATCH 标签 (仅 tailored 简历出现)
| - 左: 原始 JD; 右: 关键词高亮简历
| - Download PDF 按钮
|
|
▼
[后端] GET /resumes/{id}/pdf
| 1) 构造 URL: {FRONTEND_BASE_URL}/print/resumes/{id}?template=...
| 2) Playwright 启动 headless Chromium
| 3) 等待 .resume-print + document.fonts.ready
| 4) page.pdf(format=A4, print_background=True, margins=...)
| 5) 返回 PDF 字节流
|
|
▼
[浏览器] 文件下载
|
▼
(可选) 打开 /print/cover-letter/{id}, 再次走 Playwright 导出求职信 PDF

```

关键交互点

- **PDF 路由是 Server Component** (`app/print/resumes/{id}/page.tsx`)，从 `searchParams` 读取模板参数，通过 `API_BASE` 服务端直接调用后端获取数据。不要给 `print/*` 加 `'use client'`。
- **Margin 实际由 Playwright 应用** (`docs/agent/design/print-pdf-design-spec.md:35-39`)，HTML 里置零边距以保证多页一致。
- **Page Break** 由 `apps/frontend/components/preview/use-pagination.ts` 计算，确保 `.resume-item` 不跨页、章节标题不孤立。
- **健康检查** 仅访问内部后端 `127.0.0.1:8000/api/v1/health` (见 `Dockerfile:123-124`)。

9. 小结与下一步

至此你应该掌握了：

- i18n 的文件结构、客户端/服务端 API、添加新语言的 5 步。
- 自定义章节的四种类型与控制面板。
- 模板系统的 CSS 变量机制、添加新模板的清单。
- AI Enrichment 与 JD Match 的 API 与 UI 入口。
- Swiss 设计系统的色彩、字体、组件规范。
- 完整的端到端流程。

继续阅读 [10-部署与扩展.md](#) 了解如何部署、添加新 LLM provider / 模板 / 章节类型，以及性能调优。

10 - 部署与扩展

本章面向需要把 Resume Matcher 部署上线、调整环境配置、添加新 provider / 模板 / 语言的开发者。我们将逐项拆解本地开发、Docker 镜像、PDF 渲染流水线，并给出扩展点与性能调优建议。

1. 本地开发部署

1.1 后端

后端使用 `uv` 管理依赖（`pyproject.toml` 与 `requirements.txt`），Python 3.13+，核心栈：FastAPI、Pydantic v2、TinyDB、LiteLLM、markdown、Playwright。

```
cd apps/backend
uv sync                                # 创建 .venv 并安装依赖
uv run uvicorn app.main:app --reload --port 8000 # 开发服务器
uv run app                               # 等价于 app.main:main, 支持 HOST/PORT/RELOAD
uv run playwright install chromium       # 一次性: 下载 Chromium
```

API 文档自动暴露在 `http://localhost:8000/docs` (Swagger UI) 和 `/redoc`。

1.2 前端

```
cd apps/frontend
npm install
npm run dev      # next dev --turbo, :3000
npm run build    # next build (运行 tsc, i18n 结构错位时构建失败)
npm run start
npm run lint     # eslint .
npm run format  # prettier --write .
npm run test    # vitest run
```

前端不能直接 `fetch('http://localhost:8000')`。所有后端请求都走 `lib/api/`（`apps/frontend/CLAUDE.md:67-86`），通过 `next.config.ts` 的 `rewrites` 把 `/api/*` 代理到后端的 `BACKEND_ORIGIN`。不要创建 `app/api/` 路由，会覆盖代理。

1.3 开发期间的依赖关系

- 前端在 :3000 监听浏览器访问，把 `/api/*` 代理到 :8000。
- 后端在生成 PDF 时再把请求转回 `{FRONTEND_BASE_URL}/print/*`，由 Playwright 渲染（见 §4）。
- 因此必须同时启动前后端，否则 PDF 会拿到 503 错误（`apps/backend/app/pdf.py:218-226`）。

2. Docker 部署

2.1 镜像结构（`Dockerfile`）

Dockerfile 是多阶段构建：

Stage 1: frontend-builder（`Dockerfile:7-27`）

```
FROM node:22-bookworm AS frontend-builder
ARG NEXT_PUBLIC_API_URL=/
COPY apps/frontend/package*.json ./
RUN npm ci
COPY apps/frontend/ ./
RUN npm run build
```

`NEXT_PUBLIC_API_URL` 允许构建时自定义，默认 `/` 表示请求经由 Next.js rewrite 同源转发。`NEXT_TELEMETRY_DISABLED=1` 关闭遥测。

Stage 2: 最终镜像 (Dockerfile:32-127)

```
FROM python:3.13-slim-bookworm
```

- 安装系统级 Playwright 依赖：
`libnss3 / libnspr4 / libatk1.0-0 / libatk-bridge2.0-0 / libcups2 / libdrm2 / libxkbcommon0 / libxcomposite1 / libxdamage1 / libxf86` (Dockerfile:43-64)。
- 从 `frontend-builder` 复制 Node.js 运行库 (`/usr/local/bin/node`), 让 `server.js` 可以启动。
- 复制 `apps/backend` 并 `pip install .`。
- 复制前端 standalone 输出: `.next/standalone`、`/.next/static`、`/public` (Dockerfile:87-90)。
- 创建非 root 用户 `appuser:1000` 并 `chown` 整个 `/app` (Dockerfile:104-108), 提升安全基线。
- 以 `appuser` 身份执行 `python -m playwright install chromium` (Dockerfile:111), 保证浏览器落在正确路径。
- 暴露 3000、声明 `/app/backend/data` 为 volume、健康检查 `127.0.0.1:8000/api/v1/health` (Dockerfile:114-127)。

健康检查走的是容器内的 :8000, 不依赖宿主机端口映射。

2.2 docker-compose.yml 解析

```
services:
  resume-matcher:
    image: ghcr.io/srbhr/resume-matcher
    build: .
    container_name: resume-matcher
    ports:
      - "${PORT:-3000}:3000"
    volumes:
      - resume-data:/app/backend/data
    environment:
      - FRONTEND_BASE_URL=${FRONTEND_BASE_URL:-http://localhost:3000}
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
      - LOG_LLM=${LOG_LLM:-WARNING}
      - LLM_PROVIDER=${LLM_PROVIDER:-openai}
      - LLM_MODEL=${LLM_MODEL:-}
      - LLM_API_KEY=${LLM_API_KEY:-}
      - LLM_API_BASE=${LLM_API_BASE:-}
    restart: unless-stopped

volumes:
  resume-data:
    driver: local
```

要点 (docker-compose.yml:1-42):

- 单端口暴露: 宿主机 `${PORT:-3000}` → 容器 :3000; 后端 :8000 仅在容器内部可访问, 由前端的 `rewrites` 转发。
- 数据持久化: `resume-data` 命名卷挂到 `/app/backend/data`, 数据库与上传文件不会因容器重建丢失。
- 环境变量: 所有 LLM 与日志相关变量都从宿主机 `.env` 注入, 未提供时取默认值。
- Secret File 模式: 可使用 `LLM_API_KEY_FILE` 指向挂载的 secret, 行为类似 Postgres 的 `*_FILE` 约定 (docker-compose.yml:25-27), 由 `docker/start.sh:65-88` 解析。
- 注释掉的 CORS: 默认会自动把 `FRONTEND_BASE_URL` 加入白名单。

2.3 启动脚本 (docker/start.sh)

启动脚本完成三件事:

1. 环境解析: 把 `LOG_LEVEL` / `LLM_PROVIDER` / `LLM_API_KEY[_FILE]` 等转换为真正的环境变量并校验 (docker/start.sh:65-88)。
2. 日志级别归一化: `normalize_log_level` 拒绝非法值并打印警告 (docker/start.sh:90-104)。
3. 进程编排:
 - 后端: uvicorn 监听 :8000, 30 秒内未就绪则报错退出。
 - 前端: `node server.js "$@"` 监听 :3000。
 - 任一进程退出即触发 `cleanup` 杀掉另一个 (docker/start.sh:117-131, 203-256)。

启动时还会按需安装 Chromium (docker/start.sh:191-201), 但镜像里已经预装, 正常情况只走快速路径。

2.4 启动方式

```
# 1) 创建 .env
cat > .env <<'EOF'
LLM_PROVIDER=openai
LLM_MODEL=gpt-5-nano-2025-08-07
LLM_API_KEY=sk-your-key
FRONTEND_BASE_URL=http://localhost:3000
LOG_LEVEL=INFO
EOF

# 2) 启动
docker compose up -d
docker compose logs -f

# 3) 升级 / 重建
docker compose build --pull
docker compose up -d
```

3. 生产环境部署注意事项

3.1 反向代理与域名

`FRONTEND_BASE_URL` 必须写成浏览器可访问的公网 / 内网 URL，因为 Playwright 通过这个 URL 抓取 `/print/*`。如果前端挂在 `https://resume.example.com`，设置：

```
FRONTEND_BASE_URL=https://resume.example.com
```

同时确认后端的 `CORS_ORIGINS` 包含该域名（默认值在 `apps/backend/.env.example` 中）。

3.2 HTTPS 与 Cookie

- 反向代理 (Nginx / Caddy / Traefik) 终止 TLS，前端始终走 443。
- 如部署在内网并启用 `secure` cookie，确保代理转发 `X-Forwarded-Proto: https`。

3.3 数据与备份

- 数据库与上传都落在 `/app/backend/data`，对应 `resume-data` 卷。定期
`docker run --rm -v resume-data:/data -v ${pwd}:/backup alpine tar czf /backup/resume-data.tgz /data`。
- `.env` 不要提交到仓库，建议用 secret manager。

3.4 单 worker 假设

后端的 `asyncio.Lock` 与 `config_cache` 都假设单 `uvicorn worker`。生产部署请使用单进程 (`--workers 1`)，如需横向扩展，需要先改造 `config_cache` 与 master lock (见 `apps/backend/CLAUDE.md:148`)。

3.5 健康检查

`Dockerfile:123-124` 默认指向 `/api/v1/health`（不调用 LLM，仅 liveness）。如需更全面的状态，请额外探活 `/api/v1/status`（包含 LLM health 与 DB 统计）。

3.6 日志与告警

- `LOG_LEVEL` 控制应用日志；`LOG_LLM` 控制 LiteLLM 详细输出。
- 生产建议 `LOG_LEVEL=INFO`、`LOG_LLM=WARNING`。
- 切勿把 `LOG_LLM=DEBUG` 长期打开——会打印明文 API key (`docker/start.sh:176-178` 提示)。

4. 环境变量配置 (.env 模板)

`apps/backend/.env.example` 给出了完整列表。生产用模板：

```
# ===== LLM =====
LLM_PROVIDER=openai          # openai / anthropic / openrouter / gemini / deepseek / ollama / openai_compatible
LLM_MODEL=gpt-5-nano-2025-08-07
LLM_API_KEY=sk-your-api-key
# LLM_API_BASE=              # Ollama / 自托管需要
# REASONING_EFFORT=          # minimal | low | medium | high, 留空兼容

# ===== Server =====
HOST=0.0.0.0
PORT=8000
RELOAD=false                # 生产保持 false
LOG_LEVEL=INFO
LOG_LLM=WARNING

# ===== Frontend / CORS =====
FRONTEND_BASE_URL=https://resume.example.com
CORS_ORIGINS=["https://resume.example.com"]

# ===== Docker 注入 (与 docker-compose.yml 对应) =====
PORT=3000                    # 宿主机端口
# LLM_API_KEY_FILE=/run/secrets/llm_api_key # 容器内 secret 路径
```

Docker Secret 模式： 在 `docker-compose.yml` 启用注释的 `secrets:` 段 (`docker-compose.yml:36-38`)，把密钥挂到 `/run/secrets/llm_api_key`，然后设置 `LLM_API_KEY_FILE=/run/secrets/llm_api_key`。`docker/start.sh:65-88` 的 `file_env()` 会读取并 export。

5. PDF 渲染流水线

PDF 由后端驱动，前端是模板源，后端是渲染器。流水线如 `docs/agent/design/pdf-template-guide.md:6-15` 所述。

5.1 入口

`apps/backend/app/pdf.py:251-310` 的 `render_resume_pdf()`：

```
await page.goto(url, wait_until="networkidle")
await page.wait_for_selector(selector)          # 默认 ".resume-print"
await page.evaluate("document.fonts.ready")
return await page.pdf(
    format=pdf_format,                          # A4 / Letter
    print_background=True,
    margin=pdf_margins,                          # 来自 *_BASE_URL 的 host 决定
)
```

- `url` 由后端构造：`{FRONTEND_BASE_URL}/print/resumes/{id}?template=...&pageSize=...&lang=...&...`。
- 等待选择器确保 React 树已挂载；`document.fonts.ready` 等待 web font 加载。
- `margins` 来自 `TemplateSettings.margins` (5-25mm)，通过 Playwright 注入 PDF 而不是 HTML 内边距，**保证多页一致** (`docs/agent/design/print-pdf-design-spec.md:35-39`)。

5.2 启动路径与回退

`apps/backend/app/pdf.py:33-51` 懒加载浏览器，`init_pdf_renderer` 使用 `asyncio.Lock` 防并发初始化。如果主事件循环不支持子进程 (Windows 旧版 `asyncio`)，会退化到 `asyncio.to_thread` 在新 loop 中跑同步实现 (`apps/backend/app/pdf.py:160-206, 280-302`)。

5.3 浏览器定位策略

`apps/backend/app/pdf.py:73-111` 的 `_find_chromium_executable` 在以下位置查找系统 Chrome/Chromium/Edge (按平台分支)：

- Windows: `Program Files\Google\Chrome\Application\chrome.exe`、`msedge.exe`。
- macOS: `/Applications/Google Chrome.app/...`、`/Applications/Chromium.app/...`。
- Linux: `/usr/bin/google-chrome`、`chromium`、`microsoft-edge`、Snap、Flatpak 等。

主路径失败时 fallback 到系统安装的浏览器 (`apps/backend/app/pdf.py:114-126`)。

5.4 错误分类

`apps/backend/app/pdf.py:209-227` 对 Playwright 抛错做友好分类：

- `Executable doesn't exist` → 提示运行 `python -m playwright install chromium`。
- `net::ERR_CONNECTION_REFUSED` → 提示检查前端是否启动、`FRONTEND_BASE_URL` 是否正确。
- 其它错误透传为 `PDFRenderError("PDF rendering failed: ...")`。

5.5 打印路由

路由	入口文件	关键选择器
<code>/print/resumes/[id]</code>	<code>apps/frontend/app/print/resumes/[id]/page.tsx:1</code>	<code>.resume-print</code>
<code>/print/cover-letter/[id]</code>	<code>apps/frontend/app/print/cover-letter/[id]/page.t</code>	<code>.cover-letter-print</code>

二者均为 Server Component，从 `searchParams` 读取 `template` / `pageSize` / `lang` 等参数，再通过 `lib/i18n/server.ts:translate()` 本地化标题。

关键 CSS 规则（`docs/agent/design/pdf-template-guide.md:39-50`）：`@media print` 必须把 `body *` 置为 `visibility: hidden`，再把 `.resume-print *` / `.cover-letter-print *` 置为 `visible`，否则 PDF 会全白。

5.6 分页逻辑

`apps/frontend/components/preview/use-pagination.ts:43-175` 计算分页：

- 用 `ResizeObserver` + `MutationObserver` 监听内容变化，150 ms debounce 重算。
- 找出所有 `.resume-item` 与 `[data-no-break]`，构建不可分页集合。
- 章节标题自动扩展到第一个内容元素，避免孤立。
- 算法约束：分页点不能把一个 item 拆开，且至少占满 50% 页面（`use-pagination.ts:130-144`）。

5.7 排版规则

来自 `docs/agent/design/print-pdf-design-spec.md:41-46`：

- `.resume-item` 永不跨页。
- 章节标题不孤立。
- 单页至少 50% 填充后再换页。

6. 添加新 LLM Provider

`apps/backend/app/llm.py` 通过 LiteLLM 抽象多 provider。新增 provider 一般无需改 LLM 层，只需要在 Settings 页面 / 环境变量中配置即可。但若需要把 provider 注册为 **官方可选**项，可按以下步骤：

1. **确认 LiteLLM 支持**：在 LiteLLM 文档中查找 provider 标识（如 `bedrock`、`cohere`）。
2. 在 `apps/backend/app/config.py` 中将 provider 名字加入 `LLMProvider` 字面量联合（如有显式枚举）。
3. 在 `apps/backend/app/llm.py:get_model_name()` 中添加 provider → LiteLLM model prefix 映射（如 `bedrock` → `bedrock/...`）。
4. **如果走非标准 base URL**：在 `_normalize_api_base()` 中处理尾随 `/v1` 重复问题。
5. **添加默认模型**（可选）：在 `apps/frontend/lib/api/config.ts:PROVIDER_INFO` 中追加。
6. **更新 `docker-compose.yml`** 注释（`docker-compose.yml:22-23`）与 `apps/backend/.env.example`，方便用户发现。
7. **测试**：使用 `POST /api/v1/config/llm-test`（`apps/backend/app/routers/config.py`）做活体检。

特殊情形：

- `openai_compatible`（`llama.cpp` / `vLLM` / `LM Studio`）：在 `apps/backend/app/llm.py` 故意忽略环境变量 `LLM_API_KEY` fallback，避免付费 key 误传到本地服务（`apps/backend/CLAUDE.md:103`）。
- **Ollama**：在 Docker 部署中使用 `host.docker.internal:11434` 访问宿主机 Ollama（`docker-compose.yml:29-30`）。
- **支持 `reasoning_effort`** 的模型：LiteLLM 会在不支持的 provider 上自动 drop（`litellm.drop_params = True`，`apps/backend/CLAUDE.md:96`）。

7. 添加新简历模板

完整流程见 `09-前端特性与流程.md §4.4`，要点回顾：

1. `apps/frontend/components/resume/resume-{name}.tsx` 实现 `TemplateProps` 接口。

2. 在 `apps/frontend/components/resume/index.ts` 导出。
 3. 在 `apps/frontend/lib/types/template-settings.ts` 的 `TemplateType` 联合里追加新 id。
 4. 在 `apps/frontend/components/builder/formatting-controls.tsx` 的 `TEMPLATES` 数组中注册。
 5. 在 `apps/frontend/app/print/resumes/[id]/page.tsx:132-142` 的 `parseTemplate()` 中放行新 id。
 6. 后端 `apps/backend/app/routers/resumes.py` 在生成 PDF 时会读 `accentColor`，modern 系需要正确传递。
 7. 添加 `styles/{template}/` 下的 CSS module 并复用 `_base.module.css`。
 8. 跨多 section 类型与 2 页以上内容验证。
-

8. 添加新 UI 语言

完整流程见 `09-前端特性与流程.md §2.4`，步骤摘要：

1. `apps/frontend/messages/{code}.json` 翻译并保证结构与 `en.json` 严格一致。
 2. `apps/frontend/i18n/config.ts` 注册 locale、本地名、旗帜。
 3. `apps/frontend/lib/i18n/messages.ts` import 并加入 `allMessages`。
 4. `apps/backend/app/prompts/templates.py` 更新 `LANGUAGE_NAMES` / `get_language_name()`。
 5. `npm run build` 验证类型推导通过。
 6. Settings 页面会自动显示新选项（`LanguageProvider` 读取 `locales`）。
-

9. 自定义章节步骤

完整流程见 `09-前端特性与流程.md §3.5`：

1. 在 `apps/frontend/components/builder/add-section-dialog.tsx` 中暴露新类型。
 2. 在 `apps/frontend/components/builder/forms/` 中实现对应表单组件。
 3. 在 `apps/frontend/components/resume/dynamic-resume-section.tsx` 中处理 PDF 模板渲染。
 4. 视情况扩展 `apps/backend/app/schemas/models.py` 的 `SectionType` 联合类型。
 5. 验证：Builder 中能增删/排序/隐藏，PDF 中能正确显示。
-

10. 常见问题排查

症状	排查要点
PDF 导出空白	检查 <code>globals.css</code> 是否包含 <code>@media print</code> 的 <code>.resume-print</code> 可见性规则 (<code>docs/agent/design/pdf-template-guide.md:39-50</code>)
PDF 报 <code>ERR_CONNECTION_REFUSED</code>	确认前端在 <code>\${FRONTEND_BASE_URL}</code> 上启动；Docker 内确认 <code>FRONTEND_BASE_URL</code> 与暴露端口一致 (<code>apps/backend/app/pdf.py:218-226</code>)
<code>next build</code> 失败但 <code>next dev</code> 正常	必有 locale JSON 缺失 key，类型推导报错 (<code>apps/frontend/CLAUDE.md:115-122</code>)
<code>playwright install</code> 失败	镜像内已预装，本地若失败可手动 <code>uv run playwright install chromium</code> (<code>apps/backend/CLAUDE.md:30</code>)
Docker 内 LLM 调不通	Ollama 用 <code>host.docker.internal</code> (<code>docker-compose.yml:29-30</code>)；检查 <code>LLM_API_KEY</code> 是否被 <code>LLM_API_KEY_FILE</code> 覆盖 (<code>docker/start.sh:69-72</code>)
日志出现明文 key	<code>LOG_LLM=DEBUG</code> 会打印。生产请保持 <code>WARNING</code> (<code>docker/start.sh:176-178</code>)
API 返回 401/403	确认 <code>LLM_API_KEY</code> 是否被后端从 <code>config.json</code> 覆盖；写 config 后需 <code>invalidate_config_cache()</code> (<code>apps/backend/CLAUDE.md:144</code>)
改善后日期缺月	LLM 丢精度是已知问题，后端用 <code>restore_dates_from_markdown</code> + <code>_restore_original_dates</code> 修复 (<code>apps/backend/CLAUDE.md:147</code>)
Improve 返回 400	必须先调用 <code>/improve/preview</code> 获取 <code>preview_hash</code> ， <code>/improve/confirm</code> 才会接受 (<code>apps/backend/CLAUDE.md:151</code>)
容器权限报错	镜像以 <code>appuser:1000</code> 运行，宿主挂载卷需 <code>chown -R 1000:1000</code> 或使用命名卷 (<code>docker-compose.yml:7-9</code>)
主语种选择器无新语言	检查 <code>i18n/config.ts</code> 的 <code>locales</code> 数组 (<code>apps/frontend/i18n/config.ts:5</code>)

11. 性能调优建议

11.1 后端

- **超时：** `asyncio.wait_for` 默认 240s (`apps/backend/CLAUDE.md:95`)。如使用更慢的本地模型，可提高 `LLM_API_BASE` 调优 `get_safe_max_tokens`。
- **JSON 模式：** `complete_json()` 自动 JSON-mode→prompt fallback。给 provider 加上 `num_retries=3` (已在 `get_router` 中默认配置， `apps/backend/CLAUDE.md:91`)。
- **缓存：** `config_cache` TTL 5 分钟；如需立刻生效调用 `invalidate_config_cache()` (写 `config.json` 路由自动做)。
- **PDF 并发：** `_init_lock` 保证单浏览器实例多请求复用。如需更高并发可考虑多 worker + 多浏览器，但要注意 `/app/backend/data` 是文件存储，并发写需自行加锁 (推荐外置 Postgres)。
- **Prompt 长度：** `PARSE_RESUME_PROMPT` 嵌入整个 schema 体积较大 (`apps/backend/CLAUDE.md:79-80`)。如不需特定字段，可在 `RESUME_SCHEMA_EXAMPLE` 中裁剪。

11.2 前端

- **包体优化：** `lucide-react` 用深路径 import (如 `lucide-react/dist/esm/icons/x`)，并由 `next.config.ts` 的 `optimizePackageImports` 进一步摇树 (`apps/frontend/CLAUDE.md:178-180`)。
- **超时：** `apiFetch` 默认 240 s (`apps/frontend/CLAUDE.md:84-85`)，与后端匹配。
- **打印路由：** `print/*` 保持 Server Component 避免 hydration 成本 (`apps/frontend/CLAUDE.md:182-183`)。
- **i18n 大小：** 5 个语言 × ~35 KB JSON 全部静态 import，初始 bundle 不可避免。若需按需加载，改成 `next/dynamic({ ssr: false })` 加载特定语言，但会牺牲 SSR。
- **预渲染：** `PaginatedPreview` 用隐藏的 `measurementRef` + `ResizeObserver` + `MutationObserver` (`use-pagination.ts:188-218`) 做客户端分页，权衡：避免 SSR 时算不准，但内容频繁变更会触发重算 (150 ms debounce 已缓解)。

11.3 Docker

- **多阶段构建** 显著减小最终镜像体积。

- **Playwright 系统依赖** 已全部内置，无需 `--with-deps`。
- **Chromium 冷启动**：容器内预装到 `/home/appuser/.cache/ms-playwright` (`docker/start.sh:193`)，首次启动即可用。

11.4 观测

- `/api/v1/status` 提供 LLM health (30 min 缓存) + DB 统计 (5 min 缓存)，由 `StatusCacheProvider` 同步到前端 (`apps/frontend/CLAUDE.md:96-98`)。
 - 应用日志通过 `LOG_LEVEL` 控制；LiteLLM 通过 `LOG_LLM`。
 - 建议把容器 stdout 接到 Loki/CloudWatch/GCP Logging，匹配 alert 规则（如 PDF 错误率 > 1%、健康检查失败）。
-

12. 小结

本章覆盖了：

- 本地开发 + Docker 单端口部署。
- 反向代理、Secret 管理、备份等生产最佳实践。
- 环境变量模板与 Docker Secret 模式。
- PDF 渲染流水线的源码级剖析与排错。
- 扩展点：LLM provider / 模板 / UI 语言 / 章节。
- 性能调优与可观测建议。

结合 `09-前端特性与流程.md`，你已经具备独立部署并按需扩展 Resume Matcher 的完整知识。后续可阅读 `docs/agent/` 下的 `backend-architecture.md` / `frontend-architecture.md` / `llm-integration.md` / `prompt-workflow-design.md` 等专题文档深入某个模块。